CIT4001 SOFTWARE IMPLEMENTATION AND TESTING

Indexed programme source listing

PracticeChart EHR

University of Technology, Jamaica

Group 2

This document describes the implemented course application and the evidence recorded for it. It uses fictional patient data. Human training and user acceptance remain pending.

Public course copy. Student identification numbers and private course instruments are excluded.

Prepared for group review. This is not a record of clinical approval, a live pilot, completed training, or completed acceptance testing.

Source index

This listing includes authored application code, operator scripts, tests, migrations, and project configuration. Third-party dependencies, generated bundles, credentials, uploads, and private evidence are excluded.

1. migrations/0001_practicechart.sql
2. migrations/0002_record_number_and_date_guards.sql
3. scripts/backup.ts
4. scripts/build-documents.py
5. scripts/cloud-smoke.ts
6. scripts/cold-performance.ts
7. scripts/course-analysis.ts
8. scripts/diagram-details.ts
9. scripts/grant-runtime.ts
10. scripts/local-setup.ts
11. scripts/migrate.ts
12. scripts/performance.ts
13. scripts/recovery-lib.ts
14. scripts/refresh-sample.ts
15. scripts/regions.py
16. scripts/report-content.ts
17. scripts/restore-test.ts
18. scripts/seed.ts
19. scripts/test-run.ts
20. src/app/api/[...path]/route.ts
21. src/app/app/[[...path]]/page.tsx
22. src/app/globals.css
23. src/app/layout.tsx
24. src/app/login/page.tsx
25. src/app/page.tsx
26. src/app/print/[type]/page.tsx
27. src/components/print-document.tsx
28. src/components/workspace.tsx
29. src/lib/auth.ts

30. src/lib/db.ts
31. src/lib/schema.ts
32. src/lib/service.ts
33. src/lib/storage.ts
34. src/lib/validation.ts
35. src/workflows/daily.ts
36. tests/browser/workflows.spec.ts
37. tests/integration.test.ts
38. tests/recovery.test.ts
39. tests/validation.test.ts
40. package.json
41. tsconfig.json
42. next.config.ts
43. playwright.config.ts
44. vitest.config.ts
45. docker-compose.yml
46. vercel.json

migrations/0001_practicechart.sql

```
CREATE EXTENSION IF NOT EXISTS btree_gist;
CREATE SEQUENCE mrn_seq;
CREATE SEQUENCE invoice_seq;
CREATE TABLE users(id uuid PRIMARY KEY DEFAULT gen_random_uuid(), username text NOT NULL UNIQUE, name text NOT NULL, role text NOT NULL CHECK(role IN ('doctor','front_desk')), password_hash text NOT NULL, active boolean NOT NULL DEFAULT true, protected boolean NOT NULL DEFAULT false, version integer NOT NULL DEFAULT 1, created_at timestamptz NOT NULL DEFAULT now());
CREATE TABLE sessions(id text PRIMARY KEY, user_id uuid NOT NULL REFERENCES users(id), expires_at timestamptz NOT NULL, created_at timestamptz NOT NULL DEFAULT now());
CREATE INDEX sessions_user_idx ON sessions(user_id);
CREATE TABLE login_attempts(key text PRIMARY KEY, count integer NOT NULL DEFAULT 1, expires_at timestamptz NOT NULL);
CREATE TABLE patients(id uuid PRIMARY KEY DEFAULT gen_random_uuid(), mrn text NOT NULL UNIQUE DEFAULT ('PC-' || lpad(nextval('mrn_seq')::text,6,'0')), first_name text NOT NULL,last_name text NOT NULL,dob date NOT NULL CHECK(dob <= CURRENT_DATE), phone text NOT NULL,address text NOT NULL,gender text NOT NULL DEFAULT 'Not stated',emergency_contact text NOT NULL DEFAULT '',insurance text NOT NULL DEFAULT '',provisional_reason text NOT NULL DEFAULT '',merged_into uuid REFERENCES patients(id),archived boolean NOT NULL DEFAULT false,version integer NOT NULL DEFAULT 1,created_at timestamptz NOT NULL DEFAULT now(), CHECK(merged_into IS NULL OR merged_into<>id));
CREATE INDEX patient_name_idx ON patients(lower(last_name),lower(first_name));
CREATE TABLE doctor_hours(doctor_id uuid NOT NULL REFERENCES users(id),weekday integer NOT NULL CHECK(weekday BETWEEN 1 AND 7),start_min integer NOT NULL CHECK(start_min>=0),end_min integer NOT NULL CHECK(end_min<=1440 AND end_min>start_min),PRIMARY KEY(doctor_id,weekday));
CREATE TABLE appointments(id uuid PRIMARY KEY DEFAULT gen_random_uuid(),patient_id uuid NOT NULL REFERENCES patients(id),doctor_id uuid NOT NULL REFERENCES users(id), start_at timestamptz NOT NULL,end_at timestamptz NOT NULL,status text NOT NULL DEFAULT 'confirmed' CHECK(status IN ('confirmed','arrived','in_consultation','completed','cancelled')),version integer NOT NULL DEFAULT 1,created_at timestamptz NOT NULL DEFAULT now(),CHECK(end_at>start_at),CHECK(end_at-start_at<=interval '4 hours'),EXCLUDE USING gist(doctor_id WITH =,tstzrange(start_at,end_at,'[)') WITH &&) WHERE(status IN ('confirmed','arrived','in_consultation')),EXCLUDE USING gist(patient_id WITH =,tstzrange(start_at,end_at,'[)') WITH &&) WHERE(status IN ('confirmed','arrived','in_consultation')));
CREATE INDEX appointments_date_idx ON appointments(start_at);
CREATE TABLE schedule_blocks(id uuid PRIMARY KEY DEFAULT gen_random_uuid(),doctor_id uuid NOT NULL REFERENCES users(id),start_at timestamptz NOT NULL,end_at timestamptz NOT NULL,CHECK(end_at>start_at));
CREATE TABLE visits(id uuid PRIMARY KEY DEFAULT gen_random_uuid(),patient_id uuid NOT NULL REFERENCES patients(id),doctor_id uuid NOT NULL REFERENCES users(id),appointment_id uuid UNIQUE REFERENCES appointments(id),status text NOT NULL DEFAULT 'open' CHECK(status IN('open','ready','filed')),revision integer NOT NULL DEFAULT 0,visit_at timestamptz NOT NULL DEFAULT now(),filed_at timestamptz);
CREATE TABLE clinical_revisions(id uuid PRIMARY KEY DEFAULT gen_random_uuid(),visit_id uuid NOT NULL REFERENCES visits(id),revision integer NOT NULL,author_id uuid NOT NULL REFERENCES users(id),state text NOT NULL CHECK(state IN('draft','final')),content jsonb NOT NULL,reason text NOT NULL DEFAULT '',created_at timestamptz NOT NULL DEFAULT now(),UNIQUE(visit_id,revision));
CREATE TABLE prescriptions(id uuid PRIMARY KEY DEFAULT gen_random_uuid(),visit_id uuid NOT NULL REFERENCES visits(id),revision_id uuid NOT NULL REFERENCES clinical_revisions(id),patient_id uuid NOT NULL REFERENCES patients(id),content jsonb NOT NULL,created_at timestamptz NOT NULL DEFAULT now());
CREATE TABLE documents(id uuid PRIMARY KEY DEFAULT gen_random_uuid(),patient_id uuid NOT NULL REFERENCES patients(id),title text NOT NULL,kind text NOT NULL CHECK(kind IN('Lab report','Diagnostic report')),pathname text NOT NULL UNIQUE,mime text NOT NULL CHECK(mime IN('application/pdf','image/png','image/jpeg')),size integer NOT NULL CHECK(size BETWEEN 1 AND 4194304),sha256 text NOT NULL,replaces_id uuid REFERENCES documents(id),author_id uuid NOT NULL REFERENCES users(id),created_at timestamptz NOT NULL DEFAULT now());
CREATE TABLE invoices(id uuid PRIMARY KEY DEFAULT gen_random_uuid(),number text NOT NULL UNIQUE DEFAULT ('INV-'||lpad(nextval('invoice_seq')::text,6,'0')),patient_id uuid NOT NULL REFERENCES patients(id),lines jsonb NOT NULL,total_cents integer NOT NULL CHECK(total_cents>=0),status text NOT NULL DEFAULT 'draft' CHECK(status IN('draft','issued','void')),version integer NOT NULL DEFAULT 1,reason text NOT NULL DEFAULT '',created_at timestamptz NOT NULL DEFAULT now());
CREATE TABLE audit_events(id uuid PRIMARY KEY DEFAULT gen_random_uuid(),actor_id uuid REFERENCES users(id),action text NOT NULL,entity text NOT NULL,entity_id text NOT NULL DEFAULT '',metadata jsonb NOT NULL DEFAULT '{}',created_at timestamptz NOT NULL DEFAULT now());
CREATE INDEX audit_date_idx ON audit_events(created_at DESC);
CREATE TABLE reminders(id uuid PRIMARY KEY DEFAULT gen_random_uuid(),appointment_id uuid NOT NULL REFERENCES appointments(id),due_date date NOT NULL,UNIQUE(appointment_id,due_date));
CREATE TABLE backup_runs(id uuid PRIMARY KEY DEFAULT gen_random_uuid(),status text NOT NULL,pathname text,created_at timestamptz NOT NULL DEFAULT now());
CREATE FUNCTION immutable_record() RETURNS trigger LANGUAGE plpgsql AS $$BEGIN RAISE EXCEPTION 'This history is append only' USING ERRCODE='23514'; END$$;
CREATE TRIGGER revisions_immutable BEFORE UPDATE OR DELETE ON clinical_revisions FOR EACH ROW EXECUTE FUNCTION immutable_record();
CREATE TRIGGER audit_immutable BEFORE UPDATE OR DELETE ON audit_events FOR EACH ROW EXECUTE FUNCTION immutable_record();
CREATE TRIGGER prescriptions_immutable BEFORE UPDATE OR DELETE ON prescriptions FOR EACH ROW EXECUTE FUNCTION immutable_record();
CREATE FUNCTION guard_doctor() RETURNS trigger LANGUAGE plpgsql AS $$ BEGIN
 PERFORM pg_advisory_xact_lock(4001001);
```

```
 IF OLD.role='doctor' AND OLD.active AND (NOT NEW.active OR NEW.role<>'doctor') AND NOT EXISTS(SELECT 1 FROM users WHERE role='doctor' AND active AND id<>OLD.id) THEN RAISE EXCEPTION 'Keep at least one active doctor' USING ERRCODE='23514'; END IF;
 RETURN NEW; END$$;
CREATE TRIGGER doctor_guard BEFORE UPDATE ON users FOR EACH ROW EXECUTE FUNCTION guard_doctor();
CREATE FUNCTION validate_booking() RETURNS trigger LANGUAGE plpgsql AS $$DECLARE local_start timestamp;local_end timestamp; BEGIN
 PERFORM 1 FROM users WHERE id=NEW.doctor_id AND role='doctor' AND active FOR UPDATE;
 IF NOT FOUND THEN RAISE EXCEPTION 'Doctor is not available' USING ERRCODE='23514'; END IF;
 IF NEW.status IN('confirmed','arrived','in_consultation') THEN
 local_start:=NEW.start_at AT TIME ZONE 'America/Jamaica';local_end:=NEW.end_at AT TIME ZONE 'America/Jamaica';
 IF local_start::date<>local_end::date OR NOT EXISTS(SELECT 1 FROM doctor_hours WHERE doctor_id=NEW.doctor_id AND weekday=extract(isodow FROM local_start) AND extract(epoch FROM local_start::time)/60>=start_min AND extract(epoch FROM local_end::time)/60<=end_min) THEN RAISE EXCEPTION 'Outside doctor hours' USING ERRCODE='23514'; END IF;
 IF EXISTS(SELECT 1 FROM schedule_blocks WHERE doctor_id=NEW.doctor_id AND tstzrange(start_at,end_at,'[)') && tstzrange(NEW.start_at,NEW.end_at,'[)')) THEN RAISE EXCEPTION 'Doctor time is blocked' USING ERRCODE='23514'; END IF;
 END IF;RETURN NEW;END$$;
CREATE TRIGGER booking_hours BEFORE INSERT OR UPDATE ON appointments FOR EACH ROW EXECUTE FUNCTION validate_booking();
CREATE FUNCTION validate_block() RETURNS trigger LANGUAGE plpgsql AS $$BEGIN
 PERFORM 1 FROM users WHERE id=NEW.doctor_id FOR UPDATE;
 IF EXISTS(SELECT 1 FROM appointments WHERE doctor_id=NEW.doctor_id AND status IN('confirmed','arrived','in_consultation') AND tstzrange(start_at,end_at,'[)') && tstzrange(NEW.start_at,NEW.end_at,'[)')) THEN RAISE EXCEPTION 'This block conflicts with an appointment' USING ERRCODE='23514'; END IF;RETURN NEW;END$$;
CREATE TRIGGER block_guard BEFORE INSERT OR UPDATE ON schedule_blocks FOR EACH ROW EXECUTE FUNCTION validate_block();
```

migrations/0002_record_number_and_date_guards.sql

```
CREATE FUNCTION next_record_number(prefix text, sequence_name regclass) RETURNS text LANGUAGE plpgsql AS $$
DECLARE number_text text;
BEGIN
  number_text := nextval(sequence_name)::text;
  RETURN prefix || lpad(number_text, greatest(6, length(number_text)), '0');
END$$;
ALTER TABLE patients ALTER COLUMN mrn SET DEFAULT next_record_number('PC-', 'mrn_seq');
ALTER TABLE invoices ALTER COLUMN number SET DEFAULT next_record_number('INV-', 'invoice_seq');
ALTER TABLE patients DROP CONSTRAINT patients_dob_check;
ALTER TABLE patients ADD CONSTRAINT patient_dob_clinic_day CHECK(dob <= (CURRENT_TIMESTAMP AT TIME ZONE 'America/Jamaica')::date);
```

scripts/backup.ts

```
import "dotenv/config";
import { Pool } from "pg";
import { put, list, del } from "@vercel/blob";
import { mkdir, writeFile } from "node:fs/promises";
import { readPrivateFile } from "../src/lib/storage";
import { pgTool, recoveryKeys, encryptBundle, sha } from "./recovery-lib";
const url = process.env.DATABASE_URL_UNPOOLED!;
if (!new URL(url).pathname.startsWith("/practicechart"))
  throw Error("Backup must use a separate PracticeChart database.");
const pool = new Pool({ connectionString: url });
const c = await pool.connect();
const createdAt = new Date().toISOString();
const fileName = createdAt.replaceAll(":", "-") + ".pcbackup";
try {
  await c.query("BEGIN ISOLATION LEVEL REPEATABLE READ READ ONLY");
  const snapshot = (await c.query("SELECT pg_export_snapshot() AS snapshot"))
    .rows[0].snapshot;
  const docs = (
    await c.query("SELECT id,pathname,sha256 FROM documents ORDER BY id")
  ).rows as { id: string; pathname: string; sha256: string }[];
  const tables = (
    await c.query(
      "SELECT tablename FROM pg_tables WHERE schemaname='public' ORDER BY tablename",
    )
  ).rows as { tablename: string }[];
  const counts: Record<string, number> = {};
  const rowHashes: Record<string, string> = {};
  for (const { tablename } of tables) {
    if (!/^[a-z_]+$/.test(tablename)) throw Error("Unknown table name.");
    counts[tablename] = Number(
      (await c.query(`SELECT count(*) FROM ${tablename}`)).rows[0].count,
    );
    const rows = (
      await c.query(
        `SELECT row_to_json(t)::text AS row FROM ${tablename} t ORDER BY row_to_json(t)::text`,
      )
    ).rows;
    rowHashes[tablename] = sha(
      Buffer.from(rows.map((r: { row: string }) => r.row).join("\n")),
    );
  }
  const dump = await pgTool("pg_dump", url, [
    "--format=custom",
    "--no-owner",
    "--no-privileges",
    "--snapshot=" + snapshot,
    "--lock-wait-timeout=20s",
  ]);
  const files = [];
  for (const doc of docs) {
    const bytes = await readPrivateFile(doc.pathname);
    if (sha(bytes) !== doc.sha256)
      throw Error("A diagnostic file failed its integrity check.");
    files.push({ ...doc, bytes: bytes.toString("base64") });
  }
  await c.query("COMMIT");
  const payload = Buffer.from(
    JSON.stringify({
      format: "practicechart-full-ehr-v1",
      createdAt,
      database: new URL(url).pathname.slice(1),
      counts,
      rowHashes,
      dumpSha256: sha(dump),
      dump: dump.toString("base64"),
      files,
    }),
  );
  const publicKey = process.env.BACKUP_PUBLIC_KEY
    ? Buffer.from(process.env.BACKUP_PUBLIC_KEY, "base64").toString()
    : await recoveryKeys();
  const encrypted = encryptBundle(payload, publicKey);
  const directory = process.env.BACKUP_DIRECTORY || "backups";
  await mkdir(directory, { recursive: true });
```

```
  await writeFile(`${directory}/${fileName}`, encrypted, { flag: "wx" });
  let pathname = `local:${fileName}`;
  if (process.env.STORAGE_DRIVER === "blob") {
    const r = await put("practicechart/backups/" + fileName, encrypted, {
      access: "private",
      contentType: "application/octet-stream",
      addRandomSuffix: false,
    });
    pathname = r.pathname;
    const older = (await list({ prefix: "practicechart/backups/" })).blobs.sort(
      (a, b) => b.uploadedAt.valueOf() - a.uploadedAt.valueOf(),
    );
    for (const old of older.slice(7)) await del(old.url);
  }
  await pool.query(
    "INSERT INTO backup_runs(status,pathname) VALUES('complete',$1)",
    [pathname],
  );
  console.log(
    JSON.stringify({
      status: "complete",
      kind: "full EHR database and uploaded files",
      file: `${directory}/${fileName}`,
      tables: tables.length,
      documents: files.length,
      encrypted: true,
    }),
  );
} catch (e) {
  await c.query("ROLLBACK").catch(() => {});
  await pool
    .query("INSERT INTO backup_runs(status) VALUES('failed')")
    .catch(() => {});
  throw e;
} finally {
  c.release();

  await pool.end();
}
```

scripts/build-documents.py

```
"""Build public course PDFs and private submission cover copies.

Use the task's bundled document runtime. No secrets or private course scans are read.
"""
from pathlib import Path
from xml.sax.saxutils import escape
import re
import json
import textwrap
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, Table, TableStyle, Image, PageBreak, Preformatted, KeepTogether
from reportlab.lib import colors
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from reportlab.lib.enums import TA_LEFT
from reportlab.lib.pagesizes import A4
from PIL import Image as PILImage

ROOT = Path.cwd()
OUT = ROOT / 'docs' / 'submission'
PRIVATE = ROOT / '.private' / 'submission'
OUT.mkdir(parents=True, exist_ok=True)
PRIVATE.mkdir(parents=True, exist_ok=True)
styles = getSampleStyleSheet()
styles.add(ParagraphStyle(name='ReportBody', fontName='Helvetica', fontSize=10, leading=15, spaceAfter=9, textColor=colors.HexColor('#202a38')))
styles.add(ParagraphStyle(name='ReportTitle', fontName='Helvetica-Bold', fontSize=25, leading=31, spaceAfter=20, textColor=colors.black))
styles.add(ParagraphStyle(name='ReportH1', fontName='Helvetica-Bold', fontSize=16, leading=21, spaceBefore=16, spaceAfter=9, keepWithNext=True, textColor=colors.black))
styles.add(ParagraphStyle(name='ReportH2', fontName='Helvetica-Bold', fontSize=12, leading=17, spaceBefore=12, spaceAfter=7, keepWithNext=True, textColor=colors.black))
styles.add(ParagraphStyle(name='ReportCell', fontName='Helvetica', fontSize=7.5, leading=10, wordWrap='CJK', textColor=colors.HexColor('#202a38')))
styles.add(ParagraphStyle(name='Caption', fontName='Helvetica', fontSize=8, leading=11, spaceAfter=12, textColor=colors.HexColor('#49576b')))
WIDTH = A4[0] - 92

def clean(s):
    s = re.sub(r'\[([^\]]+)\]\([^)]*\)', r'\1', s)
    s = s.replace('**', '').replace('`', '')
    return s.replace('\u2014', ', ')

def para(s, style='ReportBody'):
    return Paragraph(escape(clean(s)), styles[style])

def footer(canvas, doc):
    canvas.saveState()
    canvas.setStrokeColor(colors.HexColor('#ccd4e0'))
    canvas.line(46, 38, A4[0] - 46, 38)
    canvas.setFont('Helvetica', 8)
    canvas.setFillColor(colors.HexColor('#49576b'))
    canvas.drawString(46, 25, 'PracticeChart EHR | CIT4001 | Fictional data only')
    canvas.drawRightString(A4[0] - 46, 25, str(doc.page))
    canvas.restoreState()


def cover(title, private=False):
    content = [Spacer(1, 45), para('CIT4001 SOFTWARE IMPLEMENTATION AND TESTING', 'Caption'),
               para(title, 'ReportTitle'), para('PracticeChart EHR'),
               para('University of Technology, Jamaica'), para('Group 2'), Spacer(1, 24),
               para('This document describes the implemented course application and the evidence recorded for it. It uses fictional patient data. Human training and user acceptance remain pending.')]
    if private:
        content += [Spacer(1, 20), para('Private submission cover details', 'ReportH2')]
        for name, number in json.loads((ROOT / '.private' / 'group-cover.json').read_text()) if (ROOT / '.private' / 'group-cover.json').exists() else []:
            content.append(para(name + ' / ' + number))
        content.append(para('Confirm these cover details before submission. A listed name is not evidence of a completed code contribution.', 'Caption'))
    else:
        content.append(para('Public course copy. Student identification numbers and private course instruments are excluded.', 'Caption'))
    content += [Spacer(1, 30), para('Prepared for group review. This is not a record of clinical approval, a live pilot, completed training, or completed acceptance testing.', 'Caption'), PageBreak()]
    return content
```

```
63
64 def table(lines):
65     data = [[para(cell.strip(), 'ReportCell') for cell in line.strip().strip('|').split('|')] for line in
   lines if not re.match(r'^\s*\|\s*:?-+', line)]
66     n = max(len(row) for row in data)
67     for row in data:
68         row += [para('', 'ReportCell')] * (n - len(row))
69     weights = [1] * n
70     if n == 2:
71         weights = [.35, .65]
72     elif n == 3:
73         weights = [.2, .4, .4]
74     total = sum(weights)
75     t = Table(data, colWidths=[WIDTH * w / total for w in weights], repeatRows=1, hAlign='LEFT')
76     t.setStyle(TableStyle([('BACKGROUND', (0, 0), (-1, 0), colors.HexColor('#e9eef7')), ('VALIGN', (0, 0),
    (-1, -1), 'TOP'), ('LINEBELOW', (0, 0), (-1, 0), .7, colors.HexColor('#8e9fbb')), ('LINEBELOW', (0, 1),
   (-1, -1), .25, colors.HexColor('#d7dfea')), ('TOPPADDING', (0, 0), (-1, -1), 7), ('BOTTOMPADDING', (0, 0),
    (-1, -1), 7), ('LEFTPADDING', (0, 0), (-1, -1), 6), ('RIGHTPADDING', (0, 0), (-1, -1), 6)]))
77     return [t, Spacer(1, 12)]
78
79 def image(path, label):
80     p = ROOT / 'docs' / path
81     if not p.exists():
82         return [para('See the source diagram or screenshot: ' + path, 'Caption')]
83     with PILImage.open(p) as im:
84         w, h = im.size
85     ratio = min(WIDTH / w, 525 / h)
86     return [Image(str(p), width=w * ratio, height=h * ratio, hAlign='LEFT'), para(label, 'Caption')]
87
88 def markdown(path, omit_title=True):
89     lines = path.read_text(encoding='utf-8').splitlines()
90     output = []
91     i = 0
92     while i < len(lines):
93         line = lines[i]
94         if not line.strip():

95             i += 1
96             continue
97         if line.startswith('```'):
98             code = []
99             i += 1
100            while i < len(lines) and not lines[i].startswith('```'):
101                code += textwrap.wrap(lines[i], width=102, replace_whitespace=False,
    drop_whitespace=False) or ['']
102                i += 1
103            for start in range(0, len(code), 45):
104                output.append(Preformatted('\n'.join(code[start:start + 45]), ParagraphStyle(name='Code',
    fontName='Courier', fontSize=7, leading=10, spaceAfter=12)))
105        elif line.startswith('|'):
106            rows = []
107            while i < len(lines) and lines[i].startswith('|'):
108                rows.append(lines[i])
109                i += 1
110            output += table(rows)
111            continue
112        elif line.startswith('!['):
113            match = re.match(r'!\[([^]]*)\]\(([^)]+)\)', line)
114            if match:
115                output += image(match[2], match[1])
116        elif line.startswith('# '):
117            if not omit_title:
118                output.append(para(line[2:], 'ReportH1'))
119        elif line.startswith('## '):
120            output.append(para(line[3:], 'ReportH1'))
121        elif line.startswith('### '):
122            output.append(para(line[4:], 'ReportH2'))
123        else:
124            group = [line]
125            if not re.match(r'^(- |\d+\. )', line):
126                while i + 1 < len(lines) and lines[i + 1].strip() and not lines[i + 1].startswith(('#',
    '|', '![', '```', '- ')):
127                    i += 1
128                    group.append(lines[i])
129            output.append(para(' '.join(group)))
130        i += 1
131    return output
132
```

```
def make(name, title, sections, private=False):
    target = (PRIVATE if private else OUT) / name
    doc = SimpleDocTemplate(str(target), pagesize=A4, rightMargin=46, leftMargin=46, topMargin=46, bottomMargin=52, title=title, author='CIT4001 Group 2')
    story = cover(title, private)
    for section in sections:
        story += markdown(ROOT / 'docs' / section)
        story.append(Spacer(1, 14))
    doc.build(story, onFirstPage=footer, onLaterPages=footer)
    print(target.relative_to(ROOT))

documents = [
    ('development-report.pdf', 'Software development and implementation report', ['DEVELOPMENT_REPORT.md', 'REQUIREMENTS.md', 'PERMISSIONS.md']),
    ('programme-manual.pdf', 'Programme manual and database design', ['PROGRAMME_MANUAL.md', 'RECOVERY.md']),
    ('user-manual.pdf', 'PracticeChart user manual', ['USER_MANUAL.md']),
    ('testing-and-control-flow.pdf', 'Testing and control flow appendix', ['TEST_PLAN.md', 'CONTROL_FLOW.md', 'MODULES.md']),
    ('training-and-acceptance-forms.pdf', 'Training plan and blank acceptance forms', ['TRAINING_AND_ACCEPTANCE.md'])
]
for private in [False, True]:
    for name, title, sections in documents:
        make(name, title, sections, private)
    source_files = sorted([p for folder in ['src', 'scripts', 'tests', 'migrations'] for p in (ROOT / folder).rglob('*') if p.is_file() and p.suffix in ['.ts', '.tsx', '.css', '.py', '.mjs', '.sql'] and '.well-known' not in p.parts])
    source_files += [ROOT / f for f in ['package.json', 'tsconfig.json', 'next.config.ts', 'playwright.config.ts', 'vitest.config.ts', 'docker-compose.yml', 'vercel.json']]
    story = cover('Indexed programme source listing', private)
    story += [para('Source index', 'ReportH1'), para('This listing includes authored application code, operator scripts, tests, migrations, and project configuration. Third-party dependencies, generated bundles, credentials, uploads, and private evidence are excluded.')]
    for n, p in enumerate(source_files, 1):
        story.append(para(f'{n}. {p.relative_to(ROOT).as_posix()}', 'Caption'))
    for p in source_files:
        story += [PageBreak(), para(p.relative_to(ROOT).as_posix(), 'ReportH2')]
        wrapped = []
        for n, line in enumerate(p.read_text(encoding='utf-8').splitlines(), 1):
            parts = textwrap.wrap(line.expandtabs(2), width=106, replace_whitespace=False, drop_whitespace=False) or ['']
            wrapped += [f'{n:4d} {parts[0]}'] + ['     ' + v for v in parts[1:]]
        for start in range(0, len(wrapped), 55):
            story.append(Preformatted('\n'.join(wrapped[start:start + 55]), ParagraphStyle(name='Source', fontName='Courier', fontSize=6.8, leading=9.4, spaceAfter=8)))
    target = (PRIVATE if private else OUT) / 'indexed-source-listing.pdf'
    SimpleDocTemplate(str(target), pagesize=A4, rightMargin=46, leftMargin=46, topMargin=46, bottomMargin=52, title='Indexed programme source listing', author='CIT4001 Group 2').build(story, onFirstPage=footer, onLaterPages=footer)
    print(target.relative_to(ROOT))
```

scripts/cloud-smoke.ts

```
import { chromium } from "@playwright/test";
import { readFile, writeFile, mkdir } from "node:fs/promises";
import assert from "node:assert/strict";
const base = process.argv[2];
if (!base?.startsWith("https://"))
  throw Error("Supply the HTTPS deployment URL.");
let bypass = "";
try {
  if (!process.argv.includes("--public"))
    bypass = (await readFile(".private/preview-bypass", "utf8")).trim();
} catch {}
const browser = await chromium.launch();
const checks = [];
for (const role of ["doctor", "frontdesk"]) {
  const context = await browser.newContext({
    viewport: { width: 1440, height: 1000 },
    extraHTTPHeaders: bypass ? { "x-vercel-protection-bypass": bypass } : {},
  });
  const page = await context.newPage();
  await page.goto(base + "/login");
  await page.getByLabel("Username", { exact: true }).fill(role);
  await page
    .getByLabel("Password", { exact: true })
    .fill(role === "doctor" ? "DoctorDemo!2026" : "DeskDemo!2026");
  await page.getByRole("button", { name: "Log in", exact: true }).click();
  await page.waitForURL("**/app", { timeout: 60000 });
  await page.getByRole("heading", { name: "Patient worklist" }).waitFor();
  const people = await context.request.get(base + "/api/patients");
  assert.equal(people.status(), 200);
  const list = await people.json();
  assert.ok(list.length >= 12);
  const patient = list.find(
    (p: { firstName: string }) => p.firstName === "Lena",
  );
  assert.ok(patient);
  const detail = await (
    await context.request.get(base + "/api/patients/" + patient.id)
  ).json();
  if (role === "doctor") {
    assert.ok(Array.isArray(detail.clinical));
    const docs = await (
      await context.request.get(base + "/api/documents")
    ).json();
    const d = await context.request.get(base + "/api/documents/" + docs[0].id);
    assert.equal(d.status(), 200);
    assert.equal((await d.body()).subarray(0, 5).toString(), "%PDF-");
    assert.ok(d.headers()["cache-control"].includes("no-store"));
    const sample = await context.request.get(base + "/api/sample-report");
    const bytes = await sample.body();
    const up = await context.request.post(base + "/api/documents", {
      headers: { origin: base },
      multipart: {
        patientId: patient.id,
        title: "Fictional cloud verification report",
        kind: "Lab report",
        file: {
          name: "sample.pdf",
          mimeType: "application/pdf",
          buffer: bytes,
        },
      },
    });
    assert.equal(up.status(), 201);
    const uploaded = await up.json();
    const exact = await context.request.get(
      base + "/api/documents/" + uploaded.id,
    );
    assert.deepEqual(await exact.body(), bytes);
    checks.push(
      "doctor clinical history and exact private cloud file round trip",
    );
  } else {
    assert.equal(detail.clinical, undefined);
    assert.equal(detail.documents, undefined);
```

```
    for (const path of [
      "documents",
      "users",
      "audit",
      "reports/history?id=" + patient.id,
    ])
      assert.equal(
        (await context.request.get(base + "/api/" + path)).status(),
        403,
      );
    checks.push(
      "front desk demographic access with denied clinical, file, staff, and audit routes",
    );
  }
  await context.close();
}
await browser.close();
await mkdir("docs/evidence", { recursive: true });
await writeFile(
  process.env.CLOUD_SMOKE_EVIDENCE_PATH ||
    (process.argv.includes("--public")
      ? "docs/evidence/cloud-smoke-production.json"
      : "docs/evidence/cloud-smoke.json"),
  JSON.stringify(
    {
      testedAt: new Date().toISOString(),
      deployment: base,
      access: bypass
        ? "Deployment protection bypass used for preview checks"
        : "Public deployment",
      checks,
      status: "passed",
    },
    null,
    2,
  ),
);
console.log("Cloud browser smoke checks passed for both roles.");
```

scripts/cold-performance.ts

```ts
import "dotenv/config";
import { spawn } from "node:child_process";
import { performance } from "node:perf_hooks";
import { writeFile } from "node:fs/promises";
import { chromium } from "@playwright/test";

const results = [];
const target = new URL(
  process.env.EHR_DATABASE_URL || process.env.DATABASE_URL!,
);
if (!["localhost", "127.0.0.1"].includes(target.hostname))
  throw Error("The process-cold test requires a local fictional database.");
for (let run = 0; run < 3; run++) {
  const port = 3002 + run;
  const base = `http://localhost:${port}`;
  const boot = performance.now();
  const server = spawn(
    process.execPath,
    ["node_modules/next/dist/bin/next", "start", "--port", String(port)],
    {
      env: { ...process.env, APP_URL: base },
      stdio: "ignore",
      windowsHide: true,
    },
  );
  const browser = await chromium.launch();
  try {
    let ready = false;
    for (let attempt = 0; attempt < 80; attempt++) {
      ready = await fetch(base + "/api/health")
        .then((r) => r.ok)
        .catch(() => false);
      if (ready) break;
      await new Promise((resolve) => setTimeout(resolve, 100));
    }
    if (!ready) throw Error("The cold test server did not start.");
    const startupMs = performance.now() - boot;
    const page = await browser.newPage();
    await page.goto(base + "/login");
    await page.getByLabel("Username", { exact: true }).fill("doctor");
    await page.getByLabel("Password", { exact: true }).fill("DoctorDemo!2026");
    let start = performance.now();
    await page.getByRole("button", { name: "Log in", exact: true }).click();
    await page.waitForURL("**/app");
    await page.getByRole("heading", { name: "Patient worklist" }).waitFor();
    const loginMs = performance.now() - start;
    const list = await (
      await page.request.get(base + "/api/patients?q=Lena")
    ).json();
    const patient = list.find(
      (p: { firstName: string }) => p.firstName === "Lena",
    );
    if (!patient) throw Error("The fictional benchmark patient was not found.");
    const records = [];
    for (let visit = 0; visit < 2; visit++) {
      if (visit) await page.goto(base + "/app/patients");
      start = performance.now();
      await page.goto(base + "/app/patients/" + patient.id);
      await page
        .getByRole("heading", {
          name: patient.firstName + " " + patient.lastName,
          exact: true,
        })
        .first()
        .waitFor();
      await page.evaluate(
        () =>
          new Promise<void>((resolve) =>
            requestAnimationFrame(() => requestAnimationFrame(() => resolve())),
          ),
      );
      records.push(Number((performance.now() - start).toFixed(2)));
    }
    results.push({
```

```
      run: run + 1,
      startupMs: Number(startupMs.toFixed(2)),
      loginMs: Number(loginMs.toFixed(2)),
      firstPatientDisplayMs: records[0],
      warmPatientDisplayMs: records[1],
    });
  } finally {
    await browser.close();
    server.kill();
  }
}
const evidence = {
  testedAt: new Date().toISOString(),
  method:
    "Three fresh local production Node processes and fresh Chromium sessions. A health probe precedes login. Each process shows one first patient display and one repeat display.",
  limits:
    "Process-cold measurements. PostgreSQL and operating system caches remain warm. This is not a Vercel cold start or a cold database test. Three samples do not establish a percentile guarantee.",
  environmentSource: "performance.json",
  results,
};
await writeFile(
  "docs/evidence/cold-performance.json",
  JSON.stringify(evidence, null, 2),
);
console.log(JSON.stringify(evidence, null, 2));
```

scripts/course-analysis.ts

```
import ts from "typescript-compiler";
import { readFile, writeFile, mkdir } from "node:fs/promises";
import { instance } from "@viz-js/viz";
type Node = {
  id: number;
  label: string;
  kind: "action" | "decision" | "end";
  line: number;
};
type Edge = { from: number; to: number; label: string };
type Tail = { from: number; label: string };
const viz = await instance();
await mkdir("docs/diagrams/cfg", { recursive: true });
const narratives: Record<string, string> = {
  patientList:
    "Search demographic records with parameters and record the search event.",
  patientDetail:
    "Read a patient and visits. Add clinical history and documents only for a doctor.",
  savePatient:
    "Validate demographics. Create an MRN or update the expected record version. Warn about possible duplicates.",
  archivePatient:
    "Require a doctor and the current version before changing patient archive status.",
  reconcilePatient:
    "Require a doctor. Lock both records and link an unresolved provisional record to its primary record.",
  schedule:
    "Read the appointments for one Jamaica calendar day and record the view.",
  availability:
    "Find half-hour start times for the full selected visit length. Exclude doctor and patient conflicts and blocked periods. Ignore the current appointment during rescheduling.",
  bookAppointment:
    "Validate the interval and active patient. Insert a confirmed booking or update a current confirmed booking. Audit only a committed change.",
  appointmentStatus:
    "Change a current confirmed appointment to arrived or cancelled. Preserve the booking record.",
  startVisit:
    "Create a walk-in visit or start a matching appointment. Keep its doctor and patient association.",
  saveClinical:
    "Require doctor access. Lock the visit, check its revision, and append a draft, final entry, or reasoned final correction.",
  fileVisit:
    "Lock the visit. Require the current final diagnosis. File the visit and complete its appointment.",
  saveInvoice:
    "Validate service lines and calculate integer-cent totals before creating an invoice draft.",
  setInvoiceStatus:
    "Issue a draft or void an issued invoice with a reason. Reject stale versions.",
  invoiceList: "Read administrative invoice data and record the view.",
  manageUser:
    "Require doctor access. Create or change an account. Protect demo accounts, keep a doctor active, and revoke changed sessions.",
  changePassword:
    "Verify the current password, hash the replacement, and revoke existing sessions.",
  dashboard:
    "Read practice counts, ready visits, and in-app reminders without clinical note bodies.",

  reportData:
    "Select the report type, require clinical permission where needed, and record an export event.",
  requireRole: "Check the server account role and audit denied access.",
  getSession:
    "Hash the cookie token and find an unexpired session for an active account.",
  login:
    "Limit attempts, verify a password hash, create a random hashed session, and record the result.",
  storeFile:
    "Validate length, signature, and public demo sample rules before writing an immutable private object.",
  readPrivateFile:
    "Reject unsafe paths and retrieve a private object. Return a clear missing-file error.",
  invoiceTotal:
    "Sum integer quantity times unit cents and reject unsafe or excessive totals.",
  fileMime: "Classify only PDF, PNG, or JPEG file signatures.",
};
const files = [
  "src/lib/service.ts",
  "src/lib/auth.ts",
```

```
  "src/lib/storage.ts",
  "src/lib/validation.ts",
];
const graphs = [];
let moduleText =
  "# Processing narratives and pseudocode\n\nGenerated from the current application source. The source listing remains the detailed programme reference. SQL and library internals are represented as calls. Transaction callbacks are expanded. Array callbacks stay within their library operation. The graphs cover explicit application branches, short-circuit conditions, conditional expressions, and loops. A try block includes an explicit I/O failure branch. External library internals are not counted.\n\n";
for (const file of files) {
  const source = await readFile(file, "utf8");
  const sf = ts.createSourceFile(file, source, ts.ScriptTarget.Latest, true);
  for (const declaration of sf.statements) {
    if (
      !ts.isFunctionDeclaration(declaration) ||
      !declaration.body ||
      !declaration.name ||
      !narratives[declaration.name.text]
    )
      continue;
    const name = declaration.name.text;
    const nodes: Node[] = [
      { id: 1, label: "START", kind: "end", line: 0 },
      { id: 2, label: "END", kind: "end", line: 0 },
    ];
    const edges: Edge[] = [];
    const code = (n: ts.Node) => n.getText(sf).replace(/\s+/g, " ").trim();
    const add = (
      label: string,
      kind: Node["kind"] = "action",
      node?: ts.Node,
    ) => {
      const id = nodes.length + 1;
      nodes.push({
        id,
        label: label.slice(0, 150),
        kind,
        line: node
          ? sf.getLineAndCharacterOfPosition(node.getStart(sf)).line + 1
          : 0,
      });
      return id;
    };
    const connect = (tails: Tail[], id: number) => {
      for (const t of tails)
        edges.push({ from: t.from, to: id, label: t.label });
    };
    const action = (label: string, tails: Tail[], node?: ts.Node) => {
      const id = add(label, "action", node);
      connect(tails, id);
      return [{ from: id, label: "" }];
    };
    function condition(
      expr: ts.Expression,
      tails: Tail[],
    ): { yes: Tail[]; no: Tail[] } {
      if (ts.isParenthesizedExpression(expr))
        return condition(expr.expression, tails);
      if (
        ts.isPrefixUnaryExpression(expr) &&
        expr.operator === ts.SyntaxKind.ExclamationToken
      ) {
        const c = condition(expr.operand, tails);
        return { yes: c.no, no: c.yes };
      }
      if (
        ts.isBinaryExpression(expr) &&
        expr.operatorToken.kind === ts.SyntaxKind.AmpersandAmpersandToken
      ) {
        const a = condition(expr.left, tails),
          b = condition(expr.right, a.yes);
        return { yes: b.yes, no: [...a.no, ...b.no] };
      }
      if (
        ts.isBinaryExpression(expr) &&
        expr.operatorToken.kind === ts.SyntaxKind.BarBarToken
```

```
    ) {
      const a = condition(expr.left, tails),
        b = condition(expr.right, a.no);
      return { yes: [...a.yes, ...b.yes], no: b.no };
    }
    const id = add(code(expr), "decision", expr);
    connect(tails, id);
    return {
      yes: [{ from: id, label: "True" }],
      no: [{ from: id, label: "False" }],
    };
  }
  function expression(expr: ts.Node, tails: Tail[]): Tail[] {
    if (ts.isArrowFunction(expr) || ts.isFunctionExpression(expr))
      return tails;
    if (ts.isConditionalExpression(expr)) {
      const c = condition(expr.condition, tails);
      return [
        ...expression(expr.whenTrue, c.yes),
        ...expression(expr.whenFalse, c.no),
      ];
    }
    if (
      ts.isBinaryExpression(expr) &&
      [
        ts.SyntaxKind.AmpersandAmpersandToken,
        ts.SyntaxKind.BarBarToken,
        ts.SyntaxKind.QuestionQuestionToken,
      ].includes(expr.operatorToken.kind)
    ) {
      const id = add(
        expr.operatorToken.kind === ts.SyntaxKind.QuestionQuestionToken
          ? code(expr.left) + " is not null"
          : code(expr.left),
        "decision",
        expr.left,
      );
      connect(tails, id);
      const yes = [{ from: id, label: "True" }],
        no = [{ from: id, label: "False" }];
      return expr.operatorToken.kind === ts.SyntaxKind.AmpersandAmpersandToken
        ? [...expression(expr.right, yes), ...no]
        : [...yes, ...expression(expr.right, no)];
    }
    ts.forEachChild(expr, (child) => {
      tails = expression(child, tails);
    });
    return tails;
  }
  function transaction(expr: ts.Expression | undefined) {
    if (!expr) return;
    while (ts.isAwaitExpression(expr) || ts.isParenthesizedExpression(expr))
      expr = expr.expression;
    if (
      ts.isCallExpression(expr) &&
      ts.isPropertyAccessExpression(expr.expression) &&
      expr.expression.name.text === "transaction"
    ) {
      const fn = expr.arguments.find(
        (a) => ts.isArrowFunction(a) || ts.isFunctionExpression(a),
      );
      if (
        fn &&
        (ts.isArrowFunction(fn) || ts.isFunctionExpression(fn)) &&
        ts.isBlock(fn.body)
      )
        return fn.body;
    }
  }

  function block(
    statements: readonly ts.Statement[],
    tails: Tail[],
    returnTo = 2,
    errorTo = 2,
  ): Tail[] {
    for (const statement of statements) {
```

```
        if (!tails.length) break;
        if (ts.isIfStatement(statement)) {
          const c = condition(statement.expression, tails);
          const yes = block(
            ts.isBlock(statement.thenStatement)
              ? statement.thenStatement.statements
              : [statement.thenStatement],
            c.yes,
            returnTo,
            errorTo,
          );
          const no = statement.elseStatement
            ? block(
                ts.isBlock(statement.elseStatement)
                  ? statement.elseStatement.statements
                  : [statement.elseStatement],
                c.no,
                returnTo,
                errorTo,
              )
            : c.no;
          tails = [...yes, ...no];
          continue;
        }
        if (
          ts.isForOfStatement(statement) ||
          ts.isForInStatement(statement) ||
          ts.isWhileStatement(statement) ||
          ts.isForStatement(statement)
        ) {
          const label =
            ts.isForOfStatement(statement) || ts.isForInStatement(statement)
              ? "More items in " + code(statement.expression)
              : ts.isWhileStatement(statement)
                ? code(statement.expression)
                : statement.condition
                  ? code(statement.condition)
                  : "Continue loop";
          const id = add(label, "decision", statement);
          connect(tails, id);
          const body = ts.isBlock(statement.statement)
            ? statement.statement.statements
            : [statement.statement];
          const ends = block(
            body,
            [{ from: id, label: "True" }],
            returnTo,
            errorTo,

          );
          for (const t of ends)
            edges.push({ from: t.from, to: id, label: "Next" });
          tails = [{ from: id, label: "False" }];
          continue;
        }
        if (ts.isTryStatement(statement)) {
          const guard = add("I/O operation succeeds", "decision", statement);
          connect(tails, guard);
          const catchStart = add(
            "Handle I/O failure",
            "action",
            statement.catchClause,
          );
          edges.push({ from: guard, to: catchStart, label: "False" });
          const yes = block(
            statement.tryBlock.statements,
            [{ from: guard, label: "True" }],
            returnTo,
            catchStart,
          );
          const no = statement.catchClause
            ? block(
                statement.catchClause.block.statements,
                [{ from: catchStart, label: "" }],
                returnTo,
                errorTo,
              )
            : [];
```

```
          tails = [...yes, ...no];
          if (statement.finallyBlock)
            tails = block(
              statement.finallyBlock.statements,
              tails,
              returnTo,
              errorTo,
            );
          continue;
        }
        if (ts.isBlock(statement)) {
          tails = block(statement.statements, tails, returnTo, errorTo);
          continue;
        }
        if (ts.isReturnStatement(statement) || ts.isThrowStatement(statement)) {
          const tx = ts.isReturnStatement(statement)
            ? transaction(statement.expression)
            : undefined;
          if (tx) {
            const begin = action("BEGIN transaction", tails, statement);
            const commit = add("COMMIT transaction", "action");
            const ends = block(tx.statements, begin, commit, errorTo);
            connect(ends, commit);
            edges.push({ from: commit, to: returnTo, label: "Return" });
          } else {
            const ends = statement.expression
              ? expression(statement.expression, tails)
              : tails;
            const terminal = action(
              (ts.isThrowStatement(statement) ? "RAISE " : "RETURN ") +
                (statement.expression ? code(statement.expression) : ""),
              ends,
              statement,
            );
            connect(
              terminal,
              ts.isThrowStatement(statement) ? errorTo : returnTo,
            );
          }
          tails = [];
          continue;
        }
        if (ts.isExpressionStatement(statement)) {
          const tx = transaction(statement.expression);
          if (tx) {
            const begin = action("BEGIN transaction", tails, statement);
            const commit = add("COMMIT transaction", "action");
            connect(block(tx.statements, begin, commit, errorTo), commit);
            tails = [{ from: commit, label: "" }];
            continue;
          }
        }
        tails = expression(statement, tails);
        tails = action(code(statement), tails, statement);
      }
      return tails;
    }
    connect(block(declaration.body.statements, [{ from: 1, label: "" }]), 2);
    const used = new Set([1, 2, ...edges.flatMap((e) => [e.from, e.to])]);
    const active = nodes.filter((n) => used.has(n.id));
    const unique = edges.filter(
      (e, i) =>
        edges.findIndex(
          (x) => x.from === e.from && x.to === e.to && x.label === e.label,
        ) === i,
    );
    const seenDirected = new Set<string>();
    for (let i = 0; i < unique.length; i++) {
      const e = unique[i],
        key = e.from + ":" + e.to;
      if (seenDirected.has(key)) {
        const target = e.to;
        const id = add(
          "Follow " + (e.label || "alternate") + " branch",
          "action",
        );
        active.push(nodes.at(-1)!);
```

```
        e.to = id;
        unique.push({ from: id, to: target, label: "" });
      } else seenDirected.add(key);
    }

    const predicates = active.filter(
      (n) => unique.filter((e) => e.from === n.id).length === 2,
    ).length;
    const complexity = unique.length - active.length + 2;
    const dot = `digraph ${name} { graph [label="${name}",labelloc=t,rankdir=TB,bgcolor="white"];node [fontname="Arial",fontsize=10,style=filled,fillcolor="#f5f7fb",color="#8c9bb4"];edge [fontname="Arial",fontsize=9,color="#63748d"];${active.map((n) => `n${n.id}[shape=${n.kind === "decision" ? "diamond" : n.kind === "end" ? "oval" : "box"},label=${JSON.stringify(n.id + ": " + n.label.replace(/(.{42})/g, "$1\n"))}];`).join("\n")}${unique.map((e) => `n${e.from}->n${e.to}[label=${JSON.stringify(e.label)}];`).join("\n")}}`;
    await writeFile(`docs/diagrams/cfg/${name}.dot`, dot);
    await writeFile(
      `docs/diagrams/cfg/${name}.svg`,
      viz.renderString(dot, { format: "svg" }),
    );
    const paths: number[][] = [];
    function dfs(node: number, path: number[], visits: Map<number, number>) {
      if (paths.length >= 5000 || path.length > active.length * 3) return;
      if (node === 2) {
        paths.push([...path, node]);
        return;
      }
      if ((visits.get(node) || 0) >= 2) return;
      const next = new Map(visits);
      next.set(node, (next.get(node) || 0) + 1);
      for (const e of unique.filter((e) => e.from === node))
        dfs(e.to, [...path, node], next);
    }
    dfs(1, [], new Map());
    const basis: number[][] = [];
    const pivots = new Map<number, bigint>();
    for (const p of paths) {
      let v = 0n;
      for (let i = 0; i < p.length - 1; i++) {
        const index = unique.findIndex(
          (e) => e.from === p[i] && e.to === p[i + 1],
        );
        v ^= 1n << BigInt(index);
      }
      for (const [k, row] of [...pivots.entries()].sort((a, b) => b[0] - a[0]))
        if ((v & (1n << BigInt(k))) !== 0n) v ^= row;
      if (v) {
        const pivot = v.toString(2).length - 1;
        pivots.set(pivot, v);
        basis.push(p);
        if (basis.length === complexity) break;
      }
    }
    graphs.push({
      name,
      file,
      line: sf.getLineAndCharacterOfPosition(declaration.getStart(sf)).line + 1,
      nodes: active,
      edges: unique,
      predicates,

      edgeNodeComplexity: complexity,
      predicateComplexity: predicates + 1,
      syntacticBasisCandidates: basis,
    });
    const instructions = active.map((n) => {
      const next = unique.filter((e) => e.from === n.id);
      if (n.id === 2) return "L2: END PROCEDURE";
      if (n.kind === "decision")
        return `L${n.id}: IF ${n.label}\n  THEN GO TO L${next.find((e) => e.label === "True")?.to}\n  ELSE GO TO L${next.find((e) => e.label === "False")?.to}`;
      return `L${n.id}: ${n.id === 1 ? "START " + name : n.label}\n  ${next.map((e) => "GO TO L" + e.to).join(" OR ")}`;
    });
    moduleText += `## ${name}\n\n${narratives[name]}\n\nSource: ${file}. Graph: [${name}](diagrams/cfg/${name}.svg).\n\nThe following labelled pseudocode follows graph edges. Labels are not execution order. A callback RETURN sets its result; the COMMIT step must succeed before the service returns that result. A raised error ends the transaction with rollback. Long source operations are
```

```
    abbreviated here; use the indexed source listing for their full
    arguments.\n\n\`\`\`text\n${instructions.join("\n")}\n\`\`\`\n\n`;
437   }
438 }
439 await writeFile("docs/MODULES.md", moduleText);
440 await writeFile(
441   "docs/evidence/control-flow.json",
442   JSON.stringify(graphs, null, 2),
443 );
444 console.log(
445   `Generated ${graphs.length} source-linked module graphs and processing narratives.`,
446 );
447
448 const diagrams: Record<string, string> = {
449   use_case: `graph UseCases{graph[label="PracticeChart use
    cases",labelloc=t,rankdir=LR];node[fontname="Arial",fontsize=12];desk[shape=plaintext,label="Front
    desk"];doctor[shape=plaintext,label="Doctor"];subgraph cluster_system{label="PracticeChart
    EHR";node[shape=ellipse];login[label="Log in"];patient[label="Register and update
    patient"];schedule[label="Manage appointment"];file[label="File completed visit"];invoice[label="Create
    manual invoice"];clinical[label="Record and correct care"];lab[label="Store and retrieve lab
    report"];report[label="Print clinical patient report"];accounts[label="Manage staff
    access"];audit[label="View audit history"];}desk--login;desk--patient;desk--schedule;desk--file;desk--
    invoice;doctor--login;doctor--patient;doctor--schedule;doctor--file;doctor--invoice;doctor--
    clinical;doctor--lab;doctor--report;doctor--accounts;doctor--audit;}`,
450   sequence: `digraph Sequence{graph[label="Appointment confirmation sequence",labelloc=t,rankdir=LR];node[
    shape=box,fontname="Arial",style=filled,fillcolor="#f3f6fa"];user[label="1 Front desk\nSelect patient and
    time"];ui[label="2 Booking form\nCheck availability\nShow review"];api[label="3 Server\nAuthenticate and
    validate"];db[label="4 PostgreSQL transaction\nCheck hours and exclusions\nInsert booking and
    audit"];response[label="5 Return committed result\nShow confirmation or conflict"];user-
    >ui[label="request"];ui->api[label="confirm"];api->db[label="transaction"];db->response[label="commit or
    rollback"];response->user[label="result"];}`,
451   swimlane: `digraph Swimlane{graph[label="Visit workflow by
    role",labelloc=t,rankdir=LR];node[shape=box,fontname="Arial"];subgraph cluster_desk{label="Front
    desk";register[label="Register or find patient"];arrival[label="Mark arrival and start
    visit"];file[label="File ready visit"];invoice[label="Create invoice or attendance slip"];}subgraph
    cluster_doctor{label="Doctor";review[label="Review patient history"];record[label="Enter notes, diagnosis,
     and treatment"];final[label="Save final clinical revision"];}subgraph cluster_system{label="System and
    PostgreSQL";guard[label="Require final diagnosis"];log[label="Commit filed state and audit"];}register-

    >arrival;arrival->review;review->record;record->final;final->guard;guard->file[label="ready state
    only"];file->log;log->invoice;}`,
452   data: `digraph Data{graph[label="PracticeChart data relationships",labelloc=t,rankdir=LR];node[shape=rec
    ord,fontname="Arial",fontsize=11];users[label="users|id PK|username UNIQUE|role and password
    hash"];patients[label="patients|id PK|mrn UNIQUE|demographics and
    version"];appointments[label="appointments|id PK|patient_id FK|doctor_id FK|start_at and end_at|overlap
    exclusions"];visits[label="visits|id PK|patient_id FK|doctor_id FK|appointment_id FK UNIQUE|revision and
    status"];revisions[label="clinical_revisions|id PK|visit_id FK|author_id FK|revision UNIQUE per
    visit|content JSONB and state"];prescriptions[label="prescriptions|revision_id FK|visit_id FK|patient_id
    FK|doctor-entered content"];documents[label="documents|id PK|patient_id FK|replaces_id FK|private path and
     SHA256"];invoices[label="invoices|id PK|number UNIQUE|patient_id FK|lines and integer
    cents"];audit[label="audit_events|id PK|actor_id FK|action and record
    reference"];sessions[label="sessions|token hash PK|user_id FK|expiry"];users->sessions[label="1 to
    many"];patients->appointments[label="1 to
    many"];users->appointments[label="doctor"];patients->visits[label="1 to
    many"];appointments->visits[label="0 or 1"];users->visits[label="doctor"];visits->revisions[label="1 to
    many"];revisions->prescriptions[label="0 to many"];patients->documents[label="1 to
    many"];patients->invoices[label="1 to many"];users->audit[label="actor"];}`,
453   class: `digraph Classes{graph[label="Application domain classes and
    services",labelloc=t,rankdir=LR];node[shape=record,fontname="Arial"];User[label="Staff|id, role,
    active|requireRole()\ngetSession()"];Patient[label="Patient|id, mrn, demographics,
    version|savePatient()\narchivePatient()\nreconcilePatient()"];Appointment[label="Appointment|doctor,
    patient, interval,
    status|availability()\nbookAppointment()\nappointmentStatus()"];Visit[label="Visit|patient, doctor,
    revision, state|startVisit()\nsaveClinical()\nfileVisit()"];Clinical[label="Clinical revision|content,
    author, reason, state|append only"];Document[label="Diagnostic document|patient, path,
    digest|storeFile()\nreadPrivateFile()"];Invoice[label="Invoice|lines, total cents, status|saveInvoice()\ns
    etInvoiceStatus()"];User->Patient[label="uses"];User->Appointment[label="uses"];Patient-
    >Visit[label="has"];Appointment->Visit[label="may start"];Visit->Clinical[label="has
    revisions"];Patient->Document[label="has"];Patient->Invoice[label="has"];}`,
454   ipo: `digraph Modules{graph[label="Input processing and output modules",labelloc=t,rankdir=TB];node[shap
    e=box,fontname="Arial",style=filled,fillcolor="#f3f6fa"];ehr[label="PracticeChart
    EHR"];input[label="INPUT"];process[label="PROCESSING"];output[label="OUTPUT"];ehr->{input process
    output};credentials[label="User credentials"];demographics[label="Patient
    demographics"];requests[label="Appointment requests"];files[label="Diagnostic
    files"];clinicalInput[label="Doctor clinical entries"];input->{credentials demographics requests files
    clinicalInput};auth[label="Authentication and role checks"];patient[label="Patient
    management"];appointment[label="Appointment transactions"];care[label="Clinical revisions and
    filing"];billing[label="Invoice calculation"];recovery[label="Backup and recovery"];process->{auth patient
```

```
     appointment care billing recovery};docs[label="Clinical documents and patient
    history"];admin[label="Schedules, attendance, and invoices"];audit[label="Audit history and recovery
    evidence"];output->{docs admin audit};}`,
455   dfd: `digraph DFD{graph[label="Level 1 data
    flow",labelloc=t,rankdir=LR];node[shape=box,fontname="Arial"];staff[label="Staff"];auth[label="1
    Authenticate",shape=ellipse];patient[label="2 Manage patient",shape=ellipse];booking[label="3 Manage
    appointment",shape=ellipse];care[label="4 Record and retrieve care",shape=ellipse];report[label="5 Produce
     report",shape=ellipse];database[label="EHR PostgreSQL",shape=cylinder];blob[label="Private diagnostic
    storage",shape=cylinder];staff->auth[label="credentials"];auth->patient[label="server
    identity"];auth->booking[label="server identity"];auth->care[label="doctor
    identity"];patient->database[label="demographics"];booking->database[label="booking and
    audit"];care->database[label="clinical revisions"];care->blob[label="private document
    bytes"];database->report[label="authorised records"];blob->report[label="authorised
    document"];report->staff[label="reading or print output"];}`,
456 };
457 for (const [name, dot] of Object.entries(diagrams)) {

458   await writeFile(`docs/diagrams/${name}.dot`, dot);
459   await writeFile(
460     `docs/diagrams/${name}.svg`,
461     viz.renderString(dot, { format: "svg" }),
462   );
463 }
464 console.log(
465   "Generated use case, sequence, swimlane, data, class, module, and data flow diagrams.",
466 );
```

scripts/diagram-details.ts

```
import {writeFile,mkdir,readdir}from 'node:fs/promises';
import {chromium}from '@playwright/test';
await mkdir('docs/diagrams/png',{recursive:true});
const esc=(s:string)=>s.replaceAll('&','&amp;').replaceAll('<','&lt;').replaceAll('>','&gt;');
const head=(w:number,h:number)=>`<svg xmlns="http://www.w3.org/2000/svg" width="${w}" height="${h}"
viewBox="0 0 ${w} ${h}"><defs><marker id="arrow" markerWidth="8" markerHeight="8" refX="7" refY="4"
orient="auto"><path d="M0 0 L8 4 L0 8z" fill="#40577a"/></marker></defs><rect width="100%" height="100%"
fill="#fff"/><style>text{font-family:Arial,sans-serif;font-size:14px;fill:#233b61}.title{font-
size:22px;font-weight:bold}.small{font-size:12px}.line{stroke:#40577a;fill:none;stroke-
width:1.4}.node{fill:#f3f6fb;stroke:#879ab8}</style>`;
let svg=head(1100,790)+'<text x="550" y="34" text-anchor="middle" class="title">Appointment confirmation
sequence</text>';
const actors=['Front desk','Booking form','Server service','PostgreSQL'];const x=[130,410,690,970];
actors.forEach((name,i)=>{svg+=`<rect class="node" x="${x[i]-80}" y="65" width="160" height="44"/><text
x="${x[i]}" y="92" text-anchor="middle">${name}</text><path class="line" stroke-dasharray="5 5"
d="M${x[i]} 109V750"/>`;});
const messages:[[number,number,string],...[number,number,string][]]=[[0,1,'Select patient, doctor, date,
and time'],[1,2,'Request availability'],[2,3,'Query hours, bookings, and blocks'],[3,2,'Return available
intervals'],[2,1,'Return start times'],[1,0,'Show review, no booking saved'],[0,1,'Confirm
appointment'],[1,2,'Send validated booking request'],[2,3,'Begin transaction; insert booking and
audit'],[3,2,'Commit, or reject constraint and roll back'],[2,1,'Return committed result or
conflict'],[1,0,'Display confirmation or conflict']];
messages.forEach(([a,b,label],i)=>{const y=145+i*49;svg+=`<path class="line" marker-end="url(#arrow)"
d="M${x[a]} ${y}H${x[b]}"/><text class="small" x="${(x[a]+x[b])/2}" y="${y-9}" text-
anchor="middle">${i+1}. ${esc(label)}</text>`;});svg+='</svg>';await
writeFile('docs/diagrams/sequence.svg',svg);await
writeFile('docs/diagrams/sequence.mmd','sequenceDiagram\n participant FD as Front desk\n participant UI as
 Booking form\n participant S as Server\n participant DB as PostgreSQL\n FD->>UI: Choose patient and
interval\n UI->>S: Request availability\n S->>DB: Read hours, bookings, and blocks\n DB-->>S: Available
slots\n S-->>UI: Start times\n UI-->>FD: Show review\n FD->>UI: Confirm\n UI->>S: Submit booking\n S->>DB:
 Transaction: insert and audit\n alt Valid interval\n DB-->>S: Commit\n S-->>UI: Confirmation\n else
Conflict\n DB-->>S: Reject and roll back\n S-->>UI: Conflict\n end\n');
svg=head(1100,780)+'<text x="550" y="34" text-anchor="middle" class="title">PracticeChart use case
diagram</text><rect x="230" y="65" width="640" height="660" fill="none" stroke="#879ab8"/><text x="550"
y="92" text-anchor="middle">PracticeChart EHR</text>';
function actor(cx:number,label:string){return `<circle class="line" cx="${cx}" cy="312" r="16"/><path
class="line" d="M${cx} 328V385M${cx-28} 348H${cx+28}M${cx} 385L${cx-24} 420M${cx} 385L${cx+24} 420"/><text
 x="${cx}" y="450" text-anchor="middle">${label}</text>`;}
const left=['Log in','Register and update patient','Manage appointment','File ready visit','Create manual
invoice'];const right=['Record and correct care','Create clinical documents','Retrieve diagnostic
report','Manage staff accounts','View clinical audit'];
for(let i=0;i<5;i++){const y=155+i*118;svg+=`<path class="line" d="M120 350L270 ${y}"/><path class="line"
d="M980 350L480 ${y}"/><path class="line" d="M980 350L830 ${y}"/>`;}
left.forEach((label,i)=>{const y=155+i*118;svg+=`<ellipse class="node" cx="375" cy="${y}" rx="105"
ry="33"/><text class="small" x="375" y="${y+4}" text-
anchor="middle">${label}</text>`;});right.forEach((label,i)=>{const y=155+i*118;svg+=`<ellipse
class="node" cx="725" cy="${y}" rx="105" ry="33"/><text class="small" x="725" y="${y+4}" text-
anchor="middle">${label}</text>`;});svg+=actor(120,'Front desk')+actor(980,'Doctor')+'</svg>';await
writeFile('docs/diagrams/use_case.svg',svg);
svg=head(1050,950)+'<text x="525" y="34" text-anchor="middle" class="title">Visit workflow
swimlanes</text>';
['Front desk','PracticeChart and database','Doctor'].forEach((label,i)=>{svg+=`<rect x="${25+i*340}"
y="65" width="340" height="850" fill="none" stroke="#cad3e1"/><rect x="${25+i*340}" y="65" width="340"
height="45" fill="#eef2f8"/><text x="${195+i*340}" y="94" text-anchor="middle">${label}</text>`;});
const steps=[{lane:0,text:'Register or find patient'},{lane:1,text:'Validate and save

demographics'},{lane:0,text:'Mark arrival and start visit'},{lane:2,text:'Review history and enter
care'},{lane:1,text:'Append draft clinical revision'},{lane:2,text:'Enter diagnosis and
finalise'},{lane:1,text:'Mark visit ready'},{lane:0,text:'Request visit filing'},{lane:1,text:'Require
final diagnosis and commit logbook'},{lane:0,text:'Print attendance or create invoice'}];
steps.forEach((s,i)=>{const cx=195+s.lane*340,y=140+i*75;if(i){const previous=steps[i-1];svg+=`<path
class="line" marker-end="url(#arrow)" d="M${195+previous.lane*340} ${140+(i-1)*75+42}L${cx}
${y}"/>`;}svg+=`<rect class="node" x="${cx-150}" y="${y}" width="300" height="42" rx="3"/><text
class="small" x="${cx}" y="${y+25}" text-anchor="middle">${s.text}</text>`;});svg+='</svg>';await
writeFile('docs/diagrams/swimlane.svg',svg);
const browser=await chromium.launch();const page=await browser.newPage({deviceScaleFactor:1.5});for(const
file of(await readdir('docs/diagrams')).filter(n=>n.endsWith('.svg'))){const svg=await(await
import('node:fs/promises')).readFile('docs/diagrams/'+file,'utf8');await
page.setViewportSize({width:1600,height:1200});await page.setContent('<style>body{margin:0}svg{max-
width:1500px;height:auto}</style>'+svg);await
page.locator('svg').screenshot({path:'docs/diagrams/png/'+file.replace('.svg','.png')});}await
browser.close();console.log('Rendered course diagrams with UML lifelines, role lanes, and use case
actors.');
```

scripts/grant-runtime.ts

```
import "dotenv/config";
import { Pool } from "pg";
const pool = new Pool({ connectionString: process.env.DATABASE_URL_UNPOOLED });
const runtimeUser = new URL(process.env.DATABASE_URL!).username;
if (runtimeUser !== "practicechart_app")
  throw Error("Use the dedicated practicechart_app runtime role.");
const database = new URL(process.env.DATABASE_URL_UNPOOLED!).pathname.slice(1);
if (!/^practicechart[_a-z0-9]+$/.test(database))
  throw Error("Use a PracticeChart database.");
await pool.query(
  `GRANT CONNECT ON DATABASE ${database} TO practicechart_app; GRANT USAGE ON SCHEMA public TO
practicechart_app; GRANT SELECT,INSERT,UPDATE,DELETE ON ALL TABLES IN SCHEMA public TO practicechart_app;
GRANT USAGE,SELECT ON ALL SEQUENCES IN SCHEMA public TO practicechart_app; REVOKE UPDATE,DELETE ON
clinical_revisions,audit_events,prescriptions FROM practicechart_app; REVOKE ALL ON migrations FROM
practicechart_app; REVOKE CREATE ON SCHEMA public FROM PUBLIC;`,
);
await pool.end();
console.log("Runtime grants applied.");
```

scripts/local-setup.ts

```
import { existsSync, copyFileSync } from "node:fs";
import { config } from "dotenv";
import { spawnSync } from "node:child_process";
import { Pool } from "pg";
if (!existsSync(".env")) copyFileSync(".env.example", ".env");
config({ quiet: true });
const owner = new URL(process.env.DATABASE_URL_UNPOOLED!);
if (!["localhost", "127.0.0.1"].includes(owner.hostname))
  throw Error("Local setup requires a local database URL.");
const docker = spawnSync("docker", ["compose", "up", "-d", "--wait"], {
  stdio: "inherit",
  windowsHide: true,
});
if (docker.status !== 0) process.exit(docker.status ?? 1);
const pool = new Pool({ connectionString: owner.toString() });
const exists = await pool.query(
  "SELECT 1 FROM pg_roles WHERE rolname='practicechart_app'",
);
if (!exists.rowCount) {
  const password = decodeURIComponent(
    new URL(process.env.DATABASE_URL!).password,
  ).replaceAll("'", "''");
  await pool.query(
    `CREATE ROLE practicechart_app LOGIN PASSWORD '${password}'`,
  );
}
await pool.end();
for (const script of ["migrate", "grant-runtime", "seed"]) {
  const r = spawnSync(
    process.execPath,
    ["node_modules/tsx/dist/cli.mjs", `scripts/${script}.ts`],
    { stdio: "inherit", windowsHide: true },
  );
  if (r.status !== 0) process.exit(r.status ?? 1);
}
console.log("Local practice is ready. Run npm run dev.");
```

scripts/migrate.ts

```
import "dotenv/config";
import { Pool } from "pg";
import { readFile, readdir } from "node:fs/promises";
import { createHash } from "node:crypto";
const pool = new Pool({
  connectionString:
    process.env.DATABASE_URL_UNPOOLED || process.env.DATABASE_URL,
});
const target = new URL(
  process.env.DATABASE_URL_UNPOOLED || process.env.DATABASE_URL!,
);
if (!/^practicechart[_a-z0-9]+$/.test(target.pathname.slice(1)))
  throw Error("Migrations require a separate PracticeChart database.");
const c = await pool.connect();
try {
  await c.query("SELECT pg_advisory_lock(4001002)");
  await c.query(
    "CREATE TABLE IF NOT EXISTS migrations(name text PRIMARY KEY,sha256 text NOT NULL,applied_at
timestamptz NOT NULL DEFAULT now())",
  );
  for (const file of (await readdir("migrations"))
    .filter((f) => f.endsWith(".sql"))
    .sort()) {
    const sql = await readFile(`migrations/${file}`, "utf8");
    const hash = createHash("sha256").update(sql).digest("hex");
    const old = await c.query("SELECT sha256 FROM migrations WHERE name=$1", [
      file,
    ]);
    if (old.rowCount) {
      if (old.rows[0].sha256 !== hash)
        throw Error(`Migration changed: ${file}`);
      continue;
    }
    await c.query("BEGIN");
    try {
      await c.query(sql);
      await c.query("INSERT INTO migrations(name,sha256) VALUES($1,$2)", [
        file,
        hash,
      ]);
      await c.query("COMMIT");
      console.log(`Applied ${file}`);
    } catch (e) {
      await c.query("ROLLBACK");
      throw e;
    }
  }
} finally {
  await c.query("SELECT pg_advisory_unlock(4001002)");
  c.release();
  await pool.end();
}
```

scripts/performance.ts

```
import "dotenv/config";
import { Pool } from "pg";
import { chromium } from "@playwright/test";
import { cpus, totalmem, platform, release } from "node:os";
import { mkdir, writeFile, readFile } from "node:fs/promises";
import { hashPassword } from "../src/lib/auth";
const base = process.env.PERF_BASE_URL || "http://localhost:3000";
if (!["localhost", "127.0.0.1"].includes(new URL(base).hostname))
  throw Error("The load test requires a local presentation server.");
const duration = Number(process.env.PERF_SECONDS || 30);
if (duration < 10 || duration > 300)
  throw Error("Use a duration from 10 to 300 seconds per load.");
const owner = new URL(process.env.DATABASE_URL_UNPOOLED!);
if (!["localhost", "127.0.0.1"].includes(owner.hostname))
  throw Error("The load fixture script requires a local database.");
const pool = new Pool({ connectionString: owner.toString() });
const hash = await hashPassword("LoadDemo!2026");
await pool.query(
  "INSERT INTO users(username,name,role,password_hash) SELECT 'load_'||lpad(n::text,2,'0'),'Fictional Load Tester '||n,'doctor',$1 FROM generate_series(1,50)n ON CONFLICT(username) DO NOTHING",
  [hash],
);
await pool.query(
  "INSERT INTO patients(first_name,last_name,dob,phone,address) SELECT 'Load','Fictional '||lpad(n::text,4,'0'),'1990-01-01','876-555-0100','Fictional load test address' FROM generate_series(1,1000)n WHERE NOT EXISTS(SELECT 1 FROM patients WHERE first_name='Load')",
);
const dataset = {
  patients: Number(
    (await pool.query("SELECT count(*) FROM patients")).rows[0].count,
  ),
  clinicalRevisions: Number(
    (await pool.query("SELECT count(*) FROM clinical_revisions")).rows[0].count,
  ),
};
const version = (await pool.query("SELECT version()")).rows[0].version;
const target = (
  await pool.query(
    "SELECT id,first_name,last_name FROM patients WHERE first_name='Lena' LIMIT 1",
  )
).rows[0];
await pool.end();
const browser = await chromium.launch();
const results = [];
function summary(values: number[]) {
  const sorted = [...values].sort((a, b) => a - b);
  return {
    samples: values.length,
    minMs: Number((sorted[0] || 0).toFixed(2)),
    p50Ms: Number((sorted[Math.floor(sorted.length * 0.5)] || 0).toFixed(2)),
    p95Ms: Number(
      (
        sorted[
          Math.min(sorted.length - 1, Math.ceil(sorted.length * 0.95) - 1)
        ] || 0
      ).toFixed(2),
    ),
    maxMs: Number((sorted.at(-1) || 0).toFixed(2)),
  };
}
for (const concurrent of [1, 10, 50]) {
  const cookies: string[] = [];
  for (let i = 1; i < concurrent; i++) {
    const r = await fetch(base + "/api/login", {
      method: "POST",
      headers: { origin: base, "content-type": "application/json" },
      body: JSON.stringify({
        username: "load_" + String(i).padStart(2, "0"),
        password: "LoadDemo!2026",
      }),
    });
    if (!r.ok) throw Error("Load account login failed.");
    cookies.push(r.headers.get("set-cookie")!.split(";")[0]);
  }
```

```
  const context = await browser.newContext({
    viewport: { width: 1440, height: 1000 },
  });
  const page = await context.newPage();
  await page.goto(base + "/login");
  await page
    .getByLabel("Username", { exact: true })
    .fill("load_" + String(concurrent).padStart(2, "0"));
  await page.getByLabel("Password", { exact: true }).fill("LoadDemo!2026");
  const apiTimes: number[] = [],
    uiTimes: number[] = [];
  let errors = 0;
  const until = performance.now() + duration * 1000;
  let requests = 0;
  const workers = cookies.map(async (cookie) => {
    while (performance.now() < until) {
      for (const endpoint of [
        "/api/patients?q=Lena",
        "/api/patients/" + target.id,
      ]) {
        const start = performance.now();
        try {
          const r = await fetch(base + endpoint, { headers: { cookie } });
          await r.arrayBuffer();
          requests++;
          if (!r.ok) errors++;
          apiTimes.push(performance.now() - start);
        } catch {
          errors++;
        }
      }
      await new Promise((r) => setTimeout(r, 1000));
    }
  });
  const start = performance.now();
  await page.getByRole("button", { name: "Log in", exact: true }).click();

  await page.waitForURL("**/app");
  await page.getByRole("heading", { name: "Patient worklist" }).waitFor();
  const uiLoginMs = performance.now() - start;
  let first = true;
  let coldRecordMs = 0;
  while (performance.now() < until) {
    const start = performance.now();
    try {
      await page.goto(base + "/app/patients/" + target.id);
      await page
        .getByRole("heading", {
          name: target.first_name + " " + target.last_name,
          exact: true,
        })
        .first()
        .waitFor();
      await page.getByText("Medical record number", { exact: true }).waitFor();
      const elapsed = performance.now() - start;
      if (first) {
        coldRecordMs = elapsed;
        first = false;
      } else uiTimes.push(elapsed);
    } catch {
      errors++;
    }
    await page.waitForTimeout(1000);
  }
  await Promise.all(workers);
  results.push({
    concurrentSessions: concurrent,
    durationSeconds: duration,
    backgroundPattern:
      "Each background session requests patient search and patient detail, then waits 1 second.",
    uiPattern:
      "One authenticated browser opens the complete patient overview, then waits 1 second.",
    uiLoginMs: Number(uiLoginMs.toFixed(2)),
    firstRecordMs: Number(coldRecordMs.toFixed(2)),
    warmRecordDisplay: summary(uiTimes),
    api: summary(apiTimes),
    apiRequests: requests,
    errors,
```

```
    loginWithin3000ms: uiLoginMs < 3000,
    warmDisplaysWithin2000ms: uiTimes.filter((v) => v < 2000).length,
    totalWarmDisplays: uiTimes.length,
  });
  await context.close();
  console.log(`Measured ${concurrent} sessions. Errors: ${errors}.`);
}
await browser.close();
const pkg = JSON.parse(await readFile("package.json", "utf8"));
const report = {
  testedAt: new Date().toISOString(),
  environment: {
    target: base,
    server: "Local Next.js production build",
    platform: platform(),
    osRelease: release(),
    cpu: cpus()[0].model,
    logicalCpuCount: cpus().length,
    memoryGiB: Number((totalmem() / 1024 ** 3).toFixed(1)),
    node: process.version,
    next: pkg.dependencies.next,
    postgres: version,
    driver: "pg through Drizzle; maximum 5 connections per process",
    network: "Same Windows host; PostgreSQL 18 in Docker Desktop via localhost",
    dataset,
  },
  results,
  limits: [
    "This is a local test, not a cloud capacity guarantee.",
    "First record request is reported separately from later requests.",
    "A browser sample includes rendering. API timings do not.",
    "Only one browser login is measured at each load.",
    "This short test does not prove 99 percent availability.",
  ],
};
await mkdir("docs/evidence", { recursive: true });
await writeFile(
  "docs/evidence/performance.json",
  JSON.stringify(report, null, 2),
);
console.log(JSON.stringify(report, null, 2));
```

scripts/recovery-lib.ts

```
import "dotenv/config";
import { spawn } from "node:child_process";
import {
  generateKeyPairSync,
  randomBytes,
  publicEncrypt,
  privateDecrypt,
  createCipheriv,
  createDecipheriv,
  constants,
  createHash,
} from "node:crypto";
import { gzipSync, gunzipSync } from "node:zlib";
import { mkdir, readFile, writeFile } from "node:fs/promises";
import path from "node:path";
import { getCACertificates } from "node:tls";
export async function pgTool(
  tool: "pg_dump" | "pg_restore",
  url: string,
  args: string[],
  input?: Buffer,
) {
  const u = new URL(url);
  const docker =
    process.env.PGTOOLS_DRIVER === "docker" || process.platform === "win32";
  const remote = !["localhost", "127.0.0.1"].includes(u.hostname);
  const trustFile = path.resolve(".private/backup-trust.pem");
  if (docker && remote) {
    await mkdir(".private", { recursive: true });
    await writeFile(trustFile, getCACertificates("default").join("\n"));
  }
  const env = {
    ...process.env,
    PGHOST:
      docker && ["localhost", "127.0.0.1"].includes(u.hostname)
        ? "host.docker.internal"
        : u.hostname,
    PGPORT: u.port || "5432",
    PGUSER: decodeURIComponent(u.username),
    PGPASSWORD: decodeURIComponent(u.password),
    PGDATABASE: u.pathname.slice(1),
    PGSSLROOTCERT: remote ? (docker ? "/practicechart-ca.pem" : "system") : "",
    PGSSLMODE: ["localhost", "127.0.0.1"].includes(u.hostname)
      ? "disable"
      : "verify-full",
  };
  if (tool === "pg_restore")
    args = ["--dbname=" + u.pathname.slice(1), ...args];
  const binary = docker ? "docker" : tool;
  const command = docker
    ? [
        "run",
        "--rm",
        "-i",
        "-e",

        "PGHOST",
        "-e",
        "PGPORT",
        "-e",
        "PGUSER",
        "-e",
        "PGPASSWORD",
        "-e",
        "PGDATABASE",
        "-e",
        "PGSSLMODE",
        "-e",
        "PGSSLROOTCERT",
        ...(remote
          ? [
              "--mount",
              `type=bind,source=${trustFile},target=/practicechart-ca.pem,readonly`,
            ]
          : []),
```

```
        "postgres:18",
        tool,
        ...args,
      ]
    : args;
  return await new Promise<Buffer>((resolve, reject) => {
    const p = spawn(binary, command, { env, windowsHide: true });
    const chunks: Buffer[] = [];
    let diagnostic = "";
    p.stdout.on("data", (c) => chunks.push(c));
    p.stderr.on("data", (part) => {
      diagnostic += part.toString();
    });
    p.on("error", () => reject(Error(`${tool} could not start.`)));
    p.on("close", (code) =>
      code === 0
        ? resolve(Buffer.concat(chunks))
        : reject(
            Error(
              `${tool} failed with code ${code}. Connection details were not logged.
${diagnostic.replaceAll(env.PGPASSWORD, "[redacted]").replaceAll(env.PGHOST,
"[host]").replaceAll(env.PGUSER, "[user]").slice(0, 800)}`,
            ),
          ),
    );
    p.stdin.end(input);
  });
}
export async function recoveryKeys() {
  const root = ".private";
  await mkdir(root, { recursive: true });
  let publicKey: string;
  try {
    publicKey = await readFile(path.join(root, "recovery.pub"), "utf8");
  } catch {
    const keys = generateKeyPairSync("rsa", {

      modulusLength: 4096,
      publicKeyEncoding: { type: "spki", format: "pem" },
      privateKeyEncoding: { type: "pkcs8", format: "pem" },
    });
    await writeFile(path.join(root, "recovery.key"), keys.privateKey, {
      flag: "wx",
      mode: 0o600,
    });
    await writeFile(path.join(root, "recovery.pub"), keys.publicKey, {
      flag: "wx",
    });
    publicKey = keys.publicKey;
  }
  return publicKey;
}
export function encryptBundle(input: Buffer, publicKey: string) {
  const key = randomBytes(32),
    iv = randomBytes(12);
  const cipher = createCipheriv("aes-256-gcm", key, iv);
  const body = Buffer.concat([cipher.update(gzipSync(input)), cipher.final()]);
  const wrapped = publicEncrypt(
    {
      key: publicKey,
      oaepHash: "sha256",
      padding: constants.RSA_PKCS1_OAEP_PADDING,
    },
    key,
  );
  const metadata = Buffer.from(
    JSON.stringify({
      format: "PracticeChart full EHR recovery v1",
      key: wrapped.toString("base64"),
      iv: iv.toString("base64"),
      tag: cipher.getAuthTag().toString("base64"),
    }),
  );
  const size = Buffer.alloc(4);
  size.writeUInt32BE(metadata.length);
  return Buffer.concat([Buffer.from("PCBACKUP1"), size, metadata, body]);
}
export function decryptBundle(input: Buffer, privateKey: string) {
```

```
  if (input.subarray(0, 9).toString() !== "PCBACKUP1")
    throw Error("Unknown recovery format.");
  const size = input.readUInt32BE(9);
  const meta = JSON.parse(input.subarray(13, 13 + size).toString());
  const key = privateDecrypt(
    {
      key: privateKey,
      oaepHash: "sha256",
      padding: constants.RSA_PKCS1_OAEP_PADDING,
    },
    Buffer.from(meta.key, "base64"),
  );
  const decipher = createDecipheriv(
    "aes-256-gcm",

    key,
    Buffer.from(meta.iv, "base64"),
  );
  decipher.setAuthTag(Buffer.from(meta.tag, "base64"));
  return gunzipSync(
    Buffer.concat([
      decipher.update(input.subarray(13 + size)),
      decipher.final(),
    ]),
  );
}
export const sha = (bytes: Buffer) =>
  createHash("sha256").update(bytes).digest("hex");
```

scripts/refresh-sample.ts

```
import 'dotenv/config';
import {Pool}from 'pg';
import {createHash}from 'node:crypto';
import {samplePdf,storeFile}from '../src/lib/storage';
const pool=new Pool({connectionString:process.env.DATABASE_URL_UNPOOLED});const old=(await
pool.query("SELECT d.*,p.first_name,p.last_name,p.address FROM documents d JOIN patients p ON
p.id=d.patient_id WHERE d.title='Fictional lab report' ORDER BY d.created_at DESC LIMIT 1")).rows[0];
if(old&&old.first_name==='Lena'&&old.last_name==='Campbell'&&old.address.includes('Sample
Lane')&&old.sha256!==createHash('sha256').update(samplePdf).digest('hex')){const saved=await
storeFile(samplePdf,'application/pdf');const c=await pool.connect();try{await c.query('BEGIN');await
c.query("UPDATE documents SET title='Earlier draft sample report' WHERE id=$1",[old.id]);const
fresh=(await c.query('INSERT INTO
documents(patient_id,title,kind,pathname,mime,size,sha256,replaces_id,author_id)
VALUES($1,$2,$3,$4,$5,$6,$7,$8,$9) RETURNING id',[old.patient_id,'Fictional lab report','Lab
report',saved.pathname,saved.mime,saved.size,saved.sha256,old.id,old.author_id])).rows[0];await
c.query("INSERT INTO audit_events(action,entity,entity_id,metadata) VALUES('sample_report_replaced','docum
ent',$1,$2)",[fresh.id,JSON.stringify({replacesId:old.id,reason:'Fictional PDF fixture format
repaired.'})]);await c.query('COMMIT');console.log('Replaced the fictional PDF fixture. Earlier bytes and
metadata remain.');}catch(e){await c.query('ROLLBACK');throw e;}finally{c.release();}}else console.log('No
 fictional sample replacement is needed.');await pool.end();
```

scripts/regions.py

```
"""Count planar graph faces independently of the edge-node formula."""
import json
import os
import sys
from pathlib import Path
try:
    import networkx as nx
except ImportError:
    sys.path.insert(0, str(Path(os.environ.get('TEMP', '/tmp')) / 'practicechart-doc-libs'))
    import networkx as nx

path = Path('docs/evidence/control-flow.json')
graphs = json.loads(path.read_text())
lines = ['# Control flow and basis path analysis', '',
         'The generator reads the implemented TypeScript functions. It expands transaction callbacks. It does not expand database engines or library internals. SQL constraints have separate integration tests. Graphs use one entry and one exit. Compound conditions are split. Each explicit I/O try block has one fault branch.', '',
         'Face counts use a planar half-edge traversal. Parallel directed edges add a lens face. Predicate counts are taken from nodes with two outgoing edges. The third method uses the separate edge and node counts. These three results must agree.', '',
         '| Function | Nodes N | Edges E | Faces R | Predicates P | R | P + 1 | E - N + 2 |',
         '| --- | --- | --- | --- | --- | --- | --- | --- |']
for graph in graphs:
    g = nx.Graph()
    g.add_nodes_from(n['id'] for n in graph['nodes'])
    pairs = [tuple(sorted((e['from'], e['to']))) for e in graph['edges']]
    g.add_edges_from(pairs)
    planar, embedding = nx.check_planarity(g)
    if not planar:
        raise ValueError('Graph needs a planar presentation: ' + graph['name'])
    visited = set()
    faces = []
    for a, b in embedding.edges():
        if (a, b) not in visited:
            faces.append(embedding.traverse_face(a, b, visited))
    lenses = len(pairs) - len(set(pairs))
    regions = len(faces) + lenses
    graph['regionCount'] = regions
    graph['faceBoundaries'] = faces
    graph['parallelLensFaces'] = lenses
    assert regions == graph['predicateComplexity'] == graph['edgeNodeComplexity'], graph['name']
    n, e, p = len(graph['nodes']), len(graph['edges']), graph['predicates']
    lines.append(f"| {graph['name']} | {n} | {e} | {regions} | {p} | {regions} | {p + 1} | {e - n + 2} |")
lines += ['', '## Basis path candidates', '',
          'The following paths are independent in the generated graph. They are syntactic candidates, not a claim that every candidate is feasible or was executed. Branch conditions can depend on earlier state. For example, a stale revision cannot also reach a successful save in the same execution. Repeated file and medication paths depend on the number of records. Do not invent inputs to force an impossible path.', '',
          'Actual feasible scenarios and test outcomes are mapped in TEST_PLAN.md. Full possible-path coverage is not claimed. V8 statement and branch coverage is recorded separately. A loop can have more possible executions than its basis-path count.', '']
for graph in graphs:
    lines += [f"### {graph['name']}", '', f"Source: {graph['file']}:{graph['line']}.
[Graph](diagrams/cfg/{graph['name']}.svg).", '']

    for i, route in enumerate(graph['syntacticBasisCandidates'], 1):
        lines.append(f"- Candidate {i}: " + ' -> '.join(map(str, route)))
    lines += ['', 'Face boundaries: ' + '; '.join(' -> '.join(map(str, f)) for f in
graph['faceBoundaries']) + f". Additional parallel-edge lens faces: {graph['parallelLensFaces']}.", '']
path.write_text(json.dumps(graphs, indent=2))
Path('docs/CONTROL_FLOW.md').write_text('\n'.join(lines))
print(f'All three complexity methods agree for {len(graphs)} functions.')
```

scripts/report-content.ts

```
import {
  readFile,
  writeFile,
  copyFile,
  mkdir,
  readdir,
} from "node:fs/promises";
import { Pool } from "pg";
import "dotenv/config";
const read = async (file: string) => JSON.parse(await readFile(file, "utf8"));
const unit = await read("docs/evidence/vitest-results.json");
const browser = await read("docs/evidence/playwright-results.json");
const perf = await read("docs/evidence/performance.json");
const cold = await read("docs/evidence/cold-performance.json");
const restore = await read("docs/evidence/restore-result.json");
const coverage = await read("coverage/coverage-summary.json");
await copyFile(
  "coverage/coverage-summary.json",
  "docs/evidence/coverage-summary.json",
);
const p = new Pool({ connectionString: process.env.DATABASE_URL_UNPOOLED });
const cols = (
  await p.query(
    "SELECT table_name,column_name,data_type,is_nullable,column_default FROM information_schema.columns WHERE table_schema='public' ORDER BY table_name,ordinal_position",
  )
).rows;
await p.end();
const summary = `Vitest recorded ${unit.numPassedTests} passing tests and ${unit.numFailedTests} failed tests. Playwright recorded ${browser.stats.expected} passing browser tests and ${browser.stats.unexpected} unexpected failures. The measured Vitest library coverage was ${coverage.total.statements.pct}% of statements and ${coverage.total.branches.pct}% of branches. This is not a claim of full path coverage.`;
const performanceTable =
  "| Active sessions | Seconds | Browser login ms | First record ms | Warm display p95 ms | API requests | Request errors |\n| --- | --- | --- | --- | --- | --- | --- |\n" +
  perf.results
    .map(
      (r: {
        concurrentSessions: number;
        durationSeconds: number;
        uiLoginMs: number;
        firstRecordMs: number;
        warmRecordDisplay: { p95Ms: number };
        apiRequests: number;
        errors: number;
      }) =>
        `| ${r.concurrentSessions} | ${r.durationSeconds} | ${r.uiLoginMs} | ${r.firstRecordMs} | ${r.warmRecordDisplay.p95Ms} | ${r.apiRequests} | ${r.errors} |`,
    )
    .join("\n");
const development = `# PracticeChart software development and implementation report

## Introduction

PracticeChart is a working EHR course application for a private medical practice. It replaces the case study's paper record steps with role-controlled digital workflows. The intended users are doctors and front desk staff. Development, tests, screenshots, and the public demo use fictional records only. No live practice rollout, clinical approval, legal compliance, completed training, or human acceptance is claimed.

The project brief defines the EHR business scope. The assignment guide defines this report and the two manuals. The syllabus requires implementation, testing, metrics, maintenance, configuration management, and team participation. The old application and earlier change record were deleted. Their code and test results were not reused as EHR evidence.

## Scope and objectives

Support patient registration and retrieval, administrative updates, safe appointment scheduling, consultation records, diagnostic files, prescriptions, certificates, visit filing, manual invoices, and reports. Preserve earlier clinical entries and audit history. Provide an encrypted full backup and a measured restore test. Keep the local presentation data across restarts.

There is no patient portal, payment processor, AI clinical advice, biometric authentication, external medical practice connection, Ministry of Health connection, pharmacy integration, or laboratory
```

```
integration. Email delivery is deferred. In-app reminders are included. Hardware purchasing and staff
training are not represented as completed work.

## Requirements and constraints

REQUIREMENTS.md records source references, conflicts, and decisions. The scanned requirements include
unique usernames, role limits, audit history, daily backup, a three second login target, a two second
patient retrieval target, 50 concurrent users, and a 99 percent operational availability target. The short
 tests measure the first three performance conditions locally. They cannot prove long-term availability or
 legal compliance.

Development used a Windows computer, Node.js, Docker Desktop, PostgreSQL 18, and Chromium. The local
benchmark environment was ${perf.environment.cpu.trim()}, ${perf.environment.logicalCpuCount} logical
CPUs, ${perf.environment.memoryGiB} GiB RAM, Node ${perf.environment.node}, and Next.js
${perf.environment.next}. The cloud demo uses the existing Vercel project, separate Neon EHR databases,
and new private Blob paths. Free service limits remain a constraint.

Inputs include credentials, patient demographics, booking intervals, doctor-entered clinical data,
diagnostic files, and invoice lines. Outputs include confirmation and error messages, schedules, patient
history, prescriptions, medical certificates, visit summaries, attendance slips, invoices, audit history,
and recovery evidence. A patient report and full EHR backup are different outputs.

## Design and implementation process

The brief, assignment guide, syllabus, schedule, supporting lesson documents, and all 53 embedded images
were reviewed. Requirements were separated from old-system interviews, lesson examples, draft algorithms,
and unsupported statements. Source conflicts were recorded before implementation. The user selected manual
 invoices and in-app reminders.

The implementation used small server services, validation schemas, explicit role checks, SQL migrations,
database constraints, and a shared interface. The initial green sidebar design was rejected as too close
to the earlier application's appearance. It was replaced with ink-blue accents, warm light surfaces, top
navigation, a patient worklist, and segmented record controls. No deleted application source or saved
design reference was used.

Coding and automated checks were performed with AI assistance in this workspace. These files do not assign

 module authorship to group members. The group must review the implementation, record each member's actual
 code contribution, and complete the human course tasks in GROUP_START_GUIDE.md.

## Architectural design

The browser uses Next.js pages and authenticated route handlers. Zod validates inputs. Drizzle services
use PostgreSQL transactions. A restricted runtime database role cannot alter the schema or overwrite
protected history. Clinical and audit changes commit together. Private files are streamed only after role
checks. Workflow state contains IDs and result counts, not clinical bodies.

![Use case diagram](diagrams/png/use_case.png)

![Sequence diagram](diagrams/png/sequence.png)

![Swimlane diagram](diagrams/png/swimlane.png)

![Domain class and service diagram](diagrams/png/class.png)

![Input processing and output modules](diagrams/png/ipo.png)

The class diagram represents domain models and service operations. The implementation uses TypeScript
types, functions, and Drizzle table models. It does not pretend that every domain model is implemented as
an object-oriented class.

## Procedural design

MODULES.md contains source-linked processing narratives and graph instructions for 26 functions.
CONTROL_FLOW.md contains the three complexity calculations, planar face boundaries, and independent
syntactic candidates. TEST_PLAN.md maps feasible state and input cases to executable tests. Database
constraints are tested through PostgreSQL, rather than assigned invented JavaScript branch counts.

## Interface design

The Today page leads with a patient worklist. The patient record separates demographics, visits, doctor-
only clinical entries, and private documents. Shared forms show labels, required fields, errors, busy
states, and confirmation. Booking uses a review step before commit. Final clinical corrections require a
reason. Reduced-motion preferences remove motion. The phone view uses a compact bottom navigation.

![Doctor workspace](screenshots/doctor-overview.png)

![Phone workspace](screenshots/phone-overview.png)
```

```
## Testing and actual results

${summary}

Unit and database tests cover valid and invalid partitions, boundaries, session and account states, simultaneous bookings, stale changes, clinical corrections, document rules, and invoices. Complete browser tests cover both roles. Direct clinical and file routes are denied to the front desk. Axe recorded zero violations on the nine checked desktop pages and the checked emulated phone page. Automated scans do not replace a full accessibility review or a real phone test.

### Performance

The fixture contained ${perf.environment.dataset.patients} fictional patients and

${perf.environment.dataset.clinicalRevisions} clinical revisions. Each load ran for 30 seconds. At loads above one, each background session requested a patient search and detail, then waited one second. One browser session measured complete record display. The first display is reported separately from later displays. The first display is not a claim of a cold server start.

${performanceTable}

All measured browser logins were below three seconds. All measured warm record displays were below two seconds. Only one browser login was measured at each load. The server was a local production build. PostgreSQL ran in Docker on the same host. These results do not prove equivalent cloud performance or 99 percent availability.

A separate process-cold check used three fresh local production Node processes and fresh browser sessions. A health probe preceded login. PostgreSQL and operating system caches remained warm. This is not a Vercel cold start test. The measured browser login times were ${cold.results.map((r: { loginMs: number }) => r.loginMs).join(", ")} ms. First patient display times were ${cold.results.map((r: { firstPatientDisplayMs: number }) => r.firstPatientDisplayMs).join(", ")} ms. Repeat display times were ${cold.results.map((r: { warmPatientDisplayMs: number }) => r.warmPatientDisplayMs).join(", ")} ms. Three samples do not establish a percentile guarantee. The full method and results are in cold-performance.json.

### Recovery

The full encrypted EHR recovery bundle from the successful GitHub backup job restored into an isolated local PostgreSQL 18 database. The actual result was ${restore.status}. The test compared ${restore.tablesCompared} tables and ${restore.documentsVerified} uploaded documents. The measured restore phase took ${restore.durationSeconds} seconds. It compared all restored row values, file hashes and bytes, MRN and invoice sequencing, immutable histories, and overlap rejection. This duration excludes download, decryption, and the separate browser drill. The restored application also passed both role logins, patient search, and private file checks. RECOVERY.md explains key custody, daily scheduling, retention, and limits.

### Defects found and corrected

| Observed defect | Cause and correction | Retest |
| --- | --- | --- |
| Local login rejected a valid page origin | Internal host differed from the request host. Check the actual request host. | Both role logins pass |
| Available times changed after a newer date was selected | A late response replaced newer state. Ignore superseded responses. | Booking and rescheduling pass |
| Booking confirmation rejected a returned time | PostgreSQL timestamp output needed ISO normalisation. Normalise before submission. | Confirmation workflow passes |
| Saved textarea label lookup was inconsistent | Default text entered the nested label text. Add explicit accessible names. | Clinical correction workflow passes |
| Long bookings used a fixed half-hour availability interval | Check the full requested length, patient conflicts, and current reschedule row. Keep the existing visit length during rescheduling. | Duration, closing boundary, patient conflict, and reschedule tests pass |
| A searched patient was missing from the booking review | Keep the selected patient object and show its name and MRN. | Browser review assertion passes |
| A container did not trust the cloud certificate chain | Its trust store differed from Node. Supply the trusted Node CA set and retain hostname verification. | Cloud backup and restore evidence |

## Implementation method and maintenance

Use a phased conversion with a parallel reference process if the practice later approves real deployment. Begin with registration and scheduling. Add clinical recording and controlled document outputs after

training. This reduces the amount of change at one time. It also requires extra checks and a clear authoritative record. This is a proposed method, not a completed pilot.

Use versioned migrations, reviewed branches, a lockfile, regression tests, and a release record. Keep recovery keys and backups private. Review failed daily jobs. Check restore ability after schema or storage changes. Train staff before any real use. Keep a support contact and issue register.
```

```
## Human testing and remaining work

No completed user acceptance, staff training, live pilot, or real-device testing is claimed. Blank forms and a training plan are supplied. Each member must complete and explain a module contribution. Confirm the actual group presentation slot and private submission cover details with the tutor.

## References

University of Technology, Jamaica. (n.d.). Assignment guide [Course handout].

University of Technology, Jamaica. (n.d.). Software implementation and testing syllabus [Course outline].

University of Technology, Jamaica. (2026). Software implementation schedule 2026 to 2027 semester 1 [Course schedule].

Group 2. (2026). Software implementation group project semester 1 2026 to 2027 [Private project brief].

PostgreSQL Global Development Group. (2026). PostgreSQL documentation: Range types. https://www.postgresql.org/docs/current/rangetypes.html

Neon. (2026). Migrate data from Postgres with pg_dump and pg_restore. https://neon.com/docs/import/migrate-from-postgres

Vercel. (2026). Private storage. https://vercel.com/docs/vercel-blob/private-storage

## Appendices

See REQUIREMENTS.md, TEST_PLAN.md, CONTROL_FLOW.md, MODULES.md, RECOVERY.md, the diagrams, safe evidence files, programme manual, user manual, source listing, and blank training and acceptance forms. Private interviews and course chats are excluded from the public repository.
`;
await writeFile("docs/DEVELOPMENT_REPORT.md", development);
let programme =
  `# PracticeChart programme manual

This manual describes the implemented source, database, interfaces, file formats, and operating controls. It is for the group programmer and technical operator. Use the separate user manual for staff tasks.

## Source and architecture

The Next.js App Router hosts the login, workspace, report pages, and route handlers. The workspace is a client interface backed by authenticated server functions. Drizzle models define typed rows. SQL migrations add database-level invariants. The indexed source listing contains all authored application source, migrations, scripts, tests, and project configuration. Third-party packages and generated bundles are excluded.

The important source groups are src/app for routes and pages, src/components for interface modules, src/lib for authentication, validation, storage, and transactional services, and src/workflows for daily

work. scripts contains setup, migration, test, performance, course analysis, and recovery tools. The SQL migrations are the authority for constraints and triggers. Schema hashes prevent silent migration changes.

![Data design](diagrams/png/data.png)

![Data flow](diagrams/png/dfd.png)

## Database structure

The cloud uses a separate practicechart_ehr database. Preview uses practicechart_preview. The old ticket database is not part of this schema. Local tests use practicechart_test. Restore drills create a new timestamped local database.

| Table | Purpose |
| --- | --- |
| users | Unique staff login, role, password hash, active state, protected demo flag, and version |
| sessions | SHA-256 session token hash, account reference, and expiry |
| login_attempts | Hashed attempt key, count, and expiry |
| patients | Immutable MRN, demographics, provisional link, archive flag, and version |
| doctor_hours | Doctor weekday start and end minutes |
| appointments | Patient, doctor, interval, state, and version |
| schedule_blocks | Unavailable doctor intervals |
| visits | Patient, doctor, optional unique appointment, clinical revision number, and filing state |
| clinical_revisions | Immutable doctor content, revision, state, correction reason, and author |
| prescriptions | Exact doctor-entered medicine fields linked to a final revision |
| documents | Patient association, private path, MIME, size, digest, replacement reference, and author |
| invoices | Unique number, manual line items, integer-cent total, state, reason, and version |
| audit_events | Append-only actor, action, record reference, metadata, and time |
| reminders | Unique appointment and due date pair |
```

```
| backup_runs | Backup status, private bundle reference, and time |
| migrations | Applied migration names, hashes, and time |

MRNs and invoice numbers use independent sequences. The number formatting function does not truncate longer sequence values. Patient birth dates cannot exceed the Jamaica calendar day. Appointment intervals are positive and at most four hours. PostgreSQL GiST exclusion constraints reject active overlaps for both doctor and patient. Trigger checks serialize doctor availability changes and reject blocked or outside-hours bookings. A doctor account guard keeps at least one active doctor. Clinical, prescription, and audit histories reject update and delete operations.

The runtime role has no schema creation permission. It cannot update or delete protected history. The migration owner connection is separate from the runtime connection. Password hashes are never returned in staff API responses. Front desk patient queries omit clinical content and document metadata.

## Column dictionary

| Table | Column | PostgreSQL type | Nullable | Default |
| --- | --- | --- | --- | --- |
` +
  cols
    .map(
      (c) =>
        `| ${c.table_name} | ${c.column_name} | ${c.data_type} | ${c.is_nullable} | ${String(c.column_default || "").replaceAll("|", "/")} |`,
    )
    .join("\n") +

  `

## Server interfaces

All application data routes require a current session except login and health. The cron route requires its separate secret. Browser mutations must have the same origin as the request host. JSON uses camelCase. Validation errors return 400. Missing sessions return 401. Denied roles return 403. Missing records return 404. Stale changes, duplicate values, and overlaps return 409 where applicable. Unconfirmed service failures return 503. Private responses are not cached.

| Route group | Input contract and behaviour |
| --- | --- |
| POST /api/login | username and password; creates an eight-hour HTTP-only session cookie |
| POST /api/logout | closes the current server session and removes the cookie |
| POST /api/password | currentPassword and replacement password; closes existing sessions |
| GET or POST /api/patients | optional q search, or validated demographic fields |
| POST /api/patients/id | demographic fields and current version |
| POST /api/patients/id/archive | version and archived boolean; doctor only |
| POST /api/patients/id/reconcile | targetId and reason; doctor only |
| GET /api/availability | doctorId and Jamaica date |
| POST /api/appointments | patientId, doctorId, ISO startAt and endAt; confirms transactionally |
| POST /api/appointments/id | booking fields and current version; confirmed bookings only |
| POST /api/appointments/id/status | version and arrived or cancelled state |
| POST /api/visits | patientId, doctorId, optional matching appointmentId |
| POST /api/visits/id/clinical | expectedRevision, state, structured content, and correction reason; doctor only |
| POST /api/visits/id/file | expectedRevision; server requires the final diagnosis |
| GET or POST /api/documents | doctor-only listing or multipart patientId, title, kind, file, optional replacesId |
| GET /api/documents/id | authorised private file bytes, not a public storage URL |
| POST /api/invoices | patientId and description, quantity, unitCents line items |
| POST /api/invoices/id | version, issued or void state, and void reason |
| GET or POST /api/users | doctor-only account listing, creation, and versioned changes |
| GET /api/audit | doctor-only recent audit events |
| GET /api/reports/type | type-specific record ID and server role check |
| GET or POST /api/hours | read doctor hours; doctor-only guarded hour changes |
| POST /api/blocks | doctor-only interval creation or removal |
| GET /api/cron | secret-authenticated daily workflow start |

## Clinical and invoice payloads

Clinical content has symptoms, notes, diagnosis, treatment, allergies, vitals, labResults, medications, and certificate fields. Each medication has medication, dosage, route, frequency, duration, quantity, and instructions. The system neither infers nor recommends clinical content. Final entries require a diagnosis. A correction after a final entry requires a reason and another final revision.

Invoice lines contain description, positive integer quantity, and non-negative integer unitCents. The server calculates the total. Stored totals must fit the configured bound. Invoice states are draft, issued, and void. No payment settlement state or provider connection exists.

## File and external data structure
```

```

Diagnostic objects use a private practicechart/records or practicechart/preview prefix and a random UUID. Metadata is in PostgreSQL. Byte size, magic signature, claimed MIME, SHA-256, patient association, and optional replacement link are checked. The public demo accepts only the supplied fictional PDF. Earlier

files are kept.

Local files live under PRIVATE_STORAGE_DIR, outside public. Cloud files use private Vercel Blob. The server fetches the stored object only after account and role checks. The response uses an attachment filename and private no-store caching. There are no external medical, pharmacy, laboratory, biometric, or Ministry integrations.

The recovery format begins with PCBACKUP1, a four-byte metadata length, a JSON encryption header, and authenticated encrypted compressed payload bytes. The payload contains a custom-format PostgreSQL dump, file bytes, row counts, hashes, creation time, and database identification. Recovery keys and bundles are private. See RECOVERY.md and the source listing for exact encoding.

## Configuration and global data

Cloud functions prefer EHR_DATABASE_URL so the existing ticket connection settings can remain intact for rollback. Local commands use DATABASE_URL and DATABASE_URL_UNPOOLED. Other settings are APP_URL, DEMO_MODE, STORAGE_DRIVER, PRIVATE_STORAGE_DIR, BLOB_READ_WRITE_TOKEN, CRON_SECRET, and recovery settings. .env.example gives safe local values. Never put cloud values into NEXT_PUBLIC variables.

The process-wide PostgreSQL pool has at most five connections. The session cookie contains a random opaque token. PostgreSQL stores its SHA-256 hash. The session lifetime is eight hours. The practice time zone is America/Jamaica. Workflow runs pass record IDs and status counts. Patient note bodies and passwords do not belong in workflow state or logs.

## Installation and operation

Run npm ci, npm run setup, and npm run dev. For the presentation, stop development, build, and start the production server. The setup script preserves existing local data. Docker uses a named PostgreSQL volume. The upload folder must also be preserved. Group and operator instructions are in GROUP_START_GUIDE.md and RECOVERY.md.

## Verification

${summary}

The control flow appendix records actual source-linked graphs and calculations. The test plan identifies feasible path cases and limits. Safe evidence includes automated results, axe results, load results, and an actual restore result. No human test or training completion is inferred from these files.
`;
await writeFile("docs/PROGRAMME_MANUAL.md", programme);
await mkdir("docs/evidence", { recursive: true });
await writeFile(
  "docs/evidence/build-summary.json",
  JSON.stringify(
    {
      application: "PracticeChart EHR",
      generatedAt: new Date().toISOString(),
      unitTests: { passed: unit.numPassedTests, failed: unit.numFailedTests },
      browserTests: {
        passed: browser.stats.expected,
        failed: browser.stats.unexpected,
      },
      coverage: coverage.total,
      restore,
      performanceSource: "performance.json",

      humanAcceptance: "not performed",
      staffTraining: "planned, not completed",
      realDeviceTest: "pending",
    },
    null,
    2,
  ),
);
console.log(
  "Generated development report, programme manual, database dictionary, and evidence summary.",
);
```

scripts/restore-test.ts

```
import "dotenv/config";
import { Pool } from "pg";
import { mkdir, readFile, readdir, writeFile } from "node:fs/promises";
import path from "node:path";
import { performance } from "node:perf_hooks";
import assert from "node:assert/strict";
import { decryptBundle, pgTool, sha } from "./recovery-lib";
type Bundle = {
  format: string;
  createdAt: string;
  counts: Record<string, number>;
  rowHashes: Record<string, string>;
  dumpSha256: string;
  dump: string;
  files: { id: string; pathname: string; sha256: string; bytes: string }[];
};
const directory = process.env.BACKUP_DIRECTORY || "backups";
const source =
  process.argv[2] ||
  `${directory}/${(await readdir(directory))
    .filter((n) => n.endsWith(".pcbackup"))
    .sort()
    .at(-1)}`;
const privateKey = await readFile(
  process.env.BACKUP_PRIVATE_KEY_FILE || ".private/recovery.key",
  "utf8",
);
const payload = JSON.parse(
  decryptBundle(await readFile(source), privateKey).toString(),
) as Bundle;
assert.equal(payload.format, "practicechart-full-ehr-v1");
const owner = new URL(
  process.env.RESTORE_ADMIN_URL ||
    "postgresql://practicechart:local-owner-password@localhost:5438/practicechart_ehr",
);
if (!["localhost", "127.0.0.1"].includes(owner.hostname))
  throw Error("Restore tests must use an isolated local database.");
const name = "practicechart_restore_" + Date.now();
const admin = new Pool({ connectionString: owner.toString() });
await admin.query(`CREATE DATABASE ${name}`);
const target = new URL(owner);
target.pathname = "/" + name;
const start = performance.now();
const dump = Buffer.from(payload.dump, "base64");
assert.equal(sha(dump), payload.dumpSha256);
await pgTool(
  "pg_restore",
  target.toString(),
  ["--no-owner", "--no-privileges", "--exit-on-error", "--single-transaction"],
  dump,
);
const restored = new Pool({ connectionString: target.toString() });
try {
  for (const [t, count] of Object.entries(payload.counts)) {
    if (!/^[a-z_]+$/.test(t)) throw Error("Invalid manifest table.");

    assert.equal(
      Number((await restored.query(`SELECT count(*) FROM ${t}`)).rows[0].count),
      count,
      `Row count: ${t}`,
    );
    const rows = (
      await restored.query(
        `SELECT row_to_json(t)::text AS row FROM ${t} t ORDER BY row_to_json(t)::text`,
      )
    ).rows;
    assert.equal(
      sha(Buffer.from(rows.map((r: { row: string }) => r.row).join("\n"))),
      payload.rowHashes[t],
      `All restored values: ${t}`,
    );
  }
  const root = path.resolve(".private/restore-test", name);
  for (const f of payload.files) {
    assert.match(
```

```
      f.pathname,
      /^practicechart\/(records|preview)\/[a-f0-9-]{36}$/,
    );
    const b = Buffer.from(f.bytes, "base64");
    assert.equal(sha(b), f.sha256);
    await mkdir(path.dirname(path.join(root, f.pathname)), { recursive: true });
    await writeFile(path.join(root, f.pathname), b);
    assert.equal(sha(await readFile(path.join(root, f.pathname))), f.sha256);
  }
  await assert.rejects(
    restored.query("UPDATE clinical_revisions SET reason='overwrite test'"),
  );
  await assert.rejects(restored.query("DELETE FROM audit_events"));
  const a = (
    await restored.query(
      "SELECT * FROM appointments WHERE status IN('confirmed','arrived','in_consultation') LIMIT 1",
    )
  ).rows[0];
  if (a)
    await assert.rejects(
      restored.query(
        "INSERT INTO appointments(patient_id,doctor_id,start_at,end_at) VALUES($1,$2,$3,$4)",
        [a.patient_id, a.doctor_id, a.start_at, a.end_at],
      ),
    );
  const maximum = Number(
    (
      await restored.query(
        "SELECT coalesce(max(substring(mrn FROM 4)::bigint),0) AS n FROM patients",
      )
    ).rows[0].n,
  );
  const next = Number(
    (await restored.query("SELECT nextval('mrn_seq') AS n")).rows[0].n,
  );
  assert.ok(next > maximum, "MRN sequence remains ahead of existing records");

  const invoiceMaximum = Number(
    (
      await restored.query(
        "SELECT coalesce(max(substring(number FROM 5)::bigint),0) AS n FROM invoices",
      )
    ).rows[0].n,
  );
  const invoiceNext = Number(
    (await restored.query("SELECT nextval('invoice_seq') AS n")).rows[0].n,
  );
  assert.ok(
    invoiceNext > invoiceMaximum,
    "Invoice sequence remains ahead of existing records",
  );
  const result = {
    status: "passed",
    testedAt: new Date().toISOString(),
    sourceCreatedAt: payload.createdAt,
    kind: "Full EHR database and uploaded files",
    restoreTarget: "isolated local PostgreSQL 18 database",
    durationSeconds: Number(((performance.now() - start) / 1000).toFixed(3)),
    tablesCompared: Object.keys(payload.counts).length,
    documentsVerified: payload.files.length,
    checks: [
      "table row counts",
      "all restored row values, including MRNs and clinical revisions",
      "uploaded file hashes and bytes",
      "clinical history protection",
      "audit history protection",
      "overlapping appointment rejection",
      "MRN sequence",
      "invoice sequence",
    ],
    retainedLocalDatabase: name,
  };
  await mkdir("docs/evidence", { recursive: true });
  await writeFile(
    "docs/evidence/restore-result.json",
    JSON.stringify(result, null, 2),
  );
  console.log(JSON.stringify(result, null, 2));
```

```
} finally {
  await restored.end();
  await admin.end();
}
```

scripts/seed.ts

```
import "dotenv/config";
import { Pool } from "pg";
import { hashPassword } from "../src/lib/auth";
import { emptyClinical } from "../src/lib/validation";
import { samplePdf, storeFile } from "../src/lib/storage";
const target = new URL(
  process.env.DATABASE_URL_UNPOOLED || process.env.DATABASE_URL!,
);
if (!/^practicechart[_a-z0-9]+$/.test(target.pathname.slice(1)))
  throw Error("Fictional seeding requires a separate PracticeChart database.");
const pool = new Pool({
  connectionString:
    process.env.DATABASE_URL_UNPOOLED || process.env.DATABASE_URL,
});
const existing = await pool.query("SELECT count(*) FROM patients");
if (Number(existing.rows[0].count) > 0) {
  console.log("Seed skipped. Existing records are preserved.");
  await pool.end();
  process.exit(0);
}
const protectedAccount = process.env.DEMO_MODE === "true";
const doctor = (
  await pool.query(
    "INSERT INTO users(username,name,role,password_hash,protected) VALUES($1,$2,$3,$4,$5) RETURNING id",
    [
      "doctor",
      "Dr. Maya Bennett",
      "doctor",
      await hashPassword("DoctorDemo!2026"),
      protectedAccount,
    ],
  )
).rows[0].id;
await pool.query(
  "INSERT INTO users(username,name,role,password_hash,protected) VALUES($1,$2,$3,$4,$5)",
  [
    "frontdesk",
    "Jordan Ellis",
    "front_desk",
    await hashPassword("DeskDemo!2026"),
    protectedAccount,
  ],
);
await pool.query(
  "INSERT INTO doctor_hours(doctor_id,weekday,start_min,end_min) SELECT $1,n,480,1020 FROM
generate_series(1,5)n",
  [doctor],
);
const names = [
  ["Lena", "Campbell"],
  ["Noah", "Sterling"],
  ["Amara", "Ellis"],
  ["Theo", "Morgan"],
  ["Isla", "Reid"],
  ["Eli", "Brooks"],
  ["Mila", "Grant"],
  ["Jude", "Foster"],
  ["Zoe", "Palmer"],
  ["Owen", "Hart"],
  ["Nia", "Clarke"],
  ["Leo", "Hayes"],
];
const ids: string[] = [];
for (let i = 0; i < names.length; i++) {
  const r = await pool.query(
    "INSERT INTO patients(first_name,last_name,dob,phone,address,gender,emergency_contact)
VALUES($1,$2,$3,$4,$5,$6,$7) RETURNING id",
    [
      ...names[i],
      `${1984 + i}-05-${String(10 + i).padStart(2, "0")}`,
      "876-555-01" + String(i).padStart(2, "0"),
      `${i + 1} Sample Lane, Demo Town`,
      "Not stated",
      "Fictional contact: 876-555-0199",
```

```
    ],
  );
  ids.push(r.rows[0].id);
}
let day = new Intl.DateTimeFormat("en-CA", {
  timeZone: "America/Jamaica",
}).format(new Date());
const d = new Date(day + "T12:00:00-05:00");
if (d.getUTCDay() === 0 || d.getUTCDay() === 6) {
  d.setUTCDate(d.getUTCDate() + (d.getUTCDay() === 6 ? 2 : 1));
  day = d.toISOString().slice(0, 10);
}
for (let i = 0; i < 5; i++)
  await pool.query(
    "INSERT INTO appointments(patient_id,doctor_id,start_at,end_at,status) VALUES($1,$2,$3,$4,$5)",
    [
      ids[i],
      doctor,
      `${day}T${String(9 + Math.floor(i / 2)).padStart(2, "0")}:${i % 2 ? "30" : "00"}:00-05:00`,
      `${day}T${String(9 + Math.floor((i + 1) / 2)).padStart(2, "0")}:${i % 2 ? "00" : "30"}:00-05:00`,
      i === 0 ? "completed" : i === 1 ? "arrived" : "confirmed",
    ],
  );
for (let i = 0; i < 3; i++) {
  const status = i === 0 ? "filed" : i === 1 ? "ready" : "open";
  const v = (
    await pool.query(
      "INSERT INTO visits(patient_id,doctor_id,status,revision,visit_at,filed_at)
VALUES($1,$2,$3,$4,now()-make_interval(days=>$5),CASE WHEN $3='filed' THEN now() ELSE NULL END) RETURNING
id",
      [ids[i], doctor, status, i === 2 ? 0 : 1, i + 1],
    )
  ).rows[0].id;
  if (i < 2)
    await pool.query(
      "INSERT INTO clinical_revisions(visit_id,revision,author_id,state,content)

VALUES($1,1,$2,'final',$3)",
      [
        v,
        doctor,
        JSON.stringify({
          ...emptyClinical,
          symptoms: "Fictional follow-up visit.",
          notes: "Sample record for course demonstration only.",
          diagnosis: "Sample diagnosis entered by the demonstration doctor.",
          treatment: "Sample treatment record. Not medical advice.",
          allergies: "Not recorded in this fictional case.",
          certificate:
            "This fictional certificate shows the document format only.",
        }),
      ],
    );
}
await pool.query(
  "INSERT INTO invoices(patient_id,lines,total_cents,status) VALUES($1,$2,350000,'issued')",
  [
    ids[0],
    JSON.stringify([
      { description: "Sample consultation", quantity: 1, unitCents: 350000 },
    ]),
  ],
);
const file = await storeFile(samplePdf, "application/pdf");
await pool.query(
  "INSERT INTO documents(patient_id,title,kind,pathname,mime,size,sha256,author_id)
VALUES($1,$2,$3,$4,$5,$6,$7,$8)",
  [
    ids[0],
    "Fictional lab report",
    "Lab report",
    file.pathname,
    file.mime,
    file.size,
    file.sha256,
    doctor,
  ],
);
```

```
await pool.query(
  "INSERT INTO audit_events(actor_id,action,entity,metadata)
  VALUES($1,'seed','demo','{\"fictional\":true}')",
  [doctor],
);
await pool.end();
console.log(
  "Fictional seed complete. doctor / DoctorDemo!2026 and frontdesk / DeskDemo!2026",
);
```

scripts/test-run.ts

```
import "dotenv/config";
import { Pool } from "pg";
import { spawnSync } from "node:child_process";
const original = new URL(process.env.DATABASE_URL_UNPOOLED!);
if (!["localhost", "127.0.0.1"].includes(original.hostname))
  throw Error("Tests require a local Docker database.");
const p = new Pool({ connectionString: original.toString() });
await p.query("DROP DATABASE IF EXISTS practicechart_test WITH (FORCE)");
await p.query("CREATE DATABASE practicechart_test");
await p.end();
const owner = new URL(original);
owner.pathname = "/practicechart_test";
const runtime = new URL(process.env.DATABASE_URL!);
if (
  !["localhost", "127.0.0.1"].includes(runtime.hostname) ||
  runtime.port !== original.port
)
  throw Error(
    "Test runtime and owner connections must use the same local PostgreSQL server.",
  );
runtime.pathname = "/practicechart_test";
const env = {
  ...process.env,
  DATABASE_URL: runtime.toString(),
  EHR_DATABASE_URL: runtime.toString(),
  DATABASE_URL_UNPOOLED: owner.toString(),
  PRIVATE_STORAGE_DIR: ".private/test-uploads",
  DEMO_MODE: "false",
  STORAGE_DRIVER: "local",
};
for (const args of [
  ["node_modules/tsx/dist/cli.mjs", "scripts/migrate.ts"],
  ["node_modules/tsx/dist/cli.mjs", "scripts/grant-runtime.ts"],
  ["node_modules/tsx/dist/cli.mjs", "scripts/seed.ts"],
  ["node_modules/vitest/vitest.mjs", "run", "--coverage"],
]) {
  const result = spawnSync(process.execPath, args, { env, stdio: "inherit" });
  if (result.status !== 0) process.exit(result.status ?? 1);
}
```

src/app/api/[...path]/route.ts

```ts
import { NextRequest, NextResponse } from "next/server";
import { and, eq, sql } from "drizzle-orm";
import { z } from "zod";
import { db } from "@/lib/db";
import { users, sessions, documents, patients } from "@/lib/schema";
import {
  AppError,
  getSession,
  login,
  SESSION_COOKIE,
  digest,
  requireRole,
} from "@/lib/auth";
import * as service from "@/lib/service";
import { readPrivateFile, storeFile, samplePdf } from "@/lib/storage";
import { idSchema } from "@/lib/validation";
import { start } from "workflow/api";
import { dailyPracticeTasks } from "@/workflows/daily";
export const runtime = "nodejs";
export const dynamic = "force-dynamic";
function camel(value: unknown): unknown {
  if (value instanceof Date) return value.toISOString();
  if (Array.isArray(value)) return value.map(camel);
  if (value && typeof value === "object")
    return Object.fromEntries(
      Object.entries(value).map(([k, v]) => [
        k.replace(/_([a-z])/g, (_, c: string) => c.toUpperCase()),
        camel(v),
      ]),
    );
  return value;
}
function json(value: unknown, status = 200) {
  return NextResponse.json(camel(value), {
    status,
    headers: { "Cache-Control": "private, no-store" },
  });
}
export async function GET(req: NextRequest) {
  return handle(req);
}
export async function POST(req: NextRequest) {
  return handle(req);
}
export async function PATCH(req: NextRequest) {
  return handle(req);
}
async function handle(req: NextRequest) {
  try {
    const route = req.nextUrl.pathname.replace(/^\/api\//, "");
    const parts = route.split("/");
    const id = parts[1];
    const post = req.method !== "GET";
    if (route === "health")
      return json({ status: "ok", application: "PracticeChart EHR" });
    if (route === "cron" && req.method === "GET") {
      if (
        !process.env.CRON_SECRET ||
        req.headers.get("authorization") !== `Bearer ${process.env.CRON_SECRET}`
      )
        throw new AppError(401, "Access denied.");
      const run = await start(dailyPracticeTasks, []);
      return json({ started: true, runId: run.runId });
    }
    if (post) {
      const origin = req.headers.get("origin");
      const host = req.headers.get("host") || req.nextUrl.host;
      if (!origin || origin !== `${req.nextUrl.protocol}//${host}`)
        throw new AppError(403, "Open the form on this website and try again.");
      if (Number(req.headers.get("content-length") || 0) > 4400000)
        throw new AppError(413, "The upload is too large.");
    }
    if (route === "login" && post) {
      const d = z
```

```
        .object({
          username: z.string().min(1).max(40),
          password: z.string().min(1).max(128),
        })
        .parse(await req.json());
      const result = await login(
        d.username,
        d.password,
        req.headers.get("x-forwarded-for")?.split(",")[0].trim() || "local",
      );
      const res = json(result.user);
      res.cookies.set(SESSION_COOKIE, result.token, {
        httpOnly: true,
        secure: req.nextUrl.protocol === "https:",
        sameSite: "lax",
        path: "/",
        maxAge: 28800,
      });
      return res;
    }
    const token = req.cookies.get(SESSION_COOKIE)?.value;
    const actor = await getSession(token);
    if (!actor) throw new AppError(401, "Please log in to continue.");
    if (route === "logout" && post) {
      await db.transaction(async (tx) => {
        await tx.delete(sessions).where(eq(sessions.id, digest(token!)));
        await service.event(tx, actor, "logout", "session");
      });
      const r = json({ ok: true });
      r.cookies.delete(SESSION_COOKIE);
      return r;
    }
    if (route === "me")
      return json({ user: actor, demo: process.env.DEMO_MODE === "true" });
    if (route === "password" && post)
      return json(await service.changePassword(actor, await req.json()));

    if (route === "dashboard" && !post)
      return json(await service.dashboard(actor));
    if (route === "doctors" && !post)
      return json(
        await db
          .select({ id: users.id, name: users.name })
          .from(users)
          .where(and(eq(users.role, "doctor"), eq(users.active, true))),
      );
    if (route === "patients")
      return json(
        post
          ? await service.savePatient(actor, await req.json())
          : await service.patientList(
              actor,
              req.nextUrl.searchParams.get("q") || "",
            ),
      );
    if (parts[0] === "patients" && id) {
      if (parts[2] === "archive" && post)
        return json(await service.archivePatient(actor, id, await req.json()));
      if (parts[2] === "reconcile" && post)
        return json(
          await service.reconcilePatient(actor, id, await req.json()),
        );
      return json(
        post
          ? await service.savePatient(actor, await req.json(), id)
          : await service.patientDetail(actor, id),
      );
    }
    if (route === "appointments")
      return json(
        post
          ? await service.bookAppointment(actor, await req.json())
          : await service.schedule(
              actor,
              req.nextUrl.searchParams.get("date") || undefined,
            ),
      );
    if (parts[0] === "appointments" && post)
```

```
      return json(
        parts[2] === "status"
          ? await service.appointmentStatus(actor, id, await req.json())
          : await service.bookAppointment(actor, await req.json(), id),
      );
    if (route === "availability" && !post)
      return json(
        await service.availability(
          actor,
          req.nextUrl.searchParams.get("doctorId") || "",
          req.nextUrl.searchParams.get("date") || "",
          {
            duration: Number(req.nextUrl.searchParams.get("duration") || 30),
            patientId: req.nextUrl.searchParams.get("patientId") || undefined,
            excludeId: req.nextUrl.searchParams.get("excludeId") || undefined,
          },
        ),
      );
    if (route === "visits" && post)
      return json(await service.startVisit(actor, await req.json()));
    if (parts[0] === "visits" && post) {
      if (parts[2] === "clinical")
        return json(await service.saveClinical(actor, id, await req.json()));
      if (parts[2] === "file")
        return json(await service.fileVisit(actor, id, await req.json()));
    }
    if (route === "invoices")
      return json(
        post
          ? await service.saveInvoice(actor, await req.json())
          : await service.invoiceList(actor),
      );
    if (parts[0] === "invoices" && post)
      return json(await service.setInvoiceStatus(actor, id, await req.json()));
    if (route === "users") {
      await requireRole(actor);
      return json(
        post
          ? await service.manageUser(actor, await req.json())
          : await db
              .select({
                id: users.id,
                username: users.username,
                name: users.name,
                role: users.role,
                active: users.active,
                protected: users.protected,
                version: users.version,
              })
              .from(users)
              .orderBy(users.name),
      );
    }
    if (parts[0] === "users" && post)
      return json(await service.manageUser(actor, await req.json(), id));
    if (route === "audit" && !post) {
      await requireRole(actor);
      await service.event(db, actor, "view", "audit");
      return json(
        await service.rows(
          sql`SELECT a.id,a.action,a.entity,a.entity_id,a.metadata,a.created_at,u.name AS actor_name FROM audit_events a LEFT JOIN users u ON u.id=a.actor_id ORDER BY a.created_at DESC LIMIT 200`,
        ),
      );
    }
    if (route === "hours") {
      if (!post)
        return json({
          hours: await service.rows(
            sql`SELECT h.*,u.name AS doctor_name FROM doctor_hours h JOIN users u ON u.id=h.doctor_id ORDER BY u.name,h.weekday`,
          ),
          blocks: await service.rows(
            sql`SELECT b.*,u.name AS doctor_name FROM schedule_blocks b JOIN users u ON u.id=b.doctor_id WHERE end_at>now() ORDER BY start_at`,
          ),
```

```
        });
      await requireRole(actor);
      const d = z
        .object({
          doctorId: idSchema,
          weekday: z.number().int().min(1).max(7),
          startMin: z.number().int().min(0).max(1439),
          endMin: z.number().int().min(1).max(1440),
        })
        .refine((x) => x.endMin > x.startMin)
        .parse(await req.json());
      return json(
        await db.transaction(async (tx) => {
          await tx.execute(
            sql`SELECT id FROM users WHERE id=${d.doctorId}::uuid FOR UPDATE`,
          );
          const conflicts = await service.rows(
            sql`SELECT id FROM appointments WHERE doctor_id=${d.doctorId}::uuid AND start_at>now() AND
    status IN('confirmed','arrived','in_consultation') AND extract(isodow FROM start_at AT TIME ZONE
    'America/Jamaica')=${d.weekday} AND (extract(epoch FROM (start_at AT TIME ZONE
    'America/Jamaica')::time)/60<${d.startMin} OR extract(epoch FROM (end_at AT TIME ZONE
    'America/Jamaica')::time)/60>${d.endMin})`,
            tx,
          );
          if (conflicts.length)
            throw new AppError(
              409,
              "Move conflicting appointments before you change these hours.",
            );
          await tx.execute(
            sql`INSERT INTO doctor_hours(doctor_id,weekday,start_min,end_min)
    VALUES(${d.doctorId},${d.weekday},${d.startMin},${d.endMin}) ON CONFLICT(doctor_id,weekday) DO UPDATE SET
    start_min=excluded.start_min,end_min=excluded.end_min`,
          );
          await service.event(tx, actor, "update", "doctor_hours", d.doctorId);
          return { ok: true };
        }),
      );
    }
    if (route === "blocks" && post) {
      await requireRole(actor);
      const d = z
        .object({
          doctorId: idSchema,
          startAt: z.string().datetime({ offset: true }),
          endAt: z.string().datetime({ offset: true }),
        })
        .refine((v) => Date.parse(v.endAt) > Date.parse(v.startAt))
        .parse(await req.json());
      return json(
        await db.transaction(async (tx) => {
          const r = await service.rows(
            sql`INSERT INTO schedule_blocks(doctor_id,start_at,end_at)
    VALUES(${d.doctorId},${d.startAt}::timestamptz,${d.endAt}::timestamptz) RETURNING id`,
            tx,
          );
          await service.event(
            tx,
            actor,
            "create",
            "schedule_block",
            String(r[0].id),
          );
          return r[0];
        }),
      );
    }
    if (parts[0] === "blocks" && post) {
      await requireRole(actor);
      return json(
        await db.transaction(async (tx) => {
          await tx.execute(
            sql`DELETE FROM schedule_blocks WHERE id=${idSchema.parse(id)}::uuid`,
          );
          await service.event(tx, actor, "delete", "schedule_block", id);
          return { ok: true };
        }),
```

```
      );
    }
    if (parts[0] === "reports" && !post)
      return json(
        await service.reportData(
          actor,
          id,
          req.nextUrl.searchParams.get("id") || "",
        ),
      );
    if (route === "sample-report" && !post) {
      await requireRole(actor);
      return new NextResponse(new Uint8Array(samplePdf), {
        headers: {
          "Content-Type": "application/pdf",
          "Content-Disposition":
            'attachment; filename="fictional-lab-report.pdf"',
          "Cache-Control": "no-store",
        },
      });
    }
    if (parts[0] === "documents") {
      await requireRole(actor);
      if (id && !post) {
        const [d] = await db
          .select()
          .from(documents)
          .where(eq(documents.id, idSchema.parse(id)));
        if (!d) throw new AppError(404, "The document was not found.");
        const bytes = await readPrivateFile(d.pathname);
        await service.event(db, actor, "retrieve", "document", id);
        return new NextResponse(new Uint8Array(bytes), {
          headers: {
            "Content-Type": d.mime,
            "Content-Disposition": `attachment; filename="${d.id}.${d.mime === "application/pdf" ? "pdf" : d.mime === "image/png" ? "png" : "jpg"}"`,
            "Cache-Control": "private, no-store",
          },
        });
      }
      if (post) {
        const form = await req.formData();
        const p = idSchema.parse(form.get("patientId"));
        const title = z
          .string()
          .trim()
          .min(1)
          .max(160)
          .parse(form.get("title"));
        const kind = z
          .enum(["Lab report", "Diagnostic report"])
          .parse(form.get("kind"));
        const replacesId = form.get("replacesId")
          ? idSchema.parse(form.get("replacesId"))
          : undefined;
        const file = form.get("file");
        if (!(file instanceof File)) throw new AppError(400, "Choose a file.");
        const [patient] = await db
          .select({ id: patients.id })
          .from(patients)
          .where(eq(patients.id, p));
        if (!patient) throw new AppError(404, "The patient was not found.");
        if (replacesId) {
          const [old] = await db
            .select()
            .from(documents)
            .where(eq(documents.id, replacesId));
          if (!old || old.patientId !== p)
            throw new AppError(
              400,
              "The replaced document must belong to this patient.",
            );
        }
        const saved = await storeFile(
          Buffer.from(await file.arrayBuffer()),
          file.type,
        );
```

```
        const result = await db.transaction(async (tx) => {
          const [d] = await tx
            .insert(documents)
            .values({
              patientId: p,
              title,
              kind,
              replacesId,
              authorId: actor.id,
              ...saved,
            })
            .returning({ id: documents.id, title: documents.title });
          await service.event(tx, actor, "upload", "document", d.id);
          return d;
        });
        return json(result, 201);
      }
      await service.event(db, actor, "view", "document_list");
      return json(
        await service.rows(
          sql`SELECT d.id,d.patient_id,d.title,d.kind,d.mime,d.size,d.created_at,d.replaces_id,p.first_name,p.last_name,p.mrn FROM documents d JOIN patients p ON p.id=d.patient_id ORDER BY d.created_at DESC LIMIT 100`,
        ),
      );
    }
    throw new AppError(404, "This function was not found.");
  } catch (error) {
    if (error instanceof AppError)
      return json({ error: error.message }, error.status);
    if (error instanceof z.ZodError)
      return json(
        { error: error.issues[0]?.message || "Check the form values." },
        400,
      );
    let cause = error as { code?: string; cause?: unknown };
    for (let i = 0; i < 5 && cause?.cause; i++)
      cause = cause.cause as typeof cause;
    const code = cause?.code;
    if (code === "23P01")
      return json(
        {
          error:
            "This time conflicts with another booking. Choose another time.",
        },
        409,
      );
    if (code === "23505")
      return json(
        {
          error:
            "This record already exists. Reload and check the existing record.",
        },
        409,
      );
    if (code === "23514")
      return json(
        {
          error:
            "The change breaks a record rule. Check doctor hours, booking limits, and the last active doctor.",
        },
        400,
      );
    console.error("PracticeChart request failed", {
      code: code || "internal",
      route: req.nextUrl.pathname.replace(/[a-f0-9-]{36}/g, ":id"),
    });
    return json(
      {
        error:
          "The service is not available. No successful save was confirmed. Try again.",
      },
      503,
    );
  }
```

```
444 }
```

src/app/app/[[...path]]/page.tsx

```
import { cookies } from "next/headers";
import { redirect } from "next/navigation";
import { getSession, SESSION_COOKIE } from "@/lib/auth";
import Workspace from "@/components/workspace";
export const dynamic = "force-dynamic";
export default async function AppPage() {
  const actor = await getSession((await cookies()).get(SESSION_COOKIE)?.value);
  if (!actor) redirect("/login");
  return <Workspace actor={actor} demo={process.env.DEMO_MODE === "true"} />;
}
```

src/app/globals.css

```
@import "tailwindcss";
:root {
  --ink: oklch(0.25 0.025 165);
  --muted: oklch(0.47 0.015 165);
  --brand: oklch(0.34 0.055 165);
  --brand-hover: oklch(0.28 0.05 165);
  --paper: oklch(0.977 0.005 150);
  --surface: oklch(0.998 0.002 150);
  --line: oklch(0.9 0.01 155);
  --soft: oklch(0.94 0.015 150);
  --mint: oklch(0.91 0.055 155);
  --danger: oklch(0.45 0.16 25);
  --ease: cubic-bezier(0.23, 1, 0.32, 1);
  --space-sm: 8px;
  --space-md: 16px;
  --space-lg: 24px;
  --space-xl: 32px;
}
* {
  box-sizing: border-box;
}
html {
  -webkit-tap-highlight-color: transparent;
  scroll-behavior: smooth;
}
body {
  margin: 0;
  color: var(--ink);
  background: var(--paper);
  font-family: var(--font-geist), Arial, sans-serif;
  font-size: 14px;
  line-height: 1.5;
}
button,
input,
textarea,
select {
  font: inherit;
}
button,
a {
  -webkit-user-select: none;
  user-select: none;
  touch-action: manipulation;
}
button {
  cursor: pointer;
}
button:disabled {
  cursor: wait;
  opacity: 0.55;
}
a {
  color: inherit;
  text-decoration: none;

}
button,
input,
textarea,
select,
a {
  outline-offset: 4px;
}
*:focus-visible {
  outline: 3px solid #387f6a;
}
button:active,
.button:active {
  transform: scale(0.98);
}
button {
  transition:
    background 120ms var(--ease),
    transform 120ms var(--ease);
```

```
}
h1,
h2,
h3,
p {
  margin: 0;
}
h1,
h2,
h3 {
  font-weight: 550;
  letter-spacing: -0.035em;
}
h1 {
  font-size: 32px;
  line-height: 1.2;
}
h2 {
  font-size: 23px;
}
h3 {
  font-size: 17px;
}
small {
  font-size: 12px;
}
.muted {
  color: var(--muted);
}
.eyebrow {
  font-size: 11px;
  letter-spacing: 0.15em;
  font-weight: 600;
}
.brand {
  display: flex;
  align-items: center;
  gap: 12px;
  font-size: 21px;
  font-weight: 550;
  letter-spacing: -0.5px;
}
.brand-mark {
  height: 40px;
  width: 40px;
  background: var(--mint);
  border-radius: 12px;
  display: flex;
  align-items: center;
  justify-content: center;
  font-size: 34px;
  line-height: 1;
  color: var(--brand);
  font-weight: 650;
  letter-spacing: -4px;
  padding-right: 6px;
}
.brand-mark > span {
  font-size: 20px;
  transform: translateY(-7px);
  letter-spacing: 0;
}
.brand-sub {
  display: block;
  font-size: 8px;
  letter-spacing: 2px;
  font-weight: 400;
  margin-top: 2px;
}
.button {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  gap: 8px;
  min-height: 44px;
  padding: 10px 16px;
  border-radius: 8px;
```

```
  border: 1px solid var(--line);
  background: var(--surface);
  font-weight: 550;
  white-space: nowrap;
}
.primary {
  background: var(--brand);
  color: var(--surface);
  border-color: var(--brand);
}
.danger {
  color: var(--danger);
}
.text-button {
  border: 0;
  background: transparent;
  padding: 8px;
  min-height: 44px;
  color: var(--brand);
  font-weight: 550;
  display: inline-flex;
  align-items: center;
  gap: 6px;
}
.icon-button {
  border: 1px solid var(--line);
  border-radius: 8px;
  width: 44px;
  height: 44px;
  background: var(--surface);
  display: inline-grid;
  place-items: center;
}
.status {
  display: inline-flex;
  align-items: center;
  gap: 6px;
  font-size: 11px;
  font-weight: 550;
  padding: 4px 9px;
  border-radius: 5px;
  background: var(--soft);
  color: var(--brand);
  white-space: nowrap;
}
.status::before {
  content: "";
  width: 5px;
  height: 5px;
  border-radius: 50%;
  background: currentColor;
}
.status.cancelled,
.status.void {
  background: #faeae6;
  color: #873b32;
}
.status.arrived,
.status.ready {
  background: #e5f3e9;
  color: #286345;
}
.status.completed,
.status.filed {
  background: #edf0f4;
  color: #526076;
}
.status.in_consultation {
  background: #f8efdc;
  color: #76571e;
}
.avatar {
  display: inline-grid;
  place-items: center;
  width: 38px;
  height: 38px;
```

```
  border-radius: 50%;
  background: #e6ece5;
  color: #365740;
  font-size: 12px;
  font-weight: 600;
  flex-shrink: 0;
}
.avatar.large {
  width: 64px;
  height: 64px;
  font-size: 21px;
}
.person {
  display: flex;
  align-items: center;
  gap: 12px;
}
.person strong {
  font-weight: 550;
}
.person small {
  display: block;
  color: var(--muted);
  margin-top: 2px;
}
.error {
  color: var(--danger);
  background: #fff0ec;
  border: 1px solid #ecc7be;
  padding: 12px;
  border-radius: 8px;
}
.success {
  color: var(--brand);
  background: var(--soft);
  padding: 12px 16px;
  border: 1px solid var(--line);
  border-radius: 8px;
}
.login-layout {
  min-height: 100svh;
  display: grid;
  grid-template-columns: 1fr 1fr;
}
.login-story {
  background: var(--brand);
  color: #f4f8f2;
  padding: 44px 56px;
  display: flex;
  flex-direction: column;
  position: relative;
  overflow: hidden;
}
.login-story::after {
  content: "";
  width: 600px;
  height: 600px;
  border: 1px solid #ffffff10;
  border-radius: 50%;
  position: absolute;
  bottom: -250px;
  right: -250px;
  pointer-events: none;
}
.story-content {
  margin: auto 0;
  padding: 72px 0;
  max-width: 480px;
}
.story-content .eyebrow {
  color: #bfd7ca;
}
.story-content h1 {
  font-size: clamp(40px, 4.5vw, 68px);
  line-height: 1.08;
  margin: 24px 0;
}
```

```
.story-content > p {
  font-size: 16px;
  max-width: 370px;
  color: #d0dfd6;
  line-height: 1.7;
}
.login-story footer {
  font-size: 12px;
  color: #ceded5;
}
.login-story footer span {
  display: block;
  font-size: 10px;
  margin-top: 6px;
  color: #b8c9bf;
}
.record-illustration {
  margin-top: 40px;
  background: #f7faf4;
  color: var(--ink);
  padding: 22px;
  border-radius: 14px;
  transform: rotate(-3deg);
  max-width: 360px;
  box-shadow: 0 22px 60px #102c2b30;
}

.illustration-top {
  display: flex;
  align-items: center;
  gap: 12px;
}
.illustration-top small {
  display: block;
  color: var(--muted);
  font-size: 10px;
  margin-top: 3px;
}
.illustration-top > svg {
  margin-left: auto;
  color: var(--brand);
}
.illustration-line {
  display: flex;
  align-items: center;
  justify-content: space-between;
  margin: 24px 0 18px;
  font-size: 12px;
}
.illustration-lines {
  display: grid;
  gap: 10px;
}
.illustration-lines i {
  height: 7px;
  width: 90%;
  background: #e4ece3;
  border-radius: 4px;
}
.illustration-lines i:nth-child(2) {
  width: 70%;
}
.illustration-lines i:nth-child(3) {
  width: 80%;
}
.illustration-bottom {
  display: flex;
  justify-content: space-between;
  margin-top: 22px;
  border-top: 1px solid var(--line);
  padding-top: 16px;
  font-size: 12px;
}
.login-form-side {
  display: flex;
  align-items: center;
  justify-content: center;
  padding: 50px;
```

```
  position: relative;
}
.login-box {
  width: 100%;
  max-width: 360px;
}
.login-box h2 {
  font-size: 34px;
  margin-top: 16px;
}
.login-box > p {
  margin-top: 8px;
}
.login-box form {
  margin-top: 36px;
  display: grid;
  gap: 22px;
}
label {
  display: grid;
  gap: 7px;
  font-size: 13px;
  font-weight: 550;
}
input,
select,
textarea {
  width: 100%;
  border: 1px solid #ccd8d0;
  border-radius: 7px;
  min-height: 44px;
  padding: 10px 12px;
  background: var(--surface);
  color: var(--ink);
  font-size: 16px;
  letter-spacing: 0;
}
textarea {
  min-height: 110px;
  resize: vertical;
}
input::placeholder,
textarea::placeholder {
  color: #78847c;
}
.password-input {
  position: relative;
}
.password-input input {
  padding-right: 48px;
}
.password-input button {
  position: absolute;
  right: 0;
  top: 0;
  height: 44px;
  width: 44px;
  border: 0;
  background: transparent;
  display: grid;
  place-items: center;
  color: var(--muted);
}
.login-submit {
  width: 100%;
  justify-content: space-between;
  margin-top: 4px;
}
.demo-login {
  position: relative;
  margin-top: 32px;
  border: 1px solid var(--line);
  border-radius: 9px;
  padding: 18px;
  background: var(--soft);
}
```

```
.demo-dot {
  display: inline-block;
  height: 6px;
  width: 6px;
  background: var(--brand);
  border-radius: 50%;
  margin-right: 8px;
}
.demo-login strong {
  font-size: 12px;
}
.demo-login p {
  font-size: 11px;
  margin: 7px 0 12px;
  color: var(--muted);
}
.demo-login > div {
  display: flex;
  gap: 16px;
}
.demo-login button {
  display: flex;
  align-items: center;
  gap: 3px;
  background: none;
  border: 0;
  padding: 4px 0;
  font-size: 11px;
  min-height: 32px;
  font-weight: 550;
}
.login-help {
  display: flex;
  align-items: center;
  gap: 6px;
  color: var(--muted);
  font-size: 11px;
  margin-top: 22px !important;
}
.login-copyright {
  position: absolute;
  bottom: 24px;
  left: 50px;
  right: 50px;
  font-size: 10px;
  color: var(--muted);
  display: flex;
  justify-content: space-between;
}
.app-shell {
  display: grid;
  grid-template-columns: 232px minmax(0, 1fr);
  min-height: 100dvh;
}
.sidebar {
  background: var(--brand);
  color: #edf5ed;
  position: fixed;
  inset: 0 auto 0 0;
  width: 232px;
  padding: 28px 18px 20px;
  display: flex;
  flex-direction: column;
  z-index: 30;
}
.sidebar .brand {
  font-size: 19px;
  margin: 0 8px 42px;
}
.sidebar .brand-mark {
  width: 35px;
  height: 35px;
  font-size: 29px;
  border-radius: 10px;
}
.sidebar .brand-sub {
  font-size: 7px;
```

```
  letter-spacing: 1.6px;
}
.nav-caption {
  font-size: 9px;
  letter-spacing: 1.6px;
  color: #b3c9bc;
  padding: 0 14px;
  margin: 0 0 12px;
}
.sidebar nav {
  display: grid;
  gap: 4px;
}
.sidebar nav a {
  display: flex;
  gap: 12px;
  align-items: center;
  min-height: 46px;
  padding: 11px 14px;
  border-radius: 7px;
  color: #d6e5da;
  font-size: 13px;
}
.sidebar nav a.active {
  background: #e4efdf;
  color: #214e38;
  font-weight: 600;
}
.sidebar .nav-separator {
  height: 1px;
  background: #ffffff20;
  margin: 24px 12px;
}
.sidebar-bottom {
  margin-top: auto;
}
.sidebar-note {
  margin: 20px 8px;
  padding: 16px;
  background: #ffffff0b;
  border: 1px solid #ffffff15;
  border-radius: 9px;
  color: #d4e1d6;
  font-size: 11px;
}
.sidebar-note strong {
  display: block;
  font-size: 12px;
  margin-bottom: 6px;
  color: #f3f7f0;
}
.user-block {
  border-top: 1px solid #ffffff20;
  padding: 20px 6px 0;
  display: flex;
  gap: 10px;
  align-items: center;
}
.user-block .avatar {
  background: #45664e;
  color: #eff4e9;
}
.user-block strong {
  display: block;
  font-size: 12px;
  font-weight: 500;
}
.user-block small {
  color: #c2d2c6;
  font-size: 10px;
}

.app-main {
  grid-column: 2;
  min-width: 0;
}
.topbar {
```

```
  height: 80px;
  background: var(--surface);
  border-bottom: 1px solid var(--line);
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 0 38px;
  gap: 20px;
}
.topbar-location {
  font-size: 12px;
  color: var(--muted);
  display: flex;
  gap: 10px;
  align-items: center;
}
.topbar-location strong {
  font-weight: 500;
  color: var(--ink);
}
.topbar-actions {
  display: flex;
  gap: 16px;
  align-items: center;
}
.topbar-date {
  font-size: 11px;
  color: var(--muted);
}
.workspace-content {
  padding: 34px 38px 64px;
  max-width: 1680px;
  margin: 0 auto;
}
.demo-banner {
  display: flex;
  align-items: center;
  gap: 8px;
  font-size: 11px;
  color: var(--brand);
  margin-bottom: 26px;
}
.demo-banner span:last-child {
  margin-left: auto;
  color: var(--muted);
}
.page-heading {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 20px;
  margin-bottom: 28px;
}
.page-heading h1 {
  font-size: 30px;
}
.page-heading p {
  color: var(--muted);
  font-size: 12px;
  margin-top: 7px;
}
.heading-actions {
  display: flex;
  gap: 10px;
  flex-wrap: wrap;
}
.summary-strip {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  background: var(--surface);
  border: 1px solid var(--line);
  border-radius: 10px;
  margin-bottom: 28px;
}
.summary-stat {
  padding: 22px 24px;
  position: relative;
```

```
}
.summary-stat + .summary-stat {
  border-left: 1px solid var(--line);
}
.summary-stat > span {
  display: flex;
  justify-content: space-between;
  color: var(--muted);
  font-size: 11px;
}
.summary-stat > strong {
  font-size: 30px;
  line-height: 1;
  letter-spacing: -1px;
  display: block;
  margin: 14px 0 9px;
  font-weight: 500;
}
.summary-stat > small {
  font-size: 10px;
  color: var(--muted);
}
.dashboard-grid {
  display: grid;
  grid-template-columns: minmax(0, 1fr) 290px;
  gap: 24px;
  align-items: start;
}

.panel {
  background: var(--surface);
  border: 1px solid var(--line);
  border-radius: 10px;
  overflow: hidden;
}
.panel-header {
  padding: 20px 22px;
  border-bottom: 1px solid var(--line);
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 12px;
}
.panel-header p {
  font-size: 11px;
  color: var(--muted);
  margin-top: 4px;
}
.panel-body {
  padding: 22px;
}
.table-wrap {
  overflow-x: auto;
}
table {
  border-collapse: collapse;
  width: 100%;
  text-align: left;
  font-size: 12px;
}
th {
  padding: 13px 20px;
  background: #f5f8f3;
  color: #5c6a60;
  font-size: 10px;
  font-weight: 500;
  letter-spacing: 0.04em;
  white-space: nowrap;
}
td {
  padding: 17px 20px;
  border-top: 1px solid var(--line);
  vertical-align: middle;
}
tbody tr:first-child td {
  border-top: 0;
}
td .button {
```

```
  font-size: 11px;
  min-height: 36px;
  padding: 6px 10px;
}
.time-cell {
  font-weight: 550;
  font-variant-numeric: tabular-nums;
}
.cell-sub {
  display: block;
  font-size: 10px;
  color: var(--muted);
  margin-top: 3px;
}
.table-footer {
  padding: 14px 20px;
  border-top: 1px solid var(--line);
  font-size: 10px;
  color: var(--muted);
  display: flex;
  justify-content: space-between;
}
.empty {
  padding: 40px 22px;
  text-align: center;
  color: var(--muted);
  font-size: 13px;
}
.empty svg {
  display: block;
  margin: 0 auto 12px;
  color: #91a295;
}
.empty strong {
  color: var(--ink);
  display: block;
  margin-bottom: 5px;
  font-size: 15px;
  font-weight: 500;
}
.quick-links {
  display: grid;
  gap: 0;
}
.quick-links button,
.quick-links a {
  display: flex;
  align-items: center;
  gap: 12px;
  padding: 18px 20px;
  border: 0;
  border-bottom: 1px solid var(--line);
  background: transparent;
  text-align: left;
  color: var(--ink);
}
.quick-links button:last-child,
.quick-links a:last-child {
  border: 0;
}
.quick-links svg:first-child {
  color: var(--brand);
  background: var(--soft);
  padding: 8px;
  border-radius: 8px;
  width: 36px;
  height: 36px;
}
.quick-links span {
  flex: 1;
  font-weight: 500;
  font-size: 12px;
}
.quick-links small {
  display: block;
  font-size: 10px;
```

```
  font-weight: 400;
  color: var(--muted);
  margin-top: 3px;
}
.stack {
  display: grid;
  gap: 24px;
}
.ready-item {
  padding: 16px 20px;
  border-bottom: 1px solid var(--line);
}
.ready-item:last-child {
  border: 0;
}
.ready-item > div {
  display: flex;
  justify-content: space-between;
  align-items: center;
}
.ready-item .text-button {
  padding: 6px 0;
  font-size: 11px;
  min-height: 32px;
  margin-top: 6px;
}
.toolbar {
  display: flex;
  align-items: center;
  gap: 12px;
  padding: 18px 20px;
  border-bottom: 1px solid var(--line);
  flex-wrap: wrap;
}
.search-input {
  position: relative;
  max-width: 380px;
  flex: 1;
  min-width: 180px;
}
.search-input svg {
  position: absolute;
  left: 12px;
  top: 14px;
  color: var(--muted);
}
.search-input input {
  padding-left: 38px;
  font-size: 13px;
}
.toolbar input[type="date"] {
  width: auto;
  font-size: 13px;
}
.toolbar .result-count {
  margin-left: auto;
  color: var(--muted);
  font-size: 11px;
}
.action-row {
  display: flex;
  gap: 8px;
  align-items: center;
  flex-wrap: wrap;
}
.patient-header {
  display: flex;
  gap: 20px;
  align-items: center;
  padding: 26px;
}
.patient-header h2 {
  font-size: 25px;
}
.patient-header p {
  margin-top: 7px;
  font-size: 12px;
```

```
  color: var(--muted);
}
.patient-tabs {
  display: flex;
  gap: 26px;
  overflow-x: auto;
  padding: 0 26px;
  border-top: 1px solid var(--line);
}
.patient-tabs button {
  border: 0;
  border-bottom: 2px solid transparent;
  border-radius: 0;
  background: transparent;
  padding: 16px 0;
  white-space: nowrap;
  font-size: 12px;
  color: var(--muted);

}
.patient-tabs button[aria-selected="true"] {
  color: var(--brand);
  border-bottom-color: var(--brand);
  font-weight: 600;
}
.detail-grid {
  display: grid;
  grid-template-columns: minmax(0, 1fr) 290px;
  gap: 24px;
  margin-top: 24px;
}
.facts {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 22px;
}
.facts dt {
  font-size: 10px;
  color: var(--muted);
  margin-bottom: 5px;
}
.facts dd {
  margin: 0;
  font-size: 13px;
  overflow-wrap: anywhere;
}
.timeline-item {
  padding: 20px 22px;
  border-bottom: 1px solid var(--line);
}
.timeline-item:last-child {
  border: 0;
}
.timeline-title {
  display: flex;
  justify-content: space-between;
  gap: 10px;
  align-items: center;
}
.timeline-title strong {
  font-size: 13px;
  font-weight: 550;
}
.timeline-item p {
  font-size: 12px;
  white-space: pre-wrap;
  margin: 10px 0;
  line-height: 1.8;
}
.timeline-item small {
  color: var(--muted);
  font-size: 10px;
}
.clinical-form {

  display: grid;
  gap: 20px;
}
```

```
.form-grid {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 18px;
}
.form-grid .full {
  grid-column: 1/-1;
}
.field-help {
  font-size: 11px;
  font-weight: 400;
  color: var(--muted);
}
.medication-block {
  padding: 18px;
  background: var(--soft);
  border-radius: 8px;
  margin: 12px 0;
}
.form-section {
  border-top: 1px solid var(--line);
  padding-top: 20px;
}
.form-section h3 {
  margin-bottom: 14px;
}
.note-box {
  background: var(--soft);
  padding: 14px 18px;
  border-radius: 8px;
  font-size: 12px;
  line-height: 1.7;
  color: var(--brand);
}
dialog {
  color: var(--ink);
  border: 1px solid var(--line);
  padding: 0;
  border-radius: 14px;
  width: min(620px, calc(100vw - 32px));
  max-height: 90dvh;
  background: var(--surface);
  box-shadow: 0 24px 90px #163e3626;
}
dialog::backdrop {
  background: #102a2650;
  backdrop-filter: blur(3px);
}
.dialog-header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 22px 26px;
  border-bottom: 1px solid var(--line);
}
.dialog-content {
  padding: 24px 26px;
  overflow-y: auto;
  max-height: calc(90dvh - 90px);
}
.dialog-content form {
  display: grid;
  gap: 20px;
}
.dialog-actions {
  display: flex;
  justify-content: flex-end;
  gap: 10px;
  margin-top: 6px;
}
.toast {
  position: fixed;
  bottom: 24px;
  right: 24px;
  background: var(--brand);
  color: var(--surface);
  padding: 16px 22px;
```

```
  border-radius: 9px;
  z-index: 80;
  box-shadow: 0 8px 30px #153d3626;
  max-width: calc(100vw - 40px);
}
.toast.error {
  background: #7d312b;
  color: #fff4ee;
}
.loading {
  padding: 60px;
  text-align: center;
  color: var(--muted);
}
.mobile-nav,
.mobile-brand {
  display: none;
}
.skip-link {
  position: fixed;
  left: 16px;
  top: -100px;
  z-index: 100;
  background: var(--surface);
  padding: 12px 20px;
}
.skip-link:focus {
  top: 12px;
}
.document-list {

  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(260px, 1fr));
  gap: 20px;
}
.document-card {
  background: var(--surface);
  border: 1px solid var(--line);
  padding: 22px;
  border-radius: 10px;
}
.document-card > svg {
  color: var(--brand);
  margin-bottom: 20px;
}
.document-card h3 {
  font-size: 15px;
}
.document-card p {
  font-size: 12px;
  color: var(--muted);
  margin: 8px 0 18px;
}
.document-card small {
  display: block;
  font-size: 10px;
  color: var(--muted);
  margin-bottom: 14px;
}
.report-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(260px, 1fr));
  gap: 20px;
}
.report-card {
  padding: 24px;
}
.report-card > svg {
  color: var(--brand);
  margin-bottom: 16px;
}
.report-card h3 {
  font-size: 16px;
}
.report-card p {
  color: var(--muted);
  font-size: 12px;
  margin: 10px 0 22px;
```

```
  max-width: 40ch;
}
.report-card label {
  margin-bottom: 14px;
}
.mobile-menu-panel {
  display: none;
}

.chart-row {
  display: flex;
  align-items: center;
  gap: 12px;
  margin: 16px 0;
}
.chart-row > span {
  min-width: 95px;
  font-size: 11px;
  color: var(--muted);
}
.chart-bar {
  height: 14px;
  background: var(--brand);
  border-radius: 3px;
  min-width: 4px;
}
.chart-row strong {
  font-size: 11px;
}
.print-page {
  max-width: 900px;
  margin: 40px auto;
  background: var(--surface);
  padding: 48px;
}
.print-header {
  border-bottom: 2px solid var(--brand);
  padding-bottom: 24px;
  margin-bottom: 26px;
  display: flex;
  align-items: center;
  justify-content: space-between;
}
.print-page h1 {
  font-size: 26px;
  margin: 28px 0 8px;
}
.print-page h2 {
  font-size: 18px;
  margin: 24px 0 12px;
}
.print-page p {
  white-space: pre-wrap;
  margin: 8px 0;
}
.print-page .print-note {
  font-size: 11px;
  margin-top: 36px;
  border-top: 1px solid var(--line);
  padding-top: 16px;
  color: var(--muted);
}
@media (hover: hover) and (pointer: fine) {
  .button:hover {

    background: var(--soft);
  }
  .primary:hover {
    background: var(--brand-hover);
  }
  .sidebar nav a:hover:not(.active) {
    background: #ffffff0d;
  }
  .quick-links button:hover,
  .quick-links a:hover {
    background: var(--soft);
  }
  tbody tr:hover {
```

```
    background: #f8faf6;
  }
  .text-button:hover {
    color: var(--brand-hover);
    text-decoration: underline;
  }
}
@media (max-width: 1200px) {
  .dashboard-grid,
  .detail-grid {
    grid-template-columns: minmax(0, 1fr);
  }
  .dashboard-grid > .stack {
    grid-template-columns: 1fr 1fr;
  }
  .workspace-content {
    padding: 28px;
  }
  .topbar {
    padding: 0 28px;
  }
  .summary-stat {
    padding: 20px;
  }
}
@media (max-width: 900px) {
  .sidebar {
    width: 200px;
    padding-left: 12px;
    padding-right: 12px;
  }
  .app-shell {
    grid-template-columns: 200px minmax(0, 1fr);
  }
  .sidebar .brand {
    font-size: 16px;
  }
  .topbar-date {
    display: none;
  }
  .login-story {
    padding: 32px;

  }
  .login-form-side {
    padding: 32px;
  }
  .record-illustration {
    max-width: 300px;
  }
  .story-content h1 {
    font-size: 44px;
  }
}
@media (max-width: 700px) {
  .login-layout {
    display: block;
  }
  .login-story {
    min-height: auto;
    padding: 28px;
  }
  .story-content {
    padding: 40px 0 28px;
  }
  .story-content h1 {
    font-size: 40px;
    margin: 16px 0;
  }
  .story-content > p {
    font-size: 14px;
  }
  .login-story footer,
  .record-illustration {
    display: none;
  }
  .login-form-side {
    padding: 40px 26px 80px;
```

```
}
.login-box {
  max-width: 430px;
}
.login-box h2 {
  font-size: 29px;
}
.login-copyright {
  left: 26px;
  right: 26px;
  bottom: 20px;
}
.sidebar {
  display: none;
}
.app-shell {
  display: block;
}
.app-main {
  width: 100%;
  padding-bottom: 78px;
}
.topbar {
  height: 68px;
  padding: 0 20px;
  gap: 8px;
}
.topbar-location {
  display: none;
}
.mobile-brand {
  display: flex;
  font-size: 18px;
}
.mobile-brand .brand-mark {
  width: 30px;
  height: 30px;
  font-size: 26px;
  border-radius: 8px;
}
.mobile-brand .brand-sub {
  display: none;
}
.topbar-actions {
  gap: 8px;
}
.topbar-actions .button {
  padding: 8px 12px;
  font-size: 12px;
}
.workspace-content {
  padding: 24px 18px 44px;
}
.demo-banner {
  margin-bottom: 20px;
  font-size: 10px;
}
.demo-banner span:last-child {
  display: none;
}
.page-heading {
  align-items: flex-start;
  margin-bottom: 22px;
}
.page-heading h1 {
  font-size: 25px;
}
.page-heading p {
  font-size: 11px;
  max-width: 26ch;
}
.heading-actions {
  gap: 6px;
}
.heading-actions .button {
  font-size: 11px;
```

```
    padding: 8px 10px;
  }
  .summary-strip {
    grid-template-columns: 1fr 1fr;
    margin-bottom: 22px;
  }
  .summary-stat {
    padding: 16px 18px;
  }
  .summary-stat:nth-child(3) {
    border-left: 0;
    border-top: 1px solid var(--line);
  }
  .summary-stat:nth-child(4) {
    border-top: 1px solid var(--line);
  }
  .summary-stat > strong {
    font-size: 27px;
    margin: 10px 0 7px;
  }
  .summary-stat > small {
    font-size: 9px;
  }
  .dashboard-grid {
    gap: 20px;
  }
  .dashboard-grid > .stack {
    grid-template-columns: 1fr;
  }
  .panel-header {
    padding: 18px;
  }
  .panel-header h3 {
    font-size: 16px;
  }
  td,
  th {
    padding: 13px 16px;
  }
  .desktop-cell {
    display: none;
  }
  .panel-body {
    padding: 18px;
  }
  .form-grid,
  .facts {
    grid-template-columns: 1fr;
  }
  .form-grid .full {
    grid-column: auto;
  }
  .patient-header {
    padding: 20px;

    gap: 14px;
  }
  .patient-header h2 {
    font-size: 21px;
  }
  .patient-tabs {
    padding: 0 20px;
    gap: 24px;
  }
  .patient-tabs button {
    min-height: 48px;
  }
  .detail-grid {
    margin-top: 20px;
  }
  .patient-header .avatar.large {
    width: 52px;
    height: 52px;
    font-size: 17px;
  }
  .dialog-header {
    padding: 18px;
  }
```

```
  .dialog-content {
    padding: 20px;
  }
  .mobile-nav {
    display: flex;
    justify-content: space-around;
    position: fixed;
    bottom: 0;
    left: 0;
    right: 0;
    padding: 8px 8px calc(8px + env(safe-area-inset-bottom));
    border-top: 1px solid var(--line);
    background: var(--surface);
    z-index: 40;
  }
  .mobile-nav a,
  .mobile-nav button {
    background: none;
    border: 0;
    display: flex;
    flex-direction: column;
    align-items: center;
    gap: 4px;
    font-size: 10px;
    color: var(--muted);
    min-width: 56px;
    min-height: 44px;
    justify-content: center;
  }
  .mobile-nav .active {
    color: var(--brand);
    font-weight: 600;

  }
  .mobile-menu-panel {
    display: grid;
    position: fixed;
    bottom: 78px;
    left: 16px;
    right: 16px;
    background: var(--surface);
    border: 1px solid var(--line);
    box-shadow: 0 8px 40px #163e3630;
    border-radius: 12px;
    padding: 12px;
    z-index: 45;
  }
  .mobile-menu-panel a {
    display: flex;
    align-items: center;
    gap: 12px;
    padding: 14px;
    font-size: 14px;
  }
  .toast {
    bottom: 90px;
    right: 20px;
  }
  .print-page {
    padding: 22px;
    margin: 18px;
  }
  .print-header {
    display: block;
  }
  .document-list {
    grid-template-columns: 1fr;
  }
  body {
    font-size: 14px;
  }
}
@media (prefers-reduced-motion: reduce) {
  *,
  *::before,
  *::after {
    animation: none !important;
    transition: none !important;
```

```
    scroll-behavior: auto !important;
  }
}
@media print {
  body {
    background: white;
    color: black;
  }
  .no-print {
    display: none !important;
  }
  .print-page {
    max-width: none;
    margin: 0;
    padding: 12px;
  }
  .print-page table {
    font-size: 10px;
  }
  a {
    color: black;
  }
  .print-page h2 {
    break-after: avoid;
  }
  .timeline-item,
  .print-page tr {
    break-inside: avoid;
  }
  @page {
    size: A4;
    margin: 14mm;
  }
}

.sidebar {
  overflow-y: auto;
}
.sidebar .user-block .avatar {
  flex-shrink: 0;
}
@media (max-height: 920px) and (min-width: 701px) {
  .sidebar-note {
    display: none;
  }
  .sidebar .brand {
    margin-bottom: 30px;
  }
}

/* PracticeChart uses a top navigation and a paper-like worklist. */
:root {
  --ink: oklch(0.26 0.028 258);
  --muted: oklch(0.46 0.025 258);
  --brand: oklch(0.42 0.12 263);
  --brand-hover: oklch(0.34 0.11 263);
  --paper: oklch(0.982 0.006 85);
  --surface: oklch(0.998 0.002 85);
  --line: oklch(0.89 0.012 250);
  --soft: oklch(0.95 0.015 255);
  --mint: oklch(0.91 0.04 260);
}
.app-shell {
  display: block;
}

.app-main {
  width: 100%;
  grid-column: auto;
}
.practice-header {
  display: grid;
  grid-template-columns: auto minmax(0, 1fr) auto;
  column-gap: 38px;
  align-items: center;
  background: var(--surface);
  border-bottom: 1px solid var(--line);
```

```
  padding: 22px 40px 0;
}
.practice-header .brand {
  font-size: 21px;
  letter-spacing: -0.8px;
  grid-column: 1;
  grid-row: 1;
}
.practice-header .brand-sub {
  font-size: 7px;
  letter-spacing: 1.3px;
}
.practice-header .brand-mark {
  background: var(--brand);
  color: var(--surface);
  border-radius: 4px;
}
.practice-header > .nav-caption,
.practice-header > .nav-separator,
.practice-header .sidebar-note {
  display: none;
}
.practice-header nav[aria-label="Main navigation"] {
  display: flex;
  justify-content: center;
  align-items: center;
  gap: 26px;
  grid-column: 2;
  grid-row: 1;
}
.practice-header nav[aria-label="Main navigation"] a {
  font-size: 13px;
  font-weight: 500;
  color: var(--muted);
  padding: 12px 0;
  border-bottom: 2px solid transparent;
}
.practice-header nav[aria-label="Main navigation"] a svg {
  display: none;
}
.practice-header nav[aria-label="Main navigation"] a.active {
  color: var(--brand);
  border-bottom-color: var(--brand);
}

.practice-header nav[aria-label="Management navigation"] {
  display: flex;
  justify-content: flex-end;
  gap: 24px;
  grid-column: 1/-1;
  grid-row: 2;
  padding: 8px 0 10px;
}
.practice-header nav[aria-label="Management navigation"] a {
  display: flex;
  align-items: center;
  gap: 6px;
  font-size: 10px;
  color: var(--muted);
  min-height: 28px;
}
.practice-header nav[aria-label="Management navigation"] a svg {
  width: 12px;
  height: 12px;
}
.practice-header .sidebar-bottom {
  margin: 0;
  grid-column: 3;
  grid-row: 1;
}
.practice-header .user-block {
  border: 0;
  padding: 0;
}
.practice-header .user-block strong {
  font-size: 12px;
  color: var(--ink);
}
```

```
.practice-header .user-block small {
  font-size: 10px;
  color: var(--muted);
}
.practice-header .user-block .text-button {
  color: var(--muted) !important;
}
.practice-header .user-block .avatar {
  background: var(--soft);
  color: var(--brand);
}
.topbar {
  height: 56px;
  background: var(--paper);
  border: 0;
  max-width: 1520px;
  margin: auto;
  padding: 0 40px;
}
.workspace-content {
  padding: 18px 40px 64px;
  max-width: 1520px;

}
.demo-banner {
  margin-bottom: 18px;
}
.page-heading {
  margin-bottom: 26px;
}
.page-heading h1 {
  font-family: Georgia, serif;
  font-size: 37px;
  font-weight: 400;
  letter-spacing: -1.2px;
}
.summary-strip {
  display: flex;
  background: transparent;
  border: 0;
  border-top: 1px solid var(--line);
  border-bottom: 1px solid var(--line);
  border-radius: 0;
  margin-bottom: 26px;
}
.summary-stat {
  display: grid;
  grid-template-columns: auto 1fr;
  gap: 4px 12px;
  width: 25%;
  padding: 16px 20px 16px 0;
}
.summary-stat + .summary-stat {
  border: 0;
  padding-left: 24px;
}
.summary-stat > strong {
  grid-column: 1;
  grid-row: 1/3;
  margin: 0;
  font-size: 28px;
}
.summary-stat > span {
  grid-column: 2;
  align-self: end;
  font-size: 11px;
}
.summary-stat > span svg {
  display: none;
}
.summary-stat > small {
  grid-column: 2;
  font-size: 9px;
}
.panel {
  border-radius: 6px;
  box-shadow: none;
}
```

```
.panel-header {
  background: transparent;
}
.panel-header h3 {
  font-weight: 550;
}
.avatar {
  background: #edf0f6;
  color: #384f76;
}
.record-sections {
  display: flex;
  gap: 6px;
  overflow-x: auto;
  border-top: 1px solid var(--line);
  padding: 10px 20px;
  background: var(--paper);
}
.record-sections button {
  border: 1px solid transparent;
  background: transparent;
  min-height: 40px;
  padding: 8px 14px;
  border-radius: 5px;
  font-size: 12px;
  color: var(--muted);
  white-space: nowrap;
}
.record-sections button[aria-selected="true"] {
  background: var(--surface);
  border-color: var(--line);
  color: var(--brand);
  font-weight: 600;
}
.patient-header {
  padding: 24px;
}
.patient-header h2 {
  font-family: Georgia, serif;
  font-weight: 400;
  letter-spacing: -0.6px;
}
.button {
  border-radius: 5px;
}
.login-story {
  background: var(--paper);
  color: var(--ink);
  border-right: 1px solid var(--line);
}
.login-story .brand-mark {
  background: var(--brand);
  color: var(--surface);
  border-radius: 5px;
}

.login-story::after {
  display: none;
}
.story-content .eyebrow {
  color: var(--brand);
}
.story-content h1 {
  font-family: Georgia, serif;
  font-weight: 400;
  letter-spacing: -2px;
  font-size: clamp(40px, 4vw, 64px);
}
.story-content > p {
  color: var(--muted);
}
.login-story footer,
.login-story footer span {
  color: var(--muted);
}
.record-illustration {
  transform: none;
  border: 1px solid var(--line);
```

```
  border-radius: 6px;
  box-shadow: 12px 12px 0 #e6eaf1;
  background: var(--surface);
}
.illustration-lines i {
  background: #e8edf3;
}
.login-form-side {
  background: var(--surface);
}
.login-box h2 {
  font-family: Georgia, serif;
  font-weight: 400;
  font-size: 36px;
}
.demo-login {
  background: var(--paper);
  border-radius: 5px;
}
.login-help {
  color: var(--muted);
}
dialog {
  margin: auto;
  border-radius: 8px;
}
.dialog-header h2 {
  font-size: 22px;
}
input:focus-visible,
textarea:focus-visible,
select:focus-visible {
  outline-color: var(--brand);

}
*:focus-visible {
  outline-color: var(--brand);
}
@media (max-width: 1100px) and (min-width: 701px) {
  .practice-header {
    column-gap: 24px;
    padding: 20px 26px 0;
  }
  .practice-header nav[aria-label="Main navigation"] {
    gap: 18px;
  }
  .practice-header .user-block .avatar {
    display: none;
  }
  .practice-header .brand {
    font-size: 18px;
  }
  .practice-header .brand-sub {
    display: none;
  }
  .practice-header .brand-mark {
    width: 32px;
    height: 32px;
  }
  .workspace-content,
  .topbar {
    padding-left: 26px;
    padding-right: 26px;
  }
}
@media (max-width: 700px) {
  .practice-header {
    display: none;
  }
  .topbar {
    height: 70px;
    padding: 0 18px;
    background: var(--surface);
    border-bottom: 1px solid var(--line);
  }
  .workspace-content {
    padding: 22px 18px 42px;
  }
```

```
  .page-heading h1 {
    font-size: 30px;
  }
  .summary-strip {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 0;
  }
  .summary-stat {
    width: 100%;
    padding: 15px 8px 15px 0;
  }
  .summary-stat + .summary-stat {
    padding-left: 10px;
  }
  .summary-stat:nth-child(3) {
    padding-left: 0;
  }
  .summary-stat:nth-child(n + 3) {
    border-top: 1px solid var(--line);
  }
  .summary-stat > small {
    display: none;
  }
  .summary-stat > strong {
    grid-row: 1;
    font-size: 25px;
  }
  .summary-stat > span {
    align-self: center;
    font-size: 10px;
  }
  .record-sections {
    padding: 8px 14px;
    gap: 4px;
  }
  .record-sections button {
    font-size: 12px;
    min-height: 44px;
    padding: 8px 12px;
  }
  .mobile-brand .brand-mark {
    background: var(--brand);
    color: var(--surface);
    border-radius: 4px;
  }
  .login-story {
    border-right: 0;
    border-bottom: 1px solid var(--line);
  }
  .story-content h1 {
    font-size: 42px;
  }
  .login-form-side {
    padding-top: 34px;
  }
  .demo-login button {
    min-height: 44px;
  }
}

@media (min-width: 701px) and (max-width: 1100px) {
  .practice-header {
    grid-template-columns: minmax(0, 1fr) auto;
    row-gap: 8px;
  }
  .practice-header nav[aria-label="Main navigation"] {
    grid-column: 1/-1;
    grid-row: 2;
    justify-content: flex-start;
  }
  .practice-header nav[aria-label="Management navigation"] {
    grid-row: 3;
  }
  .practice-header .sidebar-bottom {
    grid-column: 2;
```

```
1991     grid-row: 1;
1992   }
1993 }
```

src/app/layout.tsx

```tsx
import type { Metadata, Viewport } from "next";
import { Geist } from "next/font/google";
import "./globals.css";
const geist = Geist({ subsets: ["latin"], variable: "--font-geist" });
export const metadata: Metadata = {
  title: "PracticeChart EHR",
  description: "A fictional private practice EHR for the CIT4001 course.",
  robots: { index: false, follow: false },
  applicationName: "PracticeChart EHR",
};
export const viewport: Viewport = {
  width: "device-width",
  initialScale: 1,
  viewportFit: "cover",
  themeColor: "#163e36",
};
export default function RootLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <html lang="en" data-scroll-behavior="smooth">
      <body className={geist.variable}>{children}</body>
    </html>
  );
}
```

src/app/login/page.tsx

```
"use client";
import { useState } from "react";
import {
  ArrowUpRight,
  ArrowRight,
  ShieldCheck,
  Eye,
  EyeOff,
} from "lucide-react";
export default function Login() {
  const [error, setError] = useState("");
  const [busy, setBusy] = useState(false);
  const [show, setShow] = useState(false);
  return (
    <main className="login-layout">
      <section className="login-story">
        <a className="brand" href="/login">
          <span className="brand-mark">
            p<span>+</span>
          </span>
          <span>
            PracticeChart
            <span className="brand-sub">ELECTRONIC HEALTH RECORDS</span>
          </span>
        </a>
        <div className="story-content">
          <span className="eyebrow">A CLEARER VIEW OF CARE</span>
          <h1>
            Every record.
            <br />
            One clear picture.
          </h1>
          <p>
            Keep the day moving. Find a patient, prepare a visit, and keep care
            records together.
          </p>
          <div className="record-illustration" aria-hidden="true">
            <div className="illustration-top">
              <span className="avatar">LC</span>
              <div>
                <strong>Lena Campbell</strong>
                <small>Fictional patient / PC-000001</small>
              </div>
              <ShieldCheck size={22} />
            </div>
            <div className="illustration-line">
              <span>Patient record</span>
              <span className="status">Available</span>
            </div>
            <div className="illustration-lines">
              <i />
              <i />
              <i />
            </div>
            <div className="illustration-bottom">
              <span>Visit history</span>
              <ArrowUpRight size={20} />
            </div>
          </div>
        </div>
        <footer>
          Built for the practice. Designed around the patient.
          <span>CIT4001 course demonstration</span>
        </footer>
      </section>
      <section className="login-form-side">
        <div className="login-box">
          <span className="eyebrow">YOUR PRACTICE WORKSPACE</span>
          <h2>Welcome back.</h2>
          <p className="muted">Log in to begin your day.</p>
          <form
            onSubmit={async (e) => {
              e.preventDefault();
              setBusy(true);
```

```
              setError("");
              const form = new FormData(e.currentTarget);
              try {
                const r = await fetch("/api/login", {
                  method: "POST",
                  headers: { "Content-Type": "application/json" },
                  body: JSON.stringify({
                    username: form.get("username"),
                    password: form.get("password"),
                  }),
                });
                const d = await r.json();
                if (!r.ok) throw Error(d.error);
                window.location.href = "/app";
              } catch (err) {
                setError(
                  err instanceof Error
                    ? err.message
                    : "Login is not available.",
                );
                setBusy(false);
              }
            }}
          >
            <label>
              Username
              <input
                name="username"
                autoComplete="username"
                autoCapitalize="none"
                required
                placeholder="Enter your username"
              />
            </label>
            <label>
              Password

              <div className="password-input">
                <input
                  name="password"
                  type={show ? "text" : "password"}
                  autoComplete="current-password"
                  required
                  placeholder="Enter your password"
                />
                <button
                  type="button"
                  aria-label={show ? "Hide password" : "Show password"}
                  onClick={() => setShow(!show)}
                >
                  {show ? <EyeOff size={18} /> : <Eye size={18} />}
                </button>
              </div>
            </label>
            {error && (
              <p role="alert" className="error">
                {error}
              </p>
            )}
            <button className="button primary login-submit" disabled={busy}>
              {busy ? "Logging in..." : "Log in"}
              <ArrowRight size={18} />
            </button>
          </form>
          <div className="demo-login">
            <span className="demo-dot" />
            <strong>Explore the fictional practice</strong>
            <p>Use fictional data only. This is a course demo.</p>
            <div>
              <button
                onClick={() => {
                  const f = document.querySelector("form")!;
                  (f.elements.namedItem("username") as HTMLInputElement).value =
                    "doctor";
                  (f.elements.namedItem("password") as HTMLInputElement).value =
                    "DoctorDemo!2026";
                }}
              >
```

```
                Doctor account <ArrowUpRight size={14} />
              </button>
              <button
                onClick={() => {
                  const f = document.querySelector("form")!;
                  (f.elements.namedItem("username") as HTMLInputElement).value =
                    "frontdesk";
                  (f.elements.namedItem("password") as HTMLInputElement).value =
                    "DeskDemo!2026";
                }}
              >
                Front desk account <ArrowUpRight size={14} />
              </button>
            </div>

          </div>
          <p className="login-help">
            <ShieldCheck size={16} /> Your role controls which records you can
            use.
          </p>
        </div>
        <span className="login-copyright">
          PracticeChart EHR <span>Records with purpose.</span>
        </span>
      </section>
    </main>
  );
}
```

src/app/page.tsx

```
1 import { redirect } from "next/navigation";
2 export default function Home() {
3   redirect("/app");
4 }
```

src/app/print/[type]/page.tsx

```tsx
import PrintDocument from "@/components/print-document";
export default async function PrintPage({
  params,
  searchParams,
}: {
  params: Promise<{ type: string }>;
  searchParams: Promise<{ id?: string }>;
}) {
  return (
    <PrintDocument
      type={(await params).type}
      id={(await searchParams).id || ""}
    />
  );
}
```

src/components/print-document.tsx

```
"use client";
import { useEffect, useState } from "react";
import { Printer, ArrowLeft } from "lucide-react";
import { api, dateText, timeText, money } from "./workspace";
import type { ClinicalContent } from "@/lib/validation";
type Visit = {
  id: string;
  firstName: string;
  lastName: string;
  mrn: string;
  visitAt: string;
  doctorName: string;
  status: string;
};
type Revision = {
  visitId: string;
  content: ClinicalContent;
  state: string;
  revision: number;
  reason?: string;
  createdAt: string;
};
type Data = {
  patient?: {
    firstName: string;
    lastName: string;
    mrn: string;
    dob: string;
    address: string;
    phone: string;
  };
  visit?: Visit;
  visits?: Visit[];
  clinical?: Revision[] | Revision;
  documents?: { id: string; title: string; kind: string; createdAt: string }[];
  appointments?: {
    id: string;
    firstName: string;
    lastName: string;
    mrn: string;
    startAt: string;
    endAt: string;
    status: string;
    doctorName: string;
  }[];
  counts?: { day: string; visits: string; filed: string }[];
  invoice?: {
    number: string;
    firstName: string;
    lastName: string;
    mrn: string;
    address: string;
    createdAt: string;
    status: string;
    totalCents: number;
    lines: { description: string; quantity: number; unitCents: number }[];
  };
};
export default function PrintDocument({
  type,
  id,
}: {
  type: string;
  id: string;
}) {
  const [data, setData] = useState<Data | null>(null);
  const [error, setError] = useState("");
  useEffect(() => {
    api<Data>(`reports/${type}?id=${encodeURIComponent(id)}`)
      .then(setData)
      .catch((e) => setError(e.message));
  }, [type, id]);
  const titles: Record<string, string> = {
    history: "Patient history report",
```

```
    prescription: "Prescription",
    certificate: "Medical certificate",
    summary: "Clinical visit summary",
    attendance: "Attendance slip",
    invoice: "Service invoice",
    schedule: "Appointment schedule",
    visits: "Visit logbook",
  };
  const clinical =
    data?.clinical && !Array.isArray(data.clinical) ? data.clinical : undefined;
  return (
    <main className="print-page">
      <div className="no-print action-row">
        <a href="/app" className="button">
          <ArrowLeft size={16} />
          Workspace
        </a>
        <button
          className="button primary"
          disabled={!data}
          onClick={() => window.print()}
        >
          <Printer size={16} />
          Print or save PDF
        </button>
      </div>
      <header className="print-header">
        <span className="brand">PracticeChart EHR</span>
        <span className="muted">Fictional practice / Course demonstration</span>
      </header>
      <h1>{titles[type] || "Report"}</h1>
      <p className="muted">
        Prepared {dateText(new Date().toISOString())} at{" "}
        {timeText(new Date().toISOString())}, Jamaica time
      </p>
      {error && (
        <p className="error" role="alert">
          {error}
        </p>
      )}
      {!data && !error && <p role="status">Loading report...</p>}
      {data?.patient && (
        <>
          <h2>
            {data.patient.firstName} {data.patient.lastName}
          </h2>
          <p>
            {data.patient.mrn} / Date of birth: {dateText(data.patient.dob)}
          </p>
          <p>
            {data.patient.address} / {data.patient.phone}
          </p>
        </>
      )}
      {data?.visit && (
        <>
          <h2>
            {data.visit.firstName} {data.visit.lastName}
          </h2>
          <p>Medical record: {data.visit.mrn}</p>
          <p>
            Visit: {dateText(data.visit.visitAt)} at{" "}
            {timeText(data.visit.visitAt)}
          </p>
          <p>Doctor: {data.visit.doctorName}</p>
          {type === "attendance" && (
            <p>
              Visit status: {data.visit.status}. This slip contains no
              diagnosis, treatment, or payment information.
            </p>
          )}
        </>
      )}
      {clinical && type === "summary" && (
        <>
          <h2>Reason for visit</h2>
          <p>{clinical.content.symptoms || "Not recorded"}</p>
```

```
          <p>
            Prescription issued:{" "}
            {clinical.content.medications.length ? "Yes" : "No"}
          </p>
          <p>Clinical revision: {clinical.revision}</p>
        </>
      )}
      {clinical && type === "prescription" && (
        <>
          <h2>Doctor-entered prescription</h2>
          {clinical.content.medications.length ? (
            <table>
              <thead>
                <tr>
                  <th>Medicine</th>
                  <th>Directions</th>
                  <th>Quantity</th>
                </tr>
              </thead>
              <tbody>
                {clinical.content.medications.map((m, i) => (
                  <tr key={i}>
                    <td>{m.medication}</td>
                    <td>
                      {m.dosage}, {m.route}, {m.frequency}, {m.duration}
                      <br />
                      {m.instructions}
                    </td>
                    <td>{m.quantity}</td>
                  </tr>
                ))}
              </tbody>
            </table>
          ) : (
            <p>No medication was entered for this visit.</p>
          )}
        </>
      )}
      {clinical && type === "certificate" && (
        <>
          <h2>Certificate text entered by the doctor</h2>
          <p>
            {clinical.content.certificate ||
              "No medical certificate was entered for this visit."}
          </p>
        </>
      )}
      {type === "history" && Array.isArray(data?.clinical) && (
        <>
          {data.clinical.length ? (
            data.clinical.map((c) => (
              <section key={c.visitId + c.revision} className="timeline-item">
                <h2>
                  {dateText(c.createdAt)} / Revision {c.revision} / {c.state}
                </h2>
                {c.reason && (
                  <p>
                    <strong>Correction or revision reason:</strong> {c.reason}
                  </p>
                )}
                {Object.entries(c.content)
                  .filter(([k, v]) => k !== "medications" && Boolean(v))
                  .map(([k, v]) => (
                    <p key={k}>
                      <strong>{k.replace(/([A-Z])/g, " $1")}:</strong>{" "}
                      {String(v)}
                    </p>
                  ))}
                {c.content.medications.map((m, i) => (
                  <p key={i}>
                    <strong>Medication:</strong> {m.medication}; {m.dosage};{" "}
                    {m.route}; {m.frequency}; {m.duration}; quantity{" "}
                    {m.quantity}. {m.instructions}
                  </p>
                ))}
              </section>
```

```
            ))
          ) : (
            <p>No clinical entries are recorded.</p>
          )}
          <h2>Diagnostic report index</h2>
          <p>
            Retrieve the original files through the private patient record. This
            index does not contain the file contents.
          </p>
          {data.documents?.map((d) => (
            <p key={d.id}>
              {d.title} / {d.kind} / {dateText(d.createdAt)}
            </p>
          ))}
        </>
      )}
      {data?.invoice && (
        <>
          <h2>
            {data.invoice.number} / {data.invoice.status}
          </h2>
          <p>
            {data.invoice.firstName} {data.invoice.lastName} /{" "}
            {data.invoice.mrn}
          </p>
          <p>{data.invoice.address}</p>
          <p>Invoice date: {dateText(data.invoice.createdAt)}</p>
          <table>
            <thead>
              <tr>
                <th>Service</th>
                <th>Quantity</th>
                <th>Unit charge</th>
                <th>Line total</th>
              </tr>
            </thead>
            <tbody>
              {data.invoice.lines.map((l, i) => (
                <tr key={i}>
                  <td>{l.description}</td>
                  <td>{l.quantity}</td>
                  <td>{money(l.unitCents)}</td>
                  <td>{money(l.unitCents * l.quantity)}</td>
                </tr>
              ))}
            </tbody>
          </table>
          <h2>Total: {money(data.invoice.totalCents)}</h2>
          <p>Currency: JMD. This invoice is not proof of payment.</p>
        </>
      )}
      {data?.appointments && (
        <table>
          <thead>
            <tr>
              <th>Time</th>
              <th>Patient</th>
              <th>Medical record</th>
              <th>Doctor</th>
              <th>Status</th>
            </tr>
          </thead>
          <tbody>
            {data.appointments.map((a) => (
              <tr key={a.id}>
                <td>
                  {timeText(a.startAt)} to {timeText(a.endAt)}
                </td>
                <td>
                  {a.firstName} {a.lastName}
                </td>
                <td>{a.mrn}</td>
                <td>{a.doctorName}</td>
                <td>{a.status}</td>
              </tr>
            ))}
          </tbody>
```

```
        </table>
      )}
      {data?.counts && (
        <table>
          <thead>
            <tr>
              <th>Date</th>
              <th>Visits</th>
              <th>Filed visits</th>
            </tr>
          </thead>
          <tbody>
            {data.counts.map((c) => (
              <tr key={c.day}>
                <td>{dateText(c.day)}</td>
                <td>{c.visits}</td>
                <td>{c.filed}</td>
              </tr>
            ))}
          </tbody>
        </table>
      )}
      <p className="print-note">
        Fictional data only. This course output is not a clinical approval or a
        signed medical document. A patient report is not a database backup.
      </p>

    </main>
  );
}
```

src/components/workspace.tsx

```tsx
"use client";
import Link from "next/link";
import { usePathname, useRouter } from "next/navigation";
import {
  useCallback,
  useEffect,
  useRef,
  useState,
  type ReactNode,
  type FormEvent,
} from "react";
import {
  LayoutDashboard,
  Users,
  CalendarDays,
  Files,
  ChartNoAxesCombined,
  ReceiptText,
  Settings2,
  ShieldCheck,
  LogOut,
  Plus,
  Search,
  ArrowUpRight,
  ArrowRight,
  ChevronRight,
  Clock3,
  Check,
  FileText,
  X,
  MoreHorizontal,
  ClipboardList,
  Printer,
  CalendarPlus,
  UserPlus,
  Stethoscope,
  FolderOpen,
  Activity,
  Download,
  LockKeyhole,
} from "lucide-react";
import type { Staff } from "@/lib/schema";
import { emptyClinical, type ClinicalContent } from "@/lib/validation";
type Patient = {
  id: string;
  mrn: string;
  firstName: string;
  lastName: string;
  dob: string;
  phone: string;
  address: string;
  gender: string;
  emergencyContact?: string;
  insurance?: string;
  provisionalReason: string;
  mergedInto?: string;
  archived: boolean;
  version: number;
};
type Doctor = { id: string; name: string };
type Appointment = {
  id: string;
  patientId: string;
  doctorId: string;
  startAt: string;
  endAt: string;
  status: string;
  version: number;
  firstName?: string;
  lastName?: string;
  mrn?: string;
  doctorName: string;
};
type Visit = {
```

```
  id: string;
  patientId: string;
  doctorId: string;
  status: string;
  revision: number;
  visitAt: string;
  filedAt?: string;
  doctorName: string;
};
type Revision = {
  id: string;
  visitId: string;
  revision: number;
  authorName: string;
  state: string;
  reason: string;
  content: ClinicalContent;
  createdAt: string;
};
type Doc = {
  id: string;
  patientId: string;
  title: string;
  kind: string;
  mime: string;
  size: number;
  createdAt: string;
  replacesId?: string;
  firstName?: string;
  lastName?: string;
  mrn?: string;
};
type Invoice = {
  id: string;
  number: string;
  patientId: string;
  lines: { description: string; quantity: number; unitCents: number }[];
  totalCents: number;
  status: string;
  version: number;
  firstName: string;
  lastName: string;
  createdAt: string;
};
type Account = Staff & { active: boolean };
type Audit = {
  id: string;
  actorName?: string;
  action: string;
  entity: string;
  entityId: string;
  createdAt: string;
  metadata: Record<string, unknown>;
};
type Detail = {
  patient: Patient;
  visits: Visit[];
  appointments: Appointment[];
  clinical?: Revision[];
  documents?: Doc[];
};
type Dashboard = {
  counts: {
    patients: string;
    appointments: string;
    ready: string;
    filed: string;
  };
  ready: (Visit & Patient & { patientId: string })[];
  reminders: (Appointment & Patient)[];
};
type Hours = {
  hours: {
    doctorId: string;
    doctorName: string;
    weekday: number;
    startMin: number;
```

```
    endMin: number;
  }[];
  blocks: { id: string; doctorName: string; startAt: string; endAt: string }[];
};
type Trend = { day: string; visits: string; filed: string };
type Modal = {
  kind: string;
  replacesId?: string;
  patient?: Patient;
  appointment?: Appointment;
  invoice?: Invoice;
  account?: Account;
};
export async function api<T>(
  path: string,
  body?: unknown,
  method = "POST",
  signal?: AbortSignal,
): Promise<T> {
  const r = await fetch("/api/" + path, {
    method: body === undefined ? "GET" : method,
    signal,
    headers:
      body instanceof FormData
        ? {}
        : body === undefined
          ? {}
          : { "Content-Type": "application/json" },
    body:
      body === undefined
        ? undefined
        : body instanceof FormData
          ? body
          : JSON.stringify(body),
  });
  const d = await r.json();
  if (r.status === 401) {
    window.location.href = "/login";
    throw Error("Please log in.");
  }
  if (!r.ok) throw Error(d.error || "The request failed.");
  return d as T;
}
const today = () =>
  new Intl.DateTimeFormat("en-CA", { timeZone: "America/Jamaica" }).format(
    new Date(),
  );
export const dateText = (
  date: string,
  opts: Intl.DateTimeFormatOptions = {
    day: "numeric",
    month: "short",
    year: "numeric",
  },
) =>
  new Intl.DateTimeFormat("en-JM", {
    timeZone: "America/Jamaica",
    ...opts,
  }).format(new Date(date.length === 10 ? date + "T12:00:00Z" : date));
export const timeText = (date: string) =>
  dateText(date, { hour: "numeric", minute: "2-digit" });
export const money = (n: number) =>
  new Intl.NumberFormat("en-JM", { style: "currency", currency: "JMD" }).format(
    n / 100,
  );
const initials = (p: { firstName: string; lastName: string }) =>
  (p.firstName[0] + p.lastName[0]).toUpperCase();
function Status({ value }: { value: string }) {
  return (
    <span className={"status " + value}>{value.replaceAll("_", " ")}</span>
  );
}
function Person({
  patient,
  sub,
}: {
```

```
  patient: Pick<Patient, "firstName" | "lastName">;
  sub?: string;
}) {
  return (
    <div className="person">
      <span className="avatar" aria-hidden="true">
        {initials(patient)}
      </span>
      <span>
        <strong>
          {patient.firstName} {patient.lastName}
        </strong>
        {sub && <small>{sub}</small>}
      </span>
    </div>
  );
}
function Empty({
  title = "Nothing here yet.",
  children,
}: {
  title?: string;
  children?: ReactNode;
}) {
  return (
    <div className="empty">
      <FolderOpen size={24} />
      <strong>{title}</strong>
      {children}
    </div>
  );
}
function Panel({
  title,
  subtitle,
  action,
  children,
}: {
  title?: string;
  subtitle?: string;
  action?: ReactNode;
  children: ReactNode;
}) {
  return (
    <section className="panel">
      {title && (
        <div className="panel-header">
          <div>
            <h3>{title}</h3>
            {subtitle && <p>{subtitle}</p>}
          </div>
          {action}
        </div>
      )}
      {children}
    </section>
  );
}
function PrintLink({
  type,
  id,
  label = "Print",
}: {
  type: string;
  id: string;
  label?: string;
}) {
  return (
    <a
      className="button"
      target="_blank"
      rel="noopener"
      href={`/print/${type}?id=${encodeURIComponent(id)}`}
    >
      <Printer size={14} />
      {label}
    </a>
```

```
  );
}
function Dialog({
  title,
  close,
  children,
}: {
  title: string;
  close: () => void;
  children: ReactNode;
}) {
  const ref = useRef<HTMLDialogElement>(null);
  useEffect(() => {
    const dialog = ref.current!;
    dialog.showModal();
    return () => dialog.close();
  }, []);
  return (
    <dialog
      ref={ref}
      onCancel={close}
      onClick={(e) => {
        if (e.target === ref.current) close();
      }}
      aria-labelledby="dialog-title"
    >
      <div className="dialog-header">
        <h2 id="dialog-title">{title}</h2>
        <button className="icon-button" aria-label="Close form" onClick={close}>
          <X size={18} />
        </button>
      </div>
      <div className="dialog-content">{children}</div>
    </dialog>
  );
}
function Form({
  children,
  submit,
  label = "Save",
  secondary,
}: {
  children: ReactNode;
  submit: (d: FormData) => Promise<void>;
  label?: string;
  secondary?: ReactNode;
}) {
  const [busy, setBusy] = useState(false);
  const [error, setError] = useState("");
  return (
    <form
      onSubmit={async (e) => {
        e.preventDefault();
        setError("");
        setBusy(true);
        try {
          await submit(new FormData(e.currentTarget));
        } catch (err) {
          setError(err instanceof Error ? err.message : "The save failed.");
        } finally {
          setBusy(false);
        }
      }}
    >
      {children}
      {error && (
        <p className="error" role="alert">
          {error}
        </p>
      )}
      <div className="dialog-actions">
        {secondary}
        <button className="button primary" disabled={busy}>
          {busy ? "Saving..." : label}
          <Check size={16} />
        </button>
      </div>
```

```
    </form>
  );
}
const field = (d: FormData, k: string) => String(d.get(k) || "");

function PatientSelect({
  patients,
  selected,
  selectedId,
  name = "patientId",
  label = "Patient",
  excludeId,
  includeArchived = false,
  onChange,
  onSelect,
}: {
  patients: Patient[];
  selected?: Patient;
  selectedId?: string;
  name?: string;
  label?: string;
  excludeId?: string;
  includeArchived?: boolean;
  onChange?: (id: string) => void;
  onSelect?: (patient: Patient | undefined) => void;
}) {
  const [query, setQuery] = useState("");
  const [matches, setMatches] = useState<Patient[]>(patients);
  const [chosen, setChosen] = useState<Patient | undefined>(
    selected || patients.find((p) => p.id === selectedId),
  );
  const [value, setValue] = useState(selected?.id || selectedId || "");
  const [error, setError] = useState("");
  useEffect(() => {
    const controller = new AbortController();
    const timeout = setTimeout(() => {
      api<Patient[]>(
        "patients?q=" + encodeURIComponent(query),
        undefined,
        "GET",
        controller.signal,
      )
        .then(setMatches)
        .catch((e) => {
          if (!controller.signal.aborted) setError(e.message);
        });
    }, 250);
    return () => {
      clearTimeout(timeout);
      controller.abort();
    };
  }, [query]);
  useEffect(() => {
    if (!selectedId || chosen) return;
    const controller = new AbortController();
    api<Detail>("patients/" + selectedId, undefined, "GET", controller.signal)
      .then((d) => {
        setChosen(d.patient);
        onSelect?.(d.patient);
      })

      .catch((e) => {
        if (!controller.signal.aborted) setError(e.message);
      });
    return () => controller.abort();
  }, [selectedId, chosen, onSelect]);
  const options =
    chosen && !matches.some((p) => p.id === chosen.id)
      ? [chosen, ...matches]
      : matches;
  return (
    <>
      <label>
        Find patient by name or MRN
        <input
          type="search"
          value={query}
          onChange={(e) => {
```

```
            setError("");
            setQuery(e.target.value);
          }}
          placeholder="Search all patient records"
        />
      </label>
      <label>
        {label}
        <select
          aria-label={label}
          name={name}
          value={value}
          onChange={(e) => {
            setValue(e.target.value);
            setChosen(options.find((p) => p.id === e.target.value));
            onSelect?.(options.find((p) => p.id === e.target.value));
            onChange?.(e.target.value);
          }}
          required
        >
          <option value="" disabled>
            Choose a patient
          </option>
          {options
            .filter(
              (p) => (includeArchived || !p.archived) && p.id !== excludeId,
            )
            .map((p) => (
              <option key={p.id} value={p.id}>
                {p.firstName} {p.lastName} / {p.mrn}
              </option>
            ))}
        </select>
      </label>
      {error && (
        <p role="alert" className="error">
          {error}
        </p>

      )}
    </>
  );
}
function PatientForm({
  patient,
  done,
}: {
  patient?: Patient;
  done: (message: string, id?: string) => void;
}) {
  return (
    <Form
      label={patient ? "Save changes" : "Register patient"}
      submit={async (d) => {
        const values = Object.fromEntries(d);
        const r = await api<{ patient: Patient; duplicates?: Patient[] }>(
          "patients" + (patient ? "/" + patient.id : ""),
          { ...values, ...(patient ? { version: patient.version } : {}) },
        );
        done(
          r.duplicates?.length
            ? "Patient registered. A possible duplicate needs review."
            : "Patient record saved.",
          r.patient.id,
        );
      }}
    >
      <div className="note-box">
        Use fictional patient details only. The system assigns a unique medical
        record number.
      </div>
      <div className="form-grid">
        <label>
          First name
          <input
            name="firstName"
            defaultValue={patient?.firstName}
            maxLength={80}
```

```
            autoComplete="off"
            required
          />
        </label>
        <label>
          Last name
          <input
            name="lastName"
            defaultValue={patient?.lastName}
            maxLength={80}
            required
          />
        </label>
        <label>
          Date of birth
          <input
            name="dob"
            type="date"
            max={today()}
            defaultValue={patient?.dob}
            required
          />
        </label>
        <label>
          Phone
          <input
            name="phone"
            type="tel"
            defaultValue={patient?.phone}
            maxLength={30}
            minLength={7}
            required
          />
        </label>
        <label className="full">
          Address
          <input
            name="address"
            defaultValue={patient?.address}
            maxLength={300}
            required
          />
        </label>
        <label>
          Gender
          <select name="gender" defaultValue={patient?.gender || "Not stated"}>
            {["Not stated", "Female", "Male", "Other"].map((v) => (
              <option key={v}>{v}</option>
            ))}
          </select>
        </label>
        <label>
          Insurance reference (optional)
          <input
            name="insurance"
            maxLength={100}
            defaultValue={patient?.insurance}
          />
        </label>
        <label className="full">
          Emergency contact (optional)
          <input
            name="emergencyContact"
            defaultValue={patient?.emergencyContact}
            maxLength={200}
          />
        </label>
        <label className="full">
          Provisional record reason (optional)
          <textarea
            name="provisionalReason"
            defaultValue={patient?.provisionalReason}
            maxLength={500}
          />
          <span className="field-help">
            Use this only when the primary record cannot be identified. A doctor
```

```
            must review it.
          </span>
        </label>
      </div>
    </Form>
  );
}
function BookingForm({
  patients,
  doctors,
  patient,
  appointment,
  done,
}: {
  patients: Patient[];
  doctors: Doctor[];
  patient?: Patient;
  appointment?: Appointment;
  done: () => void;
}) {
  const [date, setDate] = useState(
    appointment ? appointment.startAt.slice(0, 10) : today(),
  );
  const [doctor, setDoctor] = useState(
    appointment?.doctorId || doctors[0]?.id || "",
  );
  const [slots, setSlots] = useState<{ startAt: string; endAt: string }[]>([]);
  const [slot, setSlot] = useState("");
  const [length, setLength] = useState(
    appointment
      ? (Date.parse(appointment.endAt) - Date.parse(appointment.startAt)) /
          60000
      : 30,
  );
  const [selectedPatient, setSelectedPatient] = useState(
    patient || patients.find((p) => p.id === appointment?.patientId),
  );
  const [review, setReview] = useState<{
    patientId: string;
    doctorId: string;
    startAt: string;
    endAt: string;
  } | null>(null);
  const [error, setError] = useState("");
  useEffect(() => {
    if (!doctor && doctors.length) setDoctor(doctors[0].id);
  }, [doctor, doctors]);
  useEffect(() => {
    let active = true;
    setError("");

    setSlots([]);
    setSlot("");
    if (doctor)
      api<{ startAt: string; endAt: string }[]>(
        `availability?doctorId=${doctor}&date=${date}&duration=${length}${selectedPatient ? "&patientId="
    + selectedPatient.id : ""}${appointment ? "&excludeId=" + appointment.id : ""}`,
      )
        .then((v) => {
          if (active) setSlots(v);
        })
        .catch((e) => {
          if (active) setError(e.message);
        });
    return () => {
      active = false;
    };
  }, [doctor, date, length, selectedPatient?.id, appointment?.id]);
  return (
    <Form
      label={review ? "Confirm appointment" : "Review appointment"}
      secondary={
        review ? (
          <button
            type="button"
            className="button"
            onClick={() => setReview(null)}
          >
```

```
            Change details
          </button>
        ) : null
      }
      submit={async (d) => {
        if (!review) {
          if (!selectedPatient) throw Error("Choose a patient.");
          const start = field(d, "startAt");
          if (!start) throw Error("Choose an available time.");
          setReview({
            patientId: field(d, "patientId"),
            doctorId: doctor,
            startAt: new Date(start).toISOString(),
            endAt: new Date(Date.parse(start) + length * 60000).toISOString(),
          });
          return;
        }
        await api("appointments" + (appointment ? "/" + appointment.id : ""), {
          ...review,
          ...(appointment ? { version: appointment.version } : {}),
        });
        done();
      }}
    >
      {error && (
        <p className="error" role="alert">
          {error}
        </p>

      )}
      <div hidden={Boolean(review)} className="clinical-form">
        <PatientSelect
          patients={patients}
          selectedId={appointment?.patientId}
          onSelect={setSelectedPatient}
          selected={
            patient || patients.find((p) => p.id === appointment?.patientId)
          }
        />
        <label>
          Doctor
          <select value={doctor} onChange={(e) => setDoctor(e.target.value)}>
            {doctors.map((d) => (
              <option key={d.id} value={d.id}>
                {d.name}
              </option>
            ))}
          </select>
        </label>
        <div className="form-grid">
          <label>
            Date
            <input
              type="date"
              min={today()}
              value={date}
              onChange={(e) => {
                setDate(e.target.value);
                setSlot("");
              }}
              required
            />
          </label>
          <label>
            Length
            <select
              value={length}
              onChange={(e) => setLength(Number(e.target.value))}
            >
              {[15, 30, 45, 60, 90, 120, 240].map((n) => (
                <option key={n} value={n}>
                  {n} minutes
                </option>
              ))}
            </select>
          </label>
        </div>
        <label>
```

```
          Available start time
          <select
            name="startAt"
            value={slot}
            onChange={(e) => setSlot(e.target.value)}
            required
          >
            <option value="">Choose a time</option>
            {slots.map((s) => (
              <option key={s.startAt} value={s.startAt}>
                {timeText(s.startAt)}
              </option>
            ))}
          </select>
        </label>
        {!slots.length && (
          <p className="note-box">
            No times are available on this date. Try the next weekday.
          </p>
        )}
        <p className="field-help">
          Times use Jamaica time. Availability can change before confirmation.
        </p>
      </div>
      {review && (
        <div className="note-box">
          <h3>Ready to confirm?</h3>
          <p>
            {selectedPatient?.firstName} {selectedPatient?.lastName} /{" "}
            {selectedPatient?.mrn}
          </p>
          <p>{doctors.find((d) => d.id === review.doctorId)?.name}</p>
          <p>
            {dateText(review.startAt)} / {timeText(review.startAt)} to{" "}
            {timeText(review.endAt)}
          </p>
          <p>The appointment is saved only after you confirm.</p>
        </div>
      )}
    </Form>
  );
}
function ClinicalEditor({
  visit,
  revision,
  saved,
}: {
  visit: Visit;
  revision?: Revision;
  saved: () => void;
}) {
  const [meds, setMeds] = useState<ClinicalContent["medications"]>(
    revision?.content.medications || [],
  );
  const content = revision?.content || emptyClinical;
  const final = revision?.state === "final";
  return (
    <Form
      label={final ? "Save final correction" : "Save clinical entry"}
      submit={async (d) => {
        await api(`visits/${visit.id}/clinical`, {
          expectedRevision: visit.revision,
          state: final ? "final" : field(d, "state"),
          symptoms: field(d, "symptoms"),
          notes: field(d, "notes"),
          diagnosis: field(d, "diagnosis"),
          treatment: field(d, "treatment"),
          allergies: field(d, "allergies"),
          vitals: field(d, "vitals"),
          labResults: field(d, "labResults"),
          certificate: field(d, "certificate"),
          reason: field(d, "reason"),
          medications: meds,
        });
        saved();
      }}
```

```
    >
      <div className="note-box">
        {final
          ? "This entry is final. A correction creates a new revision and keeps the previous entry."
          : "Record only the findings and decisions entered by the doctor. No diagnosis or treatment is
generated."}
      </div>
      <div className="form-grid">
        {(
          [
            "symptoms",
            "notes",
            "diagnosis",
            "treatment",
            "allergies",
            "vitals",
            "labResults",
          ] as const
        ).map((k) => (
          <label
            key={k}
            className={
              ["notes", "treatment", "labResults"].includes(k) ? "full" : ""
            }
          >
            {
              {
                symptoms: "Symptoms and concerns",
                notes: "Consultation notes",
                diagnosis: "Diagnosis",
                treatment: "Treatment record",
                allergies: "Allergies",
                vitals: "Vitals and units",
                labResults: "Manual lab results",
              }[k]
            }
            <textarea
              aria-label={
                {
                  symptoms: "Symptoms and concerns",

                  notes: "Consultation notes",
                  diagnosis: "Diagnosis",
                  treatment: "Treatment record",
                  allergies: "Allergies",
                  vitals: "Vitals and units",
                  labResults: "Manual lab results",
                }[k]
              }
              name={k}
              defaultValue={content[k]}
              maxLength={
                k === "notes"
                  ? 12000
                  : k === "symptoms" || k === "treatment"
                    ? 8000
                    : k === "labResults"
                      ? 6000
                      : k === "diagnosis"
                        ? 4000
                        : 2000
              }
            />
          </label>
        ))}
      </div>
      <section className="form-section">
        <div className="timeline-title">
          <h3>Prescription details</h3>
          <button
            type="button"
            className="text-button"
            onClick={() =>
              setMeds([
                ...meds,
                {
                  medication: "",
                  dosage: "",
```

```
                  route: "",
                  frequency: "",
                  duration: "",
                  quantity: "",
                  instructions: "",
                },
              ])
            }
            disabled={meds.length >= 20}
          >
            <Plus size={15} />
            Add medicine
          </button>
        </div>
        <p className="field-help">
          The printed prescription uses these exact doctor-entered details.
        </p>
        {meds.map((med, i) => (

          <div className="medication-block" key={i}>
            <div className="form-grid">
              {(Object.keys(med) as (keyof typeof med)[]).map((k) => (
                <label key={k}>
                  {k[0].toUpperCase() + k.slice(1)}
                  {k === "instructions" ? " (optional)" : ""}
                  <input
                    value={med[k]}
                    required={k !== "instructions"}
                    maxLength={k === "instructions" ? 500 : 150}
                    onChange={(e) =>
                      setMeds(
                        meds.map((m, j) =>
                          j === i ? { ...m, [k]: e.target.value } : m,
                        ),
                      )
                    }
                  />
                </label>
              ))}
            </div>
            <button
              className="text-button danger"
              type="button"
              onClick={() => setMeds(meds.filter((_, j) => j !== i))}
            >
              Remove medicine
            </button>
          </div>
        ))}
      </section>
      <label>
        Medical certificate text (optional)
        <textarea
          name="certificate"
          defaultValue={content.certificate}
          maxLength={6000}
        />
      </label>
      {!final && (
        <label>
          Entry state
          <select name="state" defaultValue="draft">
            <option value="draft">Draft</option>
            <option value="final">Final, ready for filing</option>
          </select>
        </label>
      )}
      <label>
        {final ? "Correction reason" : "Revision reason (optional)"}
        <textarea name="reason" maxLength={1000} required={final} />
      </label>
    </Form>
  );
}

export default function Workspace({
  actor,
  demo,
```

```
}: {
  actor: Staff;
  demo: boolean;
}) {
  const router = useRouter();
  const pathname = usePathname();
  const segments = pathname.split("/").filter(Boolean);
  const page = segments[1] || "overview";
  const patientId = page === "patients" ? segments[2] : undefined;
  const doctor = actor.role === "doctor";
  const [patients, setPatients] = useState<Patient[]>([]);
  const [doctors, setDoctors] = useState<Doctor[]>([]);
  const [appointments, setAppointments] = useState<Appointment[]>([]);
  const [detail, setDetail] = useState<Detail | null>(null);
  const [dashboard, setDashboard] = useState<Dashboard | null>(null);
  const [docs, setDocs] = useState<Doc[]>([]);
  const [invoices, setInvoices] = useState<Invoice[]>([]);
  const [accounts, setAccounts] = useState<Account[]>([]);
  const [audit, setAudit] = useState<Audit[]>([]);
  const [hours, setHours] = useState<Hours | null>(null);
  const [trends, setTrends] = useState<Trend[]>([]);
  const [date, setDate] = useState(today);
  const [search, setSearch] = useState("");
  const [tab, setTab] = useState("Overview");
  const [visitId, setVisitId] = useState("");
  const [modal, setModal] = useState<Modal | null>(null);
  const [toast, setToast] = useState("");
  const [error, setError] = useState("");
  const [loading, setLoading] = useState(true);
  const [refresh, setRefresh] = useState(0);
  const [mobileMenu, setMobileMenu] = useState(false);
  const notify = (s: string) => {
    setToast(s);
    setTimeout(() => setToast(""), 6000);
  };
  const done = (message = "Saved successfully.", id?: string) => {
    setModal(null);
    notify(message);
    setRefresh((n) => n + 1);
    if (id) router.push("/app/patients/" + id);
  };
  const load = useCallback(
    async (signal: AbortSignal) => {
      setError("");
      setLoading(true);
      try {
        const [p, d] = await Promise.all([
          api<Patient[]>("patients", undefined, "GET", signal),
          api<Doctor[]>("doctors", undefined, "GET", signal),
        ]);
        if (signal.aborted) return;
        setPatients(p);

        setDoctors(d);
        if (patientId)
          setDetail(
            await api<Detail>(
              "patients/" + patientId,
              undefined,
              "GET",
              signal,
            ),
          );
        else if (page === "overview") {
          const [s, a] = await Promise.all([
            api<Dashboard>("dashboard", undefined, "GET", signal),
            api<Appointment[]>(
              "appointments?date=" + date,
              undefined,
              "GET",
              signal,
            ),
          ]);
          setDashboard(s);
          setAppointments(a);
        } else if (page === "appointments")
          setAppointments(
            await api<Appointment[]>(
```

```
              "appointments?date=" + date,
              undefined,
              "GET",
              signal,
            ),
          );
        else if (page === "documents")
          setDocs(await api<Doc[]>("documents", undefined, "GET", signal));
        else if (page === "invoices")
          setInvoices(
            await api<Invoice[]>("invoices", undefined, "GET", signal),
          );
        else if (page === "accounts")
          setAccounts(await api<Account[]>("users", undefined, "GET", signal));
        else if (page === "audit")
          setAudit(await api<Audit[]>("audit", undefined, "GET", signal));
        else if (page === "settings")
          setHours(await api<Hours>("hours", undefined, "GET", signal));
        else if (page === "reports")
          setTrends(
            (
              await api<{ counts: Trend[] }>(
                "reports/visits?id=all",
                undefined,
                "GET",
                signal,
              )
            ).counts,
          );
      } catch (e) {
        if (signal.aborted) return;
        setError(
          e instanceof Error ? e.message : "The records could not be loaded.",
        );
      } finally {
        if (!signal.aborted) setLoading(false);
      }
    },
    [page, patientId, date],
  );
  useEffect(() => {
    const controller = new AbortController();
    void load(controller.signal);
    return () => controller.abort();
  }, [load, refresh]);
  useEffect(() => {
    setMobileMenu(false);
    setTab("Overview");
  }, [pathname]);
  useEffect(() => {
    if (page !== "patients" || patientId) return;
    let active = true;
    const t = setTimeout(() => {
      api<Patient[]>("patients?q=" + encodeURIComponent(search))
        .then((p) => {
          if (active) setPatients(p);
        })
        .catch((e) => {
          if (active) setError(e.message);
        });
    }, 250);
    return () => {
      active = false;
      clearTimeout(t);
    };
  }, [search, page, patientId, refresh]);
  const act = async (action: () => Promise<unknown>, message: string) => {
    try {
      await action();
      notify(message);
      setRefresh((n) => n + 1);
    } catch (e) {
      notify(
        "Error: " + (e instanceof Error ? e.message : "The change failed."),
      );
    }
  };
```

```
const nav = [
  { id: "overview", label: "Today", icon: LayoutDashboard },
  { id: "patients", label: "Patients", icon: Users },
  { id: "appointments", label: "Appointments", icon: CalendarDays },
  ...(doctor ? [{ id: "documents", label: "Documents", icon: Files }] : []),
  { id: "invoices", label: "Invoices", icon: ReceiptText },
  { id: "reports", label: "Reports", icon: ChartNoAxesCombined },
];

const admin = [
  ...(doctor
    ? [
        { id: "accounts", label: "Staff accounts", icon: Users },
        { id: "audit", label: "Audit history", icon: ShieldCheck },
      ]
    : []),
  { id: "settings", label: "Practice settings", icon: Settings2 },
];
const current =
  detail && detail.patient.id === patientId ? detail.patient : undefined;
const activeVisit =
  detail?.visits.find((v) => v.id === visitId) ||
  detail?.visits.find((v) => v.status === "open") ||
  detail?.visits[0];
const latest = detail?.clinical?.find(
  (c) =>
    c.visitId === activeVisit?.id && c.revision === activeVisit?.revision,
);
const pageTitle = patientId
  ? "Patient record"
  : (
      {
        overview: "Today at the practice",
        patients: "Patient directory",
        appointments: "Appointments",
        documents: "Diagnostic documents",
        invoices: "Manual invoices",
        reports: "Practice reports",
        accounts: "Staff accounts",
        audit: "Audit history",
        settings: "Practice settings",
      } as Record<string, string>
    )[page] || "Practice overview";
function appointmentTable(list: Appointment[]) {
  return list.length ? (
    <>
      <div className="table-wrap">
        <table>
          <thead>
            <tr>
              <th>TIME</th>
              <th>PATIENT</th>
              <th className="desktop-cell">DOCTOR</th>
              <th>STATUS</th>
              <th>
                <span className="sr-only">Actions</span>
              </th>
            </tr>
          </thead>
          <tbody>
            {list.map((a) => (
              <tr key={a.id}>
                <td className="time-cell">
                  {timeText(a.startAt)}

                  <span className="cell-sub">
                    {Math.round(
                      (Date.parse(a.endAt) - Date.parse(a.startAt)) / 60000,
                    )}{" "}
                    min
                  </span>
                </td>
                <td>
                  <Link href={"/app/patients/" + a.patientId}>
                    <Person
                      patient={{
                        firstName: a.firstName || "",
                        lastName: a.lastName || "",
```

```
                      }}
                      sub={a.mrn}
                    />
                  </Link>
                </td>
                <td className="desktop-cell">{a.doctorName}</td>
                <td>
                  <Status value={a.status} />
                </td>
                <td>
                  <button
                    className="icon-button"
                    aria-label={
                      "Manage appointment for " +
                      a.firstName +
                      " " +
                      a.lastName
                    }
                    onClick={() =>
                      setModal({
                        kind: "appointment-actions",
                        appointment: a,
                      })
                    }
                  >
                    <MoreHorizontal size={18} />
                  </button>
                </td>
              </tr>
            ))}
          </tbody>
        </table>
      </div>
      <div className="table-footer">
        <span>{list.length} appointments</span>
        <span>Jamaica time</span>
      </div>
    </>
  ) : (
    <Empty title="No appointments on this date.">
      Choose another date or book a visit.
    </Empty>

  );
}
return (
  <div className="app-shell">
    <a className="skip-link" href="#main-content">
      Skip to main content
    </a>
    <header className="practice-header">
      <Link href="/app" className="brand">
        <span className="brand-mark">
          p<span>+</span>
        </span>
        <span>
          PracticeChart
          <span className="brand-sub">ELECTRONIC HEALTH RECORDS</span>
        </span>
      </Link>
      <p className="nav-caption">WORKSPACE</p>
      <nav aria-label="Main navigation">
        {nav.map((n) => (
          <Link
            href={"/app/" + (n.id === "overview" ? "" : n.id)}
            key={n.id}
            className={page === n.id ? "active" : ""}
            aria-current={page === n.id ? "page" : undefined}
          >
            <n.icon size={18} strokeWidth={1.6} />
            {n.label}
          </Link>
        ))}
      </nav>
      <div className="nav-separator" />
      <p className="nav-caption">MANAGEMENT</p>
      <nav aria-label="Management navigation">
        {admin.map((n) => (
```

```
            <Link
              key={n.id}
              href={"/app/" + n.id}
              className={page === n.id ? "active" : ""}
            >
              <n.icon size={18} strokeWidth={1.6} />
              {n.label}
            </Link>
          ))}
        </nav>
        <div className="sidebar-bottom">
          <div className="sidebar-note">
            <ShieldCheck size={18} />
            <strong>Careful by design.</strong>Access follows your role. Every
            record change leaves a history.
          </div>
          <div className="user-block">
            <span className="avatar">
              {actor.name
                .replace("Dr. ", "")
                .split(" ")
                .map((n) => n[0])
                .join("")
                .slice(0, 2)}
            </span>
            <div>
              <strong>{actor.name}</strong>
              <small>{doctor ? "Doctor" : "Front desk"}</small>
            </div>
            <button
              className="text-button"
              style={{ color: "#d9e9da", marginLeft: "auto" }}
              aria-label="Log out"
              onClick={() =>
                act(async () => {
                  await api("logout", {});
                  window.location.href = "/login";
                }, "Logged out.")
              }
            >
              <LogOut size={17} />
            </button>
          </div>
        </div>
      </header>
      <div className="app-main">
        <div className="topbar">
          <div className="topbar-location">
            <span>PracticeChart</span>
            <ChevronRight size={13} />
            <strong>{pageTitle}</strong>
          </div>
          <Link href="/app" className="brand mobile-brand">
            <span className="brand-mark">
              p<span>+</span>
            </span>
            PracticeChart
          </Link>
          <div className="topbar-actions">
            <span className="topbar-date">
              {dateText(new Date().toISOString(), {
                weekday: "short",
                month: "short",
                day: "numeric",
                year: "numeric",
              })}
            </span>
            <button
              className="button primary"
              onClick={() =>
                setModal({ kind: "appointment", patient: current })
              }
            >
              <Plus size={16} />
              Book appointment
            </button>
```

```
          </div>
        </div>
        <main id="main-content" className="workspace-content" tabIndex={-1}>
          <div className="demo-banner">
            <ShieldCheck size={14} />
            <span>Fictional practice / Use fictional data only</span>
            <span>{doctor ? "Doctor workspace" : "Front desk workspace"}</span>
          </div>
          <div className="page-heading">
            <div>
              <h1>
                {patientId && current
                  ? `${current.firstName} ${current.lastName}`
                  : pageTitle}
              </h1>
              <p>
                {page === "overview"
                  ? "Your day, your patients, and what comes next."
                  : patientId
                    ? "One record. A clear history."
                    : (
                        {
                          patients: "Find a record or register a new patient.",
                          appointments:
                            "Plan the day and keep each visit on track.",
                          documents:
                            "Private reports, linked to the right patient.",
                          invoices:
                            "Service charges only. No payment processing.",
                          reports:
                            "Clear outputs for the practice and the patient.",
                          accounts: "Manage staff access with care.",
                          audit:
                            "A record of access, changes, and corrections.",
                          settings: "Set availability and manage your account.",
                        } as Record<string, string>
                      )[page]}
              </p>
            </div>
            <div className="heading-actions">
              {page === "patients" && !patientId && (
                <button
                  className="button primary"
                  onClick={() => setModal({ kind: "patient" })}
                >
                  <UserPlus size={16} />
                  Register patient
                </button>
              )}
              {patientId && current && (
                <button
                  className="button"
                  onClick={() =>
                    setModal({ kind: "patient", patient: current })
                  }
                >
                  Edit details
                </button>
              )}
              {page === "documents" && (
                <button
                  className="button primary"
                  onClick={() => setModal({ kind: "document" })}
                >
                  <Plus size={16} />
                  Add report
                </button>
              )}
              {page === "invoices" && (
                <button
                  className="button primary"
                  onClick={() => setModal({ kind: "invoice" })}
                >
                  <Plus size={16} />
                  New invoice
                </button>
              )}
```

```
            {page === "accounts" && doctor && (
              <button
                className="button primary"
                onClick={() => setModal({ kind: "user" })}
              >
                <UserPlus size={16} />
                Add account
              </button>
            )}
          </div>
        </div>
        {error ? (
          <div className="error" role="alert">
            {error}{" "}
            <button
              className="text-button"
              onClick={() => setRefresh((n) => n + 1)}
            >
              Try again
            </button>
          </div>
        ) : loading ? (
          <div className="loading" role="status">
            Loading practice records...
          </div>
        ) : (
          <>
            {page === "overview" && dashboard && (
              <>
                <section
                  className="summary-strip"
                  aria-label="Practice totals"
                >
                  {[
                    {
                      label: "Today's appointments",
                      value: dashboard.counts.appointments,
                      note: "Confirmed, arrived, or in consultation",
                      icon: CalendarDays,
                    },
                    {
                      label: "Active patients",
                      value: dashboard.counts.patients,
                      note: "Registered in the practice",
                      icon: Users,
                    },
                    {
                      label: "Ready to file",
                      value: dashboard.counts.ready,
                      note: "Clinical entries completed",
                      icon: ClipboardList,
                    },
                    {
                      label: "Filed visits",
                      value: dashboard.counts.filed,
                      note: "Recorded in the visit logbook",
                      icon: Check,
                    },
                  ].map((s) => (
                    <div className="summary-stat" key={s.label}>
                      <span>
                        {s.label}
                        <s.icon size={16} strokeWidth={1.5} />
                      </span>
                      <strong>{s.value}</strong>
                      <small>{s.note}</small>
                    </div>
                  ))}
                </section>
                <div className="dashboard-grid">
                  <Panel
                    title="Patient worklist"
                    subtitle={dateText(date, {
                      weekday: "long",
                      day: "numeric",
                      month: "long",
                    })}
```

```
                    action={
                      <Link className="text-button" href="/app/appointments">
                        View schedule
                        <ArrowUpRight size={14} />
                      </Link>
                    }
                  >
                    {appointmentTable(appointments)}
                  </Panel>
                  <div className="stack">

                    <Panel title="Quick actions">
                      <div className="quick-links">
                        <button onClick={() => setModal({ kind: "patient" })}>
                          <UserPlus />
                          <span>
                            Register a patient
                            <small>Start a new patient record</small>
                          </span>
                          <ArrowUpRight size={15} />
                        </button>
                        <button onClick={() => setModal({ kind: "visit" })}>
                          <Stethoscope />
                          <span>
                            Start a walk-in visit
                            <small>Prepare the patient for the doctor</small>
                          </span>
                          <ArrowUpRight size={15} />
                        </button>
                        <Link href="/app/reports">
                          <FileText />
                          <span>
                            Open reports
                            <small>Patient and practice outputs</small>
                          </span>
                          <ArrowUpRight size={15} />
                        </Link>
                      </div>
                    </Panel>
                    <Panel
                      title="Ready to file"
                      subtitle="Completed by the doctor"
                    >
                      {dashboard.ready.length ? (
                        dashboard.ready.map((v) => (
                          <div className="ready-item" key={v.id}>
                            <div>
                              <Link href={"/app/patients/" + v.patientId}>
                                <Person patient={v} sub={v.mrn} />
                              </Link>
                            </div>
                            <button
                              className="text-button"
                              onClick={() =>
                                act(
                                  () =>
                                    api(`visits/${v.id}/file`, {
                                      expectedRevision: v.revision,
                                    }),
                                  "Visit filed in the logbook.",
                                )
                              }
                            >
                              <Check size={13} />
                              File visit
                            </button>

                          </div>
                        ))
                      ) : (
                        <Empty title="All caught up.">
                          Completed entries will appear here.
                        </Empty>
                      )}
                    </Panel>
                    <Panel title="Upcoming reminders">
                      {dashboard.reminders.length ? (
                        dashboard.reminders.map((r) => (
```

```
                          <div className="ready-item" key={r.id}>
                            <strong>
                              {r.firstName} {r.lastName}
                            </strong>
                            <p className="muted">
                              {dateText(r.startAt)} at {timeText(r.startAt)}
                            </p>
                          </div>
                        ))
                      ) : (
                        <Empty title="No reminders due.">
                          Reminders are created by the daily job.
                        </Empty>
                      )}
                    </Panel>
                  </div>
                </div>
              </>
            )}
            {page === "patients" && !patientId && (
              <Panel>
                <div className="toolbar">
                  <label className="search-input">
                    <span className="sr-only">Search patients</span>
                    <Search size={16} />
                    <input
                      placeholder="Search by name or medical record number"
                      value={search}
                      onChange={(e) => setSearch(e.target.value)}
                      type="search"
                    />
                  </label>
                  <span className="result-count">
                    {patients.length} records shown
                  </span>
                </div>
                {patients.length ? (
                  <div className="table-wrap">
                    <table>
                      <thead>
                        <tr>
                          <th>PATIENT</th>
                          <th>MEDICAL RECORD</th>
                          <th className="desktop-cell">DATE OF BIRTH</th>
                          <th className="desktop-cell">PHONE</th>
                          <th>STATUS</th>
                          <th>
                            <span className="sr-only">Open record</span>
                          </th>
                        </tr>
                      </thead>
                      <tbody>
                        {patients.map((p) => (
                          <tr key={p.id}>
                            <td>
                              <Link href={"/app/patients/" + p.id}>
                                <Person patient={p} />
                              </Link>
                            </td>
                            <td>{p.mrn}</td>
                            <td className="desktop-cell">
                              {dateText(p.dob)}
                            </td>
                            <td className="desktop-cell">{p.phone}</td>
                            <td>
                              <Status
                                value={
                                  p.mergedInto
                                    ? "linked"
                                    : p.archived
                                      ? "archived"
                                      : p.provisionalReason
                                        ? "provisional"
                                        : "active"
                                }
                              />
                            </td>
```

```
                            <td>
                              <Link
                                className="icon-button"
                                href={"/app/patients/" + p.id}
                                aria-label={
                                  "Open " + p.firstName + " " + p.lastName
                                }
                              >
                                <ArrowUpRight size={16} />
                              </Link>
                            </td>
                          </tr>
                        ))}
                      </tbody>
                    </table>
                  </div>
                ) : (
                  <Empty title="No matching patients.">
                    Try a different name or medical record number.
                  </Empty>
                )}
              </Panel>

            )}
            {page === "appointments" && (
              <Panel>
                <div className="toolbar">
                  <label>
                    Schedule date
                    <input
                      type="date"
                      value={date}
                      onChange={(e) => setDate(e.target.value)}
                    />
                  </label>
                  <span className="result-count">
                    {dateText(date, {
                      weekday: "long",
                      day: "numeric",
                      month: "long",
                    })}
                  </span>
                  <PrintLink
                    type="schedule"
                    id={date}
                    label="Print schedule"
                  />
                </div>
                {appointmentTable(appointments)}
              </Panel>
            )}
            {patientId && current && detail && (
              <>
                <Panel>
                  <div className="patient-header">
                    <span className="avatar large">{initials(current)}</span>
                    <div>
                      <h2>
                        {current.firstName} {current.lastName}
                      </h2>
                      <p>
                        {current.mrn} <span aria-hidden="true">/</span>{" "}
                        {dateText(current.dob)}{" "}
                        <span aria-hidden="true">/</span> {current.gender}
                      </p>
                    </div>
                    <div style={{ marginLeft: "auto" }}>
                      <Status
                        value={
                          current.archived
                            ? "archived"
                            : current.provisionalReason
                              ? "provisional"
                              : "active"
                        }
                      />
                    </div>
                  </div>
```

```
                  <div
                    className="record-sections"
                    role="tablist"
                    aria-label="Patient record sections"
                  >
                    {[
                      "Overview",
                      "Visits",
                      ...(doctor ? ["Clinical record", "Documents"] : []),
                    ].map((t) => (
                      <button
                        key={t}
                        role="tab"
                        id={"tab-" + t.replaceAll(" ", "-")}
                        aria-controls="patient-tab-panel"
                        aria-selected={tab === t}
                        tabIndex={tab === t ? 0 : -1}
                        onKeyDown={(e) => {
                          if (
                            [
                              "ArrowLeft",
                              "ArrowRight",
                              "Home",
                              "End",
                            ].includes(e.key)
                          ) {
                            e.preventDefault();
                            const tabs = Array.from(
                              e.currentTarget.parentElement!.querySelectorAll<HTMLButtonElement>(
                                "[role=tab]",
                              ),
                            );
                            let i = tabs.indexOf(e.currentTarget);
                            i =
                              e.key === "Home"
                                ? 0
                                : e.key === "End"
                                  ? tabs.length - 1
                                  : (i +
                                      (e.key === "ArrowRight" ? 1 : -1) +
                                      tabs.length) %
                                    tabs.length;
                            tabs[i].click();
                            tabs[i].focus();
                          }
                        }}
                        onClick={() => setTab(t)}
                      >
                        {t}
                      </button>
                    ))}
                  </div>
                </Panel>
                <div
                  role="tabpanel"
                  id="patient-tab-panel"
                  aria-labelledby={"tab-" + tab.replaceAll(" ", "-")}
                >
                  {tab === "Overview" && (
                    <div className="detail-grid">
                      <Panel title="Patient details">
                        <div className="panel-body">
                          <dl className="facts">
                            {[
                              ["Medical record number", current.mrn],
                              ["Date of birth", dateText(current.dob)],
                              ["Phone", current.phone],
                              ["Gender", current.gender],
                              ["Address", current.address],
                              [
                                "Emergency contact",
                                current.emergencyContact || "Not recorded",
                              ],
                              [
                                "Insurance reference",
                                current.insurance || "Not recorded",
                              ],
```

```
                          ].map(([k, v]) => (
                            <div key={k}>
                              <dt>{k}</dt>
                              <dd>{v}</dd>
                            </div>
                          ))}
                        </dl>
                        {current.provisionalReason && (
                          <div className="note-box">
                            Provisional record: {current.provisionalReason}
                            {doctor && !current.mergedInto && (
                              <button
                                className="text-button"
                                onClick={() =>
                                  setModal({
                                    kind: "reconcile",
                                    patient: current,
                                  })
                                }
                              >
                                Link to primary record
                              </button>
                            )}
                          </div>
                        )}
                        {current.mergedInto && (
                          <p className="note-box">
                            This record links to{" "}
                            <Link
                              className="text-button"
                              href={"/app/patients/" + current.mergedInto}
                            >
                              the primary record <ArrowUpRight size={14} />
                            </Link>
                            . Its earlier history remains here.
                          </p>
                        )}
                      </div>
                    </Panel>
                    <div className="stack">
                      <Panel title="Record actions">
                        <div className="quick-links">
                          <button
                            onClick={() =>
                              setModal({ kind: "visit", patient: current })
                            }
                          >
                            <Stethoscope />
                            <span>Start walk-in visit</span>
                            <Plus size={14} />
                          </button>
                          <button
                            onClick={() =>
                              setModal({
                                kind: "appointment",
                                patient: current,
                              })
                            }
                          >
                            <CalendarPlus />
                            <span>Book appointment</span>
                            <Plus size={14} />
                          </button>
                          <button
                            onClick={() =>
                              setModal({
                                kind: "invoice",
                                patient: current,
                              })
                            }
                          >
                            <ReceiptText />
                            <span>Create invoice</span>
                            <Plus size={14} />
                          </button>
                          {doctor && (
                            <a
```

```
                              href={`/print/history?id=${current.id}`}
                              target="_blank"
                              rel="noopener"
                            >
                              <FileText />
                              <span>Print patient report</span>
                              <ArrowUpRight size={14} />
                            </a>
                          )}
                          {doctor && (
                            <button
                              onClick={() =>
                                setModal({
                                  kind: "archive",
                                  patient: current,
                                })
                              }
                            >
                              <FolderOpen />
                              <span>
                                {current.archived
                                  ? "Restore patient record"
                                  : "Archive patient record"}
                              </span>
                              <ArrowRight size={14} />
                            </button>
                          )}
                        </div>
                      </Panel>
                      <Panel title="Recent visits">
                        <div className="panel-body">
                          <strong style={{ fontSize: 30, fontWeight: 500 }}>
                            {detail.visits.length}
                          </strong>
                          <p className="muted">
                            Visits linked to this record
                          </p>
                          <button
                            className="text-button"
                            onClick={() => setTab("Visits")}
                          >
                            View visit history
                            <ArrowRight size={14} />
                          </button>
                        </div>
                      </Panel>
                    </div>
                  </div>
                )}
                {tab === "Visits" && (
                  <div style={{ marginTop: 24 }}>
                    <Panel
                      title="Visit history"
                      subtitle="Filing a visit preserves the patient record"
                    >
                      {detail.visits.length ? (
                        detail.visits.map((v) => (
                          <article className="timeline-item" key={v.id}>
                            <div className="timeline-title">
                              <strong>
                                {dateText(v.visitAt)} / {v.doctorName}
                              </strong>
                              <Status value={v.status} />
                            </div>
                            <p className="muted">
                              {v.revision} clinical revisions{" "}
                              {v.filedAt
                                ? "/ Filed " + dateText(v.filedAt)
                                : ""}
                            </p>
                            <div className="action-row">
                              {doctor && (
                                <button
                                  className="button"
                                  onClick={() => {
                                    setVisitId(v.id);
```

```
                                    setTab("Clinical record");
                                  }}
                                >
                                  Open clinical record
                                </button>
                              )}
                              {v.status === "ready" && (
                                <button
                                  className="button primary"
                                  onClick={() =>
                                    act(
                                      () =>
                                        api(`visits/${v.id}/file`, {
                                          expectedRevision: v.revision,
                                        }),
                                      "Visit filed.",
                                    )
                                  }
                                >
                                  <Check size={14} />
                                  File visit
                                </button>
                              )}
                              <PrintLink
                                type="attendance"
                                id={v.id}
                                label="Attendance slip"
                              />
                              {doctor && v.revision > 0 && (
                                <>
                                  <PrintLink
                                    type="summary"
                                    id={v.id}
                                    label="Visit summary"
                                  />
                                  <PrintLink
                                    type="prescription"
                                    id={v.id}
                                    label="Prescription"
                                  />
                                  <PrintLink
                                    type="certificate"
                                    id={v.id}
                                    label="Certificate"
                                  />

                                </>
                              )}
                            </div>
                          </article>
                        ))
                      ) : (
                        <Empty title="No visits recorded.">
                          Start a walk-in visit or use an appointment.
                        </Empty>
                      )}
                    </Panel>
                  </div>
                )}
                {tab === "Clinical record" && doctor && (
                  <div style={{ marginTop: 24 }}>
                    {activeVisit ? (
                      <div className="detail-grid">
                        <Panel
                          title="Clinical entry"
                          subtitle="Doctor access only"
                        >
                          <div className="panel-body">
                            <label>
                              Visit
                              <select
                                value={activeVisit.id}
                                onChange={(e) => setVisitId(e.target.value)}
                              >
                                {detail.visits.map((v) => (
                                  <option key={v.id} value={v.id}>
                                    {dateText(v.visitAt)} / {v.status} /
                                    revision {v.revision}
```

```
                                  </option>
                                ))}
                              </select>
                            </label>
                            <div style={{ marginTop: 24 }}>
                              <ClinicalEditor
                                key={
                                  activeVisit.id +
                                  ":" +
                                  activeVisit.revision
                                }
                                visit={activeVisit}
                                revision={latest}
                                saved={() => {
                                  notify("Clinical revision saved.");
                                  setRefresh((n) => n + 1);
                                  setTimeout(
                                    () => setTab("Clinical record"),
                                    10,
                                  );
                                }}
                              />
                            </div>

                          </div>
                        </Panel>
                        <Panel
                          title="Previous clinical entries"
                          subtitle="Earlier revisions stay available"
                        >
                          {detail.clinical
                            ?.filter((c) => c.visitId === activeVisit.id)
                            .map((c) => (
                              <article className="timeline-item" key={c.id}>
                                <div className="timeline-title">
                                  <strong>Revision {c.revision}</strong>
                                  <Status value={c.state} />
                                </div>
                                <small>
                                  {dateText(c.createdAt)} / {c.authorName}
                                </small>
                                {c.reason && (
                                  <p>
                                    <strong>Reason:</strong> {c.reason}
                                  </p>
                                )}
                                {Object.entries(c.content)
                                  .filter(
                                    ([k, v]) =>
                                      k !== "medications" && Boolean(v),
                                  )
                                  .map(([k, v]) => (
                                    <p key={k}>
                                      <strong>
                                        {k.replace(/([A-Z])/g, " $1")}:
                                      </strong>{" "}
                                      {String(v)}
                                    </p>
                                  ))}
                                {c.content.medications.map((m, i) => (
                                  <p key={i}>
                                    <strong>{m.medication}</strong> /{" "}
                                    {m.dosage}, {m.route}, {m.frequency},{" "}
                                    {m.duration}, quantity {m.quantity}.{" "}
                                    {m.instructions}
                                  </p>
                                ))}
                              </article>
                            )) || <Empty />}
                        </Panel>
                      </div>
                    ) : (
                      <Panel>
                        <Empty title="Start a visit first.">
                          Use Record actions to start the patient visit.
                        </Empty>
                      </Panel>
                    )}
```

```
                    </div>

                  )}
                  {tab === "Documents" && doctor && (
                    <div style={{ marginTop: 24 }}>
                      <div
                        className="action-row"
                        style={{ marginBottom: 20 }}
                      >
                        <button
                          className="button primary"
                          onClick={() =>
                            setModal({ kind: "document", patient: current })
                          }
                        >
                          <Plus size={16} />
                          Add report
                        </button>
                        <a className="button" href="/api/sample-report">
                          Download sample report
                        </a>
                      </div>
                      <DocumentCards
                        docs={detail.documents || []}
                        onReplace={(d) =>
                          setModal({
                            kind: "document",
                            patient: current,
                            replacesId: d.id,
                          })
                        }
                      />
                    </div>
                  )}
                </div>
              </>
            )}
            {page === "documents" && doctor && (
              <>
                <p className="note-box" style={{ marginBottom: 24 }}>
                  Reports are private. In the public demo, upload only the{" "}
                  <a className="text-button" href="/api/sample-report">
                    supplied sample report
                    <Download size={13} />
                  </a>
                  .
                </p>
                <DocumentCards docs={docs} />
              </>
            )}
            {page === "invoices" && (
              <Panel>
                {invoices.length ? (
                  <div className="table-wrap">
                    <table>
                      <thead>
                        <tr>

                          <th>INVOICE</th>
                          <th>PATIENT</th>
                          <th>TOTAL (JMD)</th>
                          <th>STATUS</th>
                          <th>ACTIONS</th>
                        </tr>
                      </thead>
                      <tbody>
                        {invoices.map((i) => (
                          <tr key={i.id}>
                            <td>
                              {i.number}
                              <span className="cell-sub">
                                {dateText(i.createdAt)}
                              </span>
                            </td>
                            <td>
                              {i.firstName} {i.lastName}
                            </td>
                            <td>{money(i.totalCents)}</td>
```

```
                            <td>
                              <Status value={i.status} />
                            </td>
                            <td>
                              <div className="action-row">
                                <PrintLink type="invoice" id={i.id} />
                                {i.status === "draft" && (
                                  <button
                                    className="button"
                                    onClick={() =>
                                      act(
                                        () =>
                                          api("invoices/" + i.id, {
                                            version: i.version,
                                            status: "issued",
                                          }),
                                        "Invoice issued.",
                                      )
                                    }
                                  >
                                    Issue
                                  </button>
                                )}
                                {i.status === "issued" && (
                                  <button
                                    className="button danger"
                                    onClick={() =>
                                      setModal({ kind: "void", invoice: i })
                                    }
                                  >
                                    Void
                                  </button>
                                )}
                              </div>
                            </td>

                          </tr>
                        ))}
                      </tbody>
                    </table>
                  </div>
                ) : (
                  <Empty title="No invoices yet.">
                    Create an invoice from manually entered service charges.
                  </Empty>
                )}
              </Panel>
            )}
            {page === "reports" && (
              <>
                <div className="report-grid">
                  <Panel>
                    <div className="report-card">
                      <CalendarDays size={24} />
                      <h3>Appointment schedule</h3>
                      <p>
                        Print the appointments for one day. Clinical notes are
                        not included.
                      </p>
                      <label>
                        Date
                        <input
                          type="date"
                          value={date}
                          onChange={(e) => setDate(e.target.value)}
                        />
                      </label>
                      <PrintLink
                        type="schedule"
                        id={date}
                        label="Open schedule"
                      />
                    </div>
                  </Panel>
                  {doctor && (
                    <Panel>
                      <div className="report-card">
                        <ClipboardList size={24} />
```

```
                        <h3>Patient history</h3>
                        <p>
                          Print the patient's clinical history, medications,
                          and diagnostic report list.
                        </p>
                        <PatientReportSelect patients={patients} />
                      </div>
                    </Panel>
                  )}
                  <Panel>
                    <div className="report-card">
                      <Activity size={24} />
                      <h3>Visit logbook</h3>

                      <p>
                        View visit counts and filed visits. This report
                        contains no clinical details.
                      </p>
                      <PrintLink
                        type="visits"
                        id="all"
                        label="Open logbook"
                      />
                    </div>
                  </Panel>
                </div>
                <div style={{ marginTop: 24 }}>
                  <Panel
                    title="Visits by day"
                    subtitle="Actual counts from the fictional database"
                  >
                    <div className="panel-body">
                      {trends.length ? (
                        trends.map((t) => (
                          <div className="chart-row" key={t.day}>
                            <span>{dateText(t.day)}</span>
                            <div
                              className="chart-bar"
                              style={{
                                width:
                                  Math.max(
                                    2,
                                    (Number(t.visits) /
                                      Math.max(
                                        ...trends.map((v) =>
                                          Number(v.visits),
                                        ),
                                      )) *
                                      60,
                                  ) + "%",
                              }}
                            />
                            <strong>
                              {t.visits} visits / {t.filed} filed
                            </strong>
                          </div>
                        ))
                      ) : (
                        <Empty title="No visit counts yet." />
                      )}
                    </div>
                  </Panel>
                </div>
                <p className="note-box" style={{ marginTop: 24 }}>
                  A patient report is a reading copy. It is not a full
                  database backup. Backups and restores use separate operator
                  commands.
                </p>
              </>

            )}
            {page === "accounts" && doctor && (
              <Panel>
                <div className="table-wrap">
                  <table>
                    <thead>
                      <tr>
                        <th>STAFF MEMBER</th>
```

```
                        <th>USERNAME</th>
                        <th>ROLE</th>
                        <th>STATUS</th>
                        <th>ACTION</th>
                      </tr>
                    </thead>
                    <tbody>
                      {accounts.map((u) => (
                        <tr key={u.id}>
                          <td>{u.name}</td>
                          <td>{u.username}</td>
                          <td>
                            {u.role === "doctor" ? "Doctor" : "Front desk"}
                          </td>
                          <td>
                            <Status
                              value={u.active ? "active" : "disabled"}
                            />
                          </td>
                          <td>
                            <button
                              className="button"
                              onClick={() =>
                                setModal({ kind: "user-edit", account: u })
                              }
                            >
                              {u.protected && demo ? "View access" : "Manage"}
                            </button>
                          </td>
                        </tr>
                      ))}
                    </tbody>
                  </table>
                </div>
              </Panel>
            )}
            {page === "audit" && doctor && (
              <Panel
                title="Access and change history"
                subtitle="Latest 200 events. Clinical revisions are stored with each visit."
              >
                <div className="table-wrap">
                  <table>
                    <thead>
                      <tr>
                        <th>TIME</th>
                        <th>STAFF MEMBER</th>
                        <th>ACTION</th>
                        <th>RECORD TYPE</th>
                        <th>DETAIL</th>
                      </tr>
                    </thead>
                    <tbody>
                      {audit.map((a) => (
                        <tr key={a.id}>
                          <td>
                            {dateText(a.createdAt)}
                            <span className="cell-sub">
                              {timeText(a.createdAt)}
                            </span>
                          </td>
                          <td>{a.actorName || "System"}</td>
                          <td>{a.action.replaceAll("_", " ")}</td>
                          <td>{a.entity.replaceAll("_", " ")}</td>
                          <td>
                            <small>
                              {a.entityId.slice(0, 8)}
                              {Object.keys(a.metadata).length > 0
                                ? " / " + JSON.stringify(a.metadata)
                                : ""}
                            </small>
                          </td>
                        </tr>
                      ))}
                    </tbody>
                  </table>
                </div>
```

```
              </Panel>
            )}
            {page === "settings" && (
              <div className="detail-grid">
                <Panel
                  title="Doctor availability"
                  subtitle="Jamaica time. Monday is day 1."
                >
                  <div className="table-wrap">
                    <table>
                      <thead>
                        <tr>
                          <th>DOCTOR</th>
                          <th>DAY</th>
                          <th>HOURS</th>
                        </tr>
                      </thead>
                      <tbody>
                        {hours?.hours.map((h) => (
                          <tr key={h.doctorId + h.weekday}>
                            <td>{h.doctorName}</td>
                            <td>
                              {
                                [
                                  "",
                                  "Monday",
                                  "Tuesday",
                                  "Wednesday",
                                  "Thursday",
                                  "Friday",
                                  "Saturday",
                                  "Sunday",
                                ][h.weekday]
                              }
                            </td>
                            <td>
                              {String(Math.floor(h.startMin / 60)).padStart(
                                2,
                                "0",
                              )}
                              :{String(h.startMin % 60).padStart(2, "0")} to{" "}
                              {String(Math.floor(h.endMin / 60)).padStart(
                                2,
                                "0",
                              )}
                              :{String(h.endMin % 60).padStart(2, "0")}
                            </td>
                          </tr>
                        ))}
                      </tbody>
                    </table>
                  </div>
                  {doctor && (
                    <div className="panel-body action-row">
                      <button
                        className="button"
                        onClick={() => setModal({ kind: "hours" })}
                      >
                        Change hours
                      </button>
                      <button
                        className="button"
                        onClick={() => setModal({ kind: "block" })}
                      >
                        Block time
                      </button>
                    </div>
                  )}
                  {hours?.blocks.map((b) => (
                    <div className="timeline-item" key={b.id}>
                      <strong>{b.doctorName}</strong>
                      <p>
                        {dateText(b.startAt)} / {timeText(b.startAt)} to{" "}
                        {timeText(b.endAt)}
                      </p>
                      {doctor && (
                        <button
```

```
                          className="text-button"
                          onClick={() =>
                            act(
                              () => api("blocks/" + b.id, {}),
                              "Time block removed.",
                            )
                          }
                        >
                          Remove block
                        </button>
                      )}
                    </div>
                  ))}
                </Panel>
                <div className="stack">
                  <Panel title="Your account">
                    <div className="panel-body">
                      <p>
                        <strong>{actor.name}</strong>
                      </p>
                      <p className="muted">
                        {actor.username} / {doctor ? "Doctor" : "Front desk"}
                      </p>
                      <button
                        className="button"
                        style={{ marginTop: 20 }}
                        onClick={() => setModal({ kind: "password" })}
                      >
                        <LockKeyhole size={16} />
                        Change password
                      </button>
                      <button
                        className="text-button"
                        onClick={() =>
                          act(async () => {
                            await api("logout", {});
                            window.location.href = "/login";
                          }, "Logged out.")
                        }
                      >
                        Log out
                      </button>
                    </div>
                  </Panel>
                  <Panel title="Recovery and support">
                    <div className="panel-body">
                      <p className="muted">
                        Database backups are separate from patient reports.
                        The group start guide explains the operator backup and
                        restore commands.
                      </p>
                      <p className="note-box" style={{ marginTop: 16 }}>
                        Course demo only. Human training and acceptance remain
                        pending.
                      </p>
                    </div>
                  </Panel>
                </div>

              </div>
            )}
          </>
        )}
      </main>
    </div>
    <nav className="mobile-nav" aria-label="Mobile navigation">
      {nav.slice(0, 3).map((n) => (
        <Link
          key={n.id}
          href={"/app/" + (n.id === "overview" ? "" : n.id)}
          className={page === n.id ? "active" : ""}
        >
          <n.icon size={20} />
          {n.label}
        </Link>
      ))}
      <button
```

```
          onClick={() => setMobileMenu(!mobileMenu)}
          aria-expanded={mobileMenu}
          aria-label="More navigation"
        >
          <MoreHorizontal size={22} />
          More
        </button>
      </nav>
      {mobileMenu && (
        <div className="mobile-menu-panel">
          {[...nav.slice(3), ...admin].map((n) => (
            <Link key={n.id} href={"/app/" + n.id}>
              <n.icon size={18} />
              {n.label}
            </Link>
          ))}
        </div>
      )}
      {toast && (
        <div
          role="status"
          className={"toast " + (toast.startsWith("Error:") ? "error" : "")}
        >
          {toast}
        </div>
      )}
      {modal && (
        <Dialog
          title={
            (
              {
                patient: modal.patient
                  ? "Edit patient details"
                  : "Register patient",
                appointment: modal.appointment
                  ? "Reschedule appointment"
                  : "Book appointment",

                "appointment-actions": "Appointment actions",
                invoice: "Create manual invoice",
                void: "Void invoice",
                user: "Add staff account",
                "user-edit": "Manage staff access",
                visit: "Start walk-in visit",
                password: "Change your password",
                archive: modal.patient?.archived
                  ? "Restore patient record"
                  : "Archive patient record",
                reconcile: "Link provisional record",
                document: "Add private diagnostic report",
                hours: "Change doctor hours",
                block: "Block doctor time",
              } as Record<string, string>
            )[modal.kind]
          }
          close={() => setModal(null)}
        >
          {modal.kind === "patient" && (
            <PatientForm patient={modal.patient} done={done} />
          )}
          {modal.kind === "appointment" && (
            <BookingForm
              patients={patients}
              doctors={doctors}
              patient={modal.patient}
              appointment={modal.appointment}
              done={() => done("Appointment confirmed.")}
            />
          )}
          {modal.kind === "appointment-actions" && modal.appointment && (
            <div className="clinical-form">
              <p>
                {modal.appointment.firstName} {modal.appointment.lastName} /{" "}
                {dateText(modal.appointment.startAt)} at{" "}
                {timeText(modal.appointment.startAt)}
              </p>
              <Status value={modal.appointment.status} />
              {modal.appointment.status === "confirmed" && (
```

```
              <>
                <button
                  className="button primary"
                  onClick={() =>
                    act(async () => {
                      await api(
                        `appointments/${modal.appointment!.id}/status`,
                        {
                          version: modal.appointment!.version,
                          status: "arrived",
                        },
                      );
                      setModal(null);
                    }, "Arrival recorded.")
                  }
                >
                  Mark arrived
                </button>
                <button
                  className="button"
                  onClick={() =>
                    setModal({
                      kind: "appointment",
                      appointment: modal.appointment,
                    })
                  }
                >
                  Reschedule
                </button>
                <Form
                  label="Cancel appointment"
                  submit={async () => {
                    await api(
                      `appointments/${modal.appointment!.id}/status`,
                      {
                        version: modal.appointment!.version,
                        status: "cancelled",
                      },
                    );
                    done("Appointment cancelled.");
                  }}
                >
                  <p className="note-box">
                    Cancel this appointment and release the time slot? The
                    booking history will remain.
                  </p>
                </Form>
              </>
            )}
            {["confirmed", "arrived"].includes(modal.appointment.status) && (
              <button
                className="button"
                onClick={() =>
                  act(async () => {
                    await api("visits", {
                      patientId: modal.appointment!.patientId,
                      doctorId: modal.appointment!.doctorId,
                      appointmentId: modal.appointment!.id,
                    });
                    const p = modal.appointment!.patientId;
                    setModal(null);
                    router.push("/app/patients/" + p);
                  }, "Visit started.")
                }
              >
                Start visit
              </button>
            )}
          </div>
        )}

        {modal.kind === "visit" && (
          <Form
            label="Start visit"
            submit={async (d) => {
              await api("visits", {
                patientId: field(d, "patientId"),
```

```
                doctorId: field(d, "doctorId"),
              });
              done("Visit started.", field(d, "patientId"));
            }}
          >
            <PatientSelect patients={patients} selected={modal.patient} />
            <label>
              Doctor
              <select name="doctorId" required>
                {doctors.map((d) => (
                  <option key={d.id} value={d.id}>
                    {d.name}
                  </option>
                ))}
              </select>
            </label>
            <p className="note-box">
              This starts a walk-in record. It does not reserve an appointment
              slot.
            </p>
          </Form>
        )}
        {modal.kind === "invoice" && (
          <Form
            label="Create invoice draft"
            submit={async (d) => {
              const lines = [1, 2, 3]
                .filter((i) => field(d, "description" + i))
                .map((i) => ({
                  description: field(d, "description" + i),
                  quantity: Number(field(d, "quantity" + i)),
                  unitCents: Math.round(Number(field(d, "unit" + i)) * 100),
                }));
              await api("invoices", {
                patientId: field(d, "patientId"),
                lines,
              });
              done("Invoice draft created.");
            }}
          >
            <PatientSelect patients={patients} selected={modal.patient} />
            <p className="note-box">
              Enter service charges in JMD. This records an invoice, not a
              payment.
            </p>
            {[1, 2, 3].map((i) => (
              <div className="form-grid" key={i}>
                <label className="full">
                  Service {i}
                  {i > 1 ? " (optional)" : ""}
                  <input
                    name={"description" + i}
                    maxLength={200}
                    required={i === 1}
                  />
                </label>
                <label>
                  Quantity
                  <input
                    name={"quantity" + i}
                    type="number"
                    min="1"
                    max="1000"
                    step="1"
                    defaultValue="1"
                  />
                </label>
                <label>
                  Unit charge (JMD)
                  <input
                    name={"unit" + i}
                    type="number"
                    min="0"
                    max="1000000"
                    step="0.01"
                    inputMode="decimal"
                    defaultValue="0"
```

```
                />
              </label>
            </div>
          ))}
        </Form>
      )}
      {modal.kind === "void" && modal.invoice && (
        <Form
          label="Void invoice"
          submit={async (d) => {
            await api("invoices/" + modal.invoice!.id, {
              version: modal.invoice!.version,
              status: "void",
              reason: field(d, "reason"),
            });
            done("Invoice voided. Its record was kept.");
          }}
        >
          <p>
            {modal.invoice.number} / {money(modal.invoice.totalCents)}
          </p>
          <label>
            Reason
            <textarea name="reason" required maxLength={500} />
          </label>
        </Form>
      )}

      {modal.kind === "user" && (
        <Form
          label="Create account"
          submit={async (d) => {
            await api("users", Object.fromEntries(d));
            done("Staff account created.");
          }}
        >
          <label>
            Full name
            <input name="name" maxLength={100} required />
          </label>
          <label>
            Username
            <input
              name="username"
              pattern="[a-z0-9._-]{3,40}"
              autoCapitalize="none"
              required
            />
          </label>
          <label>
            Role
            <select name="role">
              <option value="front_desk">Front desk</option>
              <option value="doctor">Doctor</option>
            </select>
          </label>
          <label>
            Initial password
            <input
              name="password"
              type="password"
              minLength={12}
              maxLength={128}
              required
              autoComplete="new-password"
            />
          </label>
          <p className="field-help">
            Use at least 12 characters. Share credentials through a private
            channel.
          </p>
        </Form>
      )}
      {modal.kind === "user-edit" &&
        modal.account &&
        (modal.account.protected && demo ? (
          <p className="note-box">
            This supplied demo account is protected. It cannot be changed or
```

```
              disabled. Create a separate fictional account to test account
              management.
            </p>
          ) : (
            <Form
              label="Save access changes"
              submit={async (d) => {
                await api("users/" + modal.account!.id, {
                  version: modal.account!.version,
                  role: field(d, "role"),
                  active: field(d, "active") === "true",
                  ...(field(d, "password")
                    ? { password: field(d, "password") }
                    : {}),
                });
                done(
                  "Account access updated. Existing sessions were closed.",
                );
              }}
            >
              <p>
                {modal.account.name} / {modal.account.username}
              </p>
              <label>
                Role
                <select name="role" defaultValue={modal.account.role}>
                  <option value="front_desk">Front desk</option>
                  <option value="doctor">Doctor</option>
                </select>
              </label>
              <label>
                Access
                <select
                  name="active"
                  defaultValue={String(modal.account.active)}
                >
                  <option value="true">Active</option>
                  <option value="false">Disabled</option>
                </select>
              </label>
              <label>
                Reset password (optional)
                <input
                  name="password"
                  type="password"
                  minLength={12}
                  maxLength={128}
                  autoComplete="new-password"
                />
              </label>
            </Form>
          ))}
        {modal.kind === "password" &&
          (actor.protected && demo ? (
            <p className="note-box">
              The supplied demo password is protected. A separate fictional
              account can test password changes.
            </p>
          ) : (
            <Form
              label="Change password and log out"
              submit={async (d) => {
                await api("password", Object.fromEntries(d));
                window.location.href = "/login";
              }}
            >
              <label>
                Current password
                <input
                  name="currentPassword"
                  type="password"
                  autoComplete="current-password"
                  required
                />
              </label>
              <label>
```

```
                New password
                <input
                  name="password"
                  type="password"
                  minLength={12}
                  maxLength={128}
                  autoComplete="new-password"
                  required
                />
              </label>
            </Form>
          ))}
        {modal.kind === "archive" && modal.patient && (
          <Form
            label={
              modal.patient.archived ? "Restore record" : "Archive record"
            }
            submit={async () => {
              await api("patients/" + modal.patient!.id + "/archive", {
                version: modal.patient!.version,
                archived: !modal.patient!.archived,
              });
              done("Patient status updated.");
            }}
          >
            <p className="note-box">
              The patient history will remain available.{" "}
              {modal.patient.archived
                ? "The record will become active again."
                : "New bookings will be blocked for this record."}
            </p>
          </Form>
        )}
        {modal.kind === "reconcile" && modal.patient && (
          <Form
            label="Link records"
            submit={async (d) => {
              await api("patients/" + modal.patient!.id + "/reconcile", {
                targetId: field(d, "targetId"),
                reason: field(d, "reason"),
              });
              done("Provisional record linked. Earlier history was kept.");
            }}
          >
            <PatientSelect
              patients={patients}
              name="targetId"
              label="Primary record"
              excludeId={modal.patient.id}
            />
            <label>
              Reason
              <textarea
                name="reason"
                minLength={3}
                maxLength={500}
                required
              />
            </label>
            <p className="note-box">
              This keeps both MRNs and their history. The provisional record
              links to the primary record.
            </p>
          </Form>
        )}
        {modal.kind === "document" && (
          <Form
            label="Store private report"
            submit={async (d) => {
              await api("documents", d);
              done("Private report stored.");
            }}
          >
            <PatientSelect patients={patients} selected={modal.patient} />
            <label>
              Report title
              <input name="title" maxLength={160} required />
```

```
              </label>
              <label>
                Report type
                <select name="kind">
                  <option>Lab report</option>
                  <option>Diagnostic report</option>
                </select>
              </label>
              <label>
                Replaces document ID (optional)
                <input
                  name="replacesId"
                  defaultValue={modal.replacesId}
                  placeholder="Keep blank for a new report"
                />
              </label>
              <label>

                Report file
                <input
                  name="file"
                  type="file"
                  accept="application/pdf,image/png,image/jpeg"
                  required
                />
              </label>
              <p className="note-box">
                Maximum 4 MiB. Earlier reports stay available.{" "}
                {demo && (
                  <>
                    Use only the{" "}
                    <a className="text-button" href="/api/sample-report">
                      fictional sample report
                    </a>
                    .
                  </>
                )}
              </p>
            </Form>
          )}
          {modal.kind === "hours" && (
            <Form
              label="Save hours"
              submit={async (d) => {
                const start = field(d, "start").split(":").map(Number),
                  end = field(d, "end").split(":").map(Number);
                await api("hours", {
                  doctorId: field(d, "doctorId"),
                  weekday: Number(field(d, "weekday")),
                  startMin: start[0] * 60 + start[1],
                  endMin: end[0] * 60 + end[1],
                });
                done("Doctor hours updated.");
              }}
            >
              <label>
                Doctor
                <select name="doctorId">
                  {doctors.map((d) => (
                    <option key={d.id} value={d.id}>
                      {d.name}
                    </option>
                  ))}
                </select>
              </label>
              <label>
                Day
                <select name="weekday">
                  {[
                    "Monday",
                    "Tuesday",
                    "Wednesday",
                    "Thursday",

                    "Friday",
                    "Saturday",
                    "Sunday",
                  ].map((v, i) => (
```

```
                    <option key={v} value={i + 1}>
                      {v}
                    </option>
                  ))}
                </select>
              </label>
              <div className="form-grid">
                <label>
                  Start
                  <input
                    name="start"
                    type="time"
                    defaultValue="08:00"
                    required
                  />
                </label>
                <label>
                  End
                  <input name="end" type="time" defaultValue="17:00" required />
                </label>
              </div>
              <p className="note-box">
                Move any conflicting future appointments before reducing hours.
              </p>
            </Form>
          )}
          {modal.kind === "block" && (
            <Form
              label="Block time"
              submit={async (d) => {
                await api("blocks", {
                  doctorId: field(d, "doctorId"),
                  startAt: field(d, "start") + ":00-05:00",
                  endAt: field(d, "end") + ":00-05:00",
                });
                done("Time blocked.");
              }}
            >
              <label>
                Doctor
                <select name="doctorId">
                  {doctors.map((d) => (
                    <option key={d.id} value={d.id}>
                      {d.name}
                    </option>
                  ))}
                </select>
              </label>
              <label>
                Start (Jamaica time)
                <input name="start" type="datetime-local" required />
              </label>
              <label>
                End (Jamaica time)
                <input name="end" type="datetime-local" required />
              </label>
            </Form>
          )}
        </Dialog>
      )}
    </div>
  );
}
function DocumentCards({
  docs,
  onReplace,
}: {
  docs: Doc[];
  onReplace?: (doc: Doc) => void;
}) {
  return docs.length ? (
    <div className="document-list">
      {docs.map((d) => (
        <article className="document-card" key={d.id}>
          <FileText size={26} strokeWidth={1.5} />
          <h3>{d.title}</h3>
          <p>
```

```
            {d.firstName ? d.firstName + " " + d.lastName + " / " : ""}
            {d.kind}
          </p>
          <small>
            {dateText(d.createdAt)} / {(d.size / 1024).toFixed(1)} KiB
            {d.replacesId ? " / Replacement, earlier file kept" : ""}
          </small>
          <a className="button" href={"/api/documents/" + d.id}>
            <Download size={14} />
            Retrieve report
          </a>
          {onReplace && (
            <button
              className="text-button"
              onClick={() => {
                onReplace(d);
              }}
            >
              Add replacement
            </button>
          )}
        </article>
      ))}
    </div>
  ) : (
    <Panel>
      <Empty title="No diagnostic reports yet.">
        Add a private report to this patient record.
      </Empty>
    </Panel>
  );
}
function PatientReportSelect({ patients }: { patients: Patient[] }) {
  const [id, setId] = useState(patients[0]?.id || "");
  return (
    <>
      <PatientSelect
        patients={patients}
        selected={patients[0]}
        onChange={setId}
        includeArchived
      />
      {id && <PrintLink type="history" id={id} label="Open patient report" />}
    </>
  );
}
```

src/lib/auth.ts

```
import { createHash, randomBytes } from "node:crypto";
import { hash, verify } from "@node-rs/argon2";
import { and, eq, gt, sql } from "drizzle-orm";
import { db } from "./db";
import { users, sessions, auditEvents, type Staff } from "./schema";
export const SESSION_COOKIE = "practicechart_session";
export class AppError extends Error {
  constructor(
    public status: number,
    message: string,
  ) {
    super(message);
  }
}
export const digest = (s: string) =>
  createHash("sha256").update(s).digest("hex");
export const hashPassword = (s: string) =>
  hash(s, { memoryCost: 19456, timeCost: 2, parallelism: 1 });
export async function auditDenied(actor: Staff | null, entity: string) {
  await db
    .insert(auditEvents)
    .values({ actorId: actor?.id, action: "access_denied", entity });
}
export async function requireRole(actor: Staff, role = "doctor") {
  if (actor.role !== role) {
    await auditDenied(actor, "clinical_or_admin");
    throw new AppError(403, "Your account cannot use this function.");
  }
}
export async function getSession(token?: string): Promise<Staff | null> {
  if (!token) return null;
  const rows = await db
    .select({
      id: users.id,
      username: users.username,
      name: users.name,
      role: users.role,
      protected: users.protected,
      version: users.version,
    })
    .from(sessions)
    .innerJoin(users, eq(sessions.userId, users.id))
    .where(
      and(
        eq(sessions.id, digest(token)),
        gt(sessions.expiresAt, new Date()),
        eq(users.active, true),
      ),
    )
    .limit(1);
  return rows[0] ?? null;
}
let dummy: Promise<string> | undefined;
export async function login(username: string, password: string, ip: string) {
  const key = digest(username.toLowerCase().trim() + ":" + ip);

  const attempt = await db.execute(
    sql`INSERT INTO login_attempts(key,count,expires_at) VALUES(${key},1,now()+interval '15 minutes') ON
  CONFLICT(key) DO UPDATE SET count=CASE WHEN login_attempts.expires_at<now() THEN 1 ELSE
  login_attempts.count+1 END,expires_at=CASE WHEN login_attempts.expires_at<now() THEN now()+interval '15
  minutes' ELSE login_attempts.expires_at END RETURNING count`,
  );
  if (Number(attempt.rows[0].count) > 8)
    throw new AppError(
      429,
      "Too many login attempts. Try again in 15 minutes.",
    );
  const [u] = await db
    .select()
    .from(users)
    .where(eq(users.username, username.toLowerCase().trim()))
    .limit(1);
  dummy ??= hashPassword(randomBytes(32).toString("hex"));
  const valid = await verify(u?.passwordHash ?? (await dummy), password).catch(
    () => false,
```

```
  );
  if (!u || !u.active || !valid) {
    await db
      .insert(auditEvents)
      .values({ action: "login_failed", entity: "session" });
    throw new AppError(401, "The username or password is incorrect.");
  }
  const token = randomBytes(32).toString("hex");
  await db.transaction(async (tx) => {
    await tx
      .insert(sessions)
      .values({
        id: digest(token),
        userId: u.id,
        expiresAt: new Date(Date.now() + 8 * 3600000),
      });
    await tx.execute(sql`DELETE FROM login_attempts WHERE key=${key}`);
    await tx
      .insert(auditEvents)
      .values({ actorId: u.id, action: "login", entity: "session" });
  });
  return {
    token,
    user: { id: u.id, name: u.name, role: u.role, username: u.username },
  };
}
```

src/lib/db.ts

```
import { Pool } from "pg";
import { drizzle } from "drizzle-orm/node-postgres";
import * as schema from "./schema";
const globalDb = globalThis as unknown as { practicePool?: Pool };
export const pool =
  globalDb.practicePool ??
  new Pool({
    connectionString: process.env.EHR_DATABASE_URL || process.env.DATABASE_URL,
    max: 5,
    idleTimeoutMillis: 10000,
    connectionTimeoutMillis: 12000,
  });
if (process.env.NODE_ENV !== "production") globalDb.practicePool = pool;
if (pool.listenerCount("error") === 0)
  pool.on("error", () =>
    console.error("PracticeChart database connection closed."),
  );
export const db = drizzle(pool, { schema });
export type Tx = Parameters<Parameters<typeof db.transaction>[0]>[0];
export type Executor = typeof db | Tx;
```

src/lib/schema.ts

```
import {
  pgTable,
  uuid,
  text,
  boolean,
  integer,
  timestamp,
  date,
  jsonb,
} from "drizzle-orm/pg-core";
export const users = pgTable("users", {
  id: uuid().primaryKey().defaultRandom(),
  username: text().notNull().unique(),
  name: text().notNull(),
  role: text().notNull(),
  passwordHash: text("password_hash").notNull(),
  active: boolean().notNull().default(true),
  protected: boolean().notNull().default(false),
  version: integer().notNull().default(1),
  createdAt: timestamp("created_at", { withTimezone: true })
    .defaultNow()
    .notNull(),
});
export const sessions = pgTable("sessions", {
  id: text().primaryKey(),
  userId: uuid("user_id")
    .notNull()
    .references(() => users.id),
  expiresAt: timestamp("expires_at", { withTimezone: true }).notNull(),
  createdAt: timestamp("created_at", { withTimezone: true })
    .defaultNow()
    .notNull(),
});
export const patients = pgTable("patients", {
  id: uuid().primaryKey().defaultRandom(),
  mrn: text().notNull().unique(),
  firstName: text("first_name").notNull(),
  lastName: text("last_name").notNull(),
  dob: date().notNull(),
  phone: text().notNull(),
  address: text().notNull(),
  gender: text().notNull().default("Not stated"),
  emergencyContact: text("emergency_contact").notNull().default(""),
  insurance: text().notNull().default(""),
  provisionalReason: text("provisional_reason").notNull().default(""),
  mergedInto: uuid("merged_into"),
  archived: boolean().notNull().default(false),
  version: integer().notNull().default(1),
  createdAt: timestamp("created_at", { withTimezone: true })
    .defaultNow()
    .notNull(),
});
export const appointments = pgTable("appointments", {
  id: uuid().primaryKey().defaultRandom(),
  patientId: uuid("patient_id")
    .notNull()
    .references(() => patients.id),
  doctorId: uuid("doctor_id")
    .notNull()
    .references(() => users.id),
  startAt: timestamp("start_at", { withTimezone: true }).notNull(),
  endAt: timestamp("end_at", { withTimezone: true }).notNull(),
  status: text().notNull().default("confirmed"),
  version: integer().notNull().default(1),
  createdAt: timestamp("created_at", { withTimezone: true })
    .defaultNow()
    .notNull(),
});
export const visits = pgTable("visits", {
  id: uuid().primaryKey().defaultRandom(),
  patientId: uuid("patient_id")
    .notNull()
    .references(() => patients.id),
  doctorId: uuid("doctor_id")
```

```
    .notNull()
    .references(() => users.id),
  appointmentId: uuid("appointment_id").references(() => appointments.id),
  status: text().notNull().default("open"),
  revision: integer().notNull().default(0),
  visitAt: timestamp("visit_at", { withTimezone: true }).defaultNow().notNull(),
  filedAt: timestamp("filed_at", { withTimezone: true }),
});
export const clinicalRevisions = pgTable("clinical_revisions", {
  id: uuid().primaryKey().defaultRandom(),
  visitId: uuid("visit_id")
    .notNull()
    .references(() => visits.id),
  revision: integer().notNull(),
  authorId: uuid("author_id")
    .notNull()
    .references(() => users.id),
  state: text().notNull(),
  content: jsonb().notNull(),
  reason: text().notNull().default(""),
  createdAt: timestamp("created_at", { withTimezone: true })
    .defaultNow()
    .notNull(),
});
export const documents = pgTable("documents", {
  id: uuid().primaryKey().defaultRandom(),
  patientId: uuid("patient_id")
    .notNull()
    .references(() => patients.id),
  title: text().notNull(),
  kind: text().notNull(),
  pathname: text().notNull().unique(),
  mime: text().notNull(),
  size: integer().notNull(),
  sha256: text().notNull(),
  replacesId: uuid("replaces_id"),

  authorId: uuid("author_id")
    .notNull()
    .references(() => users.id),
  createdAt: timestamp("created_at", { withTimezone: true })
    .defaultNow()
    .notNull(),
});
export const invoices = pgTable("invoices", {
  id: uuid().primaryKey().defaultRandom(),
  number: text().notNull().unique(),
  patientId: uuid("patient_id")
    .notNull()
    .references(() => patients.id),
  lines: jsonb().notNull(),
  totalCents: integer("total_cents").notNull(),
  status: text().notNull().default("draft"),
  version: integer().notNull().default(1),
  reason: text().notNull().default(""),
  createdAt: timestamp("created_at", { withTimezone: true })
    .defaultNow()
    .notNull(),
});
export const auditEvents = pgTable("audit_events", {
  id: uuid().primaryKey().defaultRandom(),
  actorId: uuid("actor_id").references(() => users.id),
  action: text().notNull(),
  entity: text().notNull(),
  entityId: text("entity_id").notNull().default(""),
  metadata: jsonb().notNull().default({}),
  createdAt: timestamp("created_at", { withTimezone: true })
    .defaultNow()
    .notNull(),
});
export type Staff = Pick<
  typeof users.$inferSelect,
  "id" | "username" | "name" | "role" | "protected" | "version"
>;
```

src/lib/service.ts

```
import { and, eq, sql, type SQL } from "drizzle-orm";
import { z } from "zod";
import { db, type Executor } from "./db";
import {
  users,
  patients,
  appointments,
  visits,
  clinicalRevisions,
  documents,
  invoices,
  auditEvents,
  sessions,
  type Staff,
} from "./schema";
import { AppError, requireRole, hashPassword, digest } from "./auth";
import {
  patientInput,
  appointmentInput,
  clinicalInput,
  invoiceInput,
  invoiceTotal,
  userInput,
  idSchema,
} from "./validation";
type Row = Record<string, unknown>;
export async function rows(q: SQL, ex: Executor = db) {
  return (await ex.execute(q)).rows;
}
export async function event(
  ex: Executor,
  actor: Staff,
  action: string,
  entity: string,
  id = "",
  metadata: Row = {},
) {
  await ex
    .insert(auditEvents)
    .values({ actorId: actor.id, action, entity, entityId: id, metadata });
}
async function found<T>(list: T[], message = "The record was not found.") {
  if (!list.length) throw new AppError(404, message);
  return list[0];
}
export async function patientList(actor: Staff, search = "") {
  const term = `%${search.trim().slice(0, 100).replace(/[%_]/g, "")}%`;
  return db.transaction(async (tx) => {
    await event(tx, actor, "search", "patient");
    return rows(
      sql`SELECT
id,mrn,first_name,last_name,dob,phone,address,gender,provisional_reason,merged_into,archived,version FROM
patients WHERE (mrn ILIKE ${term} OR first_name ILIKE ${term} OR last_name ILIKE ${term} OR (first_name||'
 '||last_name) ILIKE ${term}) ORDER BY last_name,first_name LIMIT 100`,
      tx,

    );
  });
}
export async function patientDetail(actor: Staff, id: string) {
  idSchema.parse(id);
  return db.transaction(async (tx) => {
    const p = await found(
      await tx.select().from(patients).where(eq(patients.id, id)),
    );
    await event(tx, actor, "view", "patient", id);
    const visitList = await rows(
      sql`SELECT v.id,v.patient_id,v.doctor_id,v.status,v.revision,v.visit_at,v.filed_at,u.name AS
doctor_name FROM visits v JOIN users u ON u.id=v.doctor_id WHERE patient_id=${id} ORDER BY visit_at DESC`,
      tx,
    );
    const appointmentList = await rows(
      sql`SELECT a.*,u.name AS doctor_name FROM appointments a JOIN users u ON u.id=a.doctor_id WHERE
patient_id=${id} ORDER BY start_at DESC`,
      tx,
```

```
    );
    if (actor.role !== "doctor")
      return { patient: p, visits: visitList, appointments: appointmentList };
    const clinical = await rows(
      sql`SELECT c.*,u.name AS author_name FROM clinical_revisions c JOIN users u ON u.id=c.author_id JOIN visits v ON v.id=c.visit_id WHERE v.patient_id=${id} ORDER BY c.created_at DESC,c.revision DESC`,
      tx,
    );
    const files = await tx
      .select({
        id: documents.id,
        title: documents.title,
        kind: documents.kind,
        mime: documents.mime,
        size: documents.size,
        createdAt: documents.createdAt,
        replacesId: documents.replacesId,
      })
      .from(documents)
      .where(eq(documents.patientId, id));
    return {
      patient: p,
      visits: visitList,
      appointments: appointmentList,
      clinical,
      documents: files,
    };
  });
}
export async function savePatient(actor: Staff, input: unknown, id?: string) {
  const data = patientInput.parse(input);
  return db.transaction(async (tx) => {
    if (id) {
      idSchema.parse(id);
      const version = z
        .object({ version: z.number().int().positive() })
        .parse(input).version;
      const [p] = await tx
        .update(patients)
        .set({ ...data, version: version + 1 })
        .where(and(eq(patients.id, id), eq(patients.version, version)))
        .returning();
      if (!p)
        throw new AppError(
          409,
          "This patient changed. Reload the record before you save.",
        );
      await event(tx, actor, "update", "patient", id, { version });
      return { patient: p };
    }
    const duplicates = await rows(
      sql`SELECT id,mrn FROM patients WHERE lower(first_name)=lower(${data.firstName}) AND lower(last_name)=lower(${data.lastName}) AND dob=${data.dob}::date LIMIT 5`,
      tx,
    );
    const [p] = await tx
      .insert(patients)
      .values({ ...data, mrn: sql`DEFAULT` })
      .returning();
    await event(tx, actor, "create", "patient", p.id);
    return { patient: p, duplicates };
  });
}
export async function archivePatient(actor: Staff, id: string, input: unknown) {
  await requireRole(actor);
  const { version, archived } = z
    .object({ version: z.number().int(), archived: z.boolean() })
    .parse(input);
  return db.transaction(async (tx) => {
    const [p] = await tx
      .update(patients)
      .set({ archived, version: version + 1 })
      .where(
        and(eq(patients.id, idSchema.parse(id)), eq(patients.version, version)),
      )
      .returning();
    if (!p) throw new AppError(409, "This record changed. Reload it.");
```

```
    await event(tx, actor, archived ? "archive" : "restore", "patient", id);
    return p;
  });
}
export async function reconcilePatient(
  actor: Staff,
  id: string,
  input: unknown,
) {
  await requireRole(actor);
  const { targetId, reason } = z
    .object({ targetId: idSchema, reason: z.string().trim().min(3).max(500) })
    .parse(input);
  idSchema.parse(id);

  if (id === targetId) throw new AppError(400, "Choose a different patient.");
  return db.transaction(async (tx) => {
    const list = await rows(
      sql`SELECT * FROM patients WHERE id IN(${id}::uuid,${targetId}::uuid) ORDER BY id FOR UPDATE`,
      tx,
    );
    if (list.length !== 2) throw new AppError(404, "Both patients must exist.");
    const source = list.find((p) => p.id === id);
    const target = list.find((p) => p.id === targetId);
    if (
      !source?.provisional_reason ||
      source.merged_into ||
      target?.merged_into
    )
      throw new AppError(
        400,
        "Use an unresolved provisional record and an active primary record.",
      );
    await tx
      .update(patients)
      .set({
        mergedInto: targetId,
        archived: true,
        version: Number(source.version) + 1,
      })
      .where(eq(patients.id, id));
    await event(tx, actor, "reconcile", "patient", id, { targetId, reason });
    return { targetId };
  });
}
export async function schedule(actor: Staff, date?: string) {
  const day =
    date && /^\d{4}-\d{2}-\d{2}$/.test(date)
      ? date
      : new Intl.DateTimeFormat("en-CA", {
          timeZone: "America/Jamaica",
        }).format(new Date());
  return db.transaction(async (tx) => {
    await event(tx, actor, "view", "schedule");
    return rows(
      sql`SELECT a.*,p.mrn,p.first_name,p.last_name,u.name AS doctor_name FROM appointments a JOIN
    patients p ON p.id=a.patient_id JOIN users u ON u.id=a.doctor_id WHERE (a.start_at AT TIME ZONE
    'America/Jamaica')::date=${day}::date ORDER BY a.start_at`,
      tx,
    );
  });
}
export async function availability(
  actor: Staff,
  doctorId: string,
  date: string,
  options: { duration?: number; patientId?: string; excludeId?: string } = {},
) {
  idSchema.parse(doctorId);
  if (!/^\d{4}-\d{2}-\d{2}$/.test(date))

    throw new AppError(400, "Choose a valid date.");
  await event(db, actor, "view", "availability");
  const { duration, patientId, excludeId } = z
    .object({
      duration: z.number().int().min(1).max(240).default(30),
      patientId: idSchema.optional(),
      excludeId: idSchema.optional(),
```

```
    })
    .parse(options);
  return rows(
    sql`SELECT slot AS start_at,slot+make_interval(mins=>${duration}) AS end_at FROM doctor_hours h CROSS JOIN LATERAL generate_series((${date}::date+make_interval(mins=>h.start_min)) AT TIME ZONE 'America/Jamaica',(${date}::date+make_interval(mins=>h.end_min-${duration})) AT TIME ZONE 'America/Jamaica',interval '30 minutes') slot WHERE h.doctor_id=${doctorId}::uuid AND h.weekday=extract(isodow FROM ${date}::date) AND slot>now() AND NOT EXISTS(SELECT 1 FROM appointments a WHERE (a.doctor_id=h.doctor_id OR a.patient_id=${patientId ?? null}::uuid) AND (${excludeId ?? null}::uuid IS NULL OR a.id<>${excludeId ?? null}::uuid) AND a.status IN('confirmed','arrived','in_consultation') AND tstzrange(a.start_at,a.end_at,'[)') && tstzrange(slot,slot+make_interval(mins=>${duration}),'[)')) AND NOT EXISTS(SELECT 1 FROM schedule_blocks b WHERE b.doctor_id=h.doctor_id AND tstzrange(b.start_at,b.end_at,'[)') && tstzrange(slot,slot+make_interval(mins=>${duration}),'[)'))`,
  );
}
export async function bookAppointment(
  actor: Staff,
  input: unknown,
  id?: string,
) {
  const data = appointmentInput.parse(input);
  return db.transaction(async (tx) => {
    const p = await found(
      await tx.select().from(patients).where(eq(patients.id, data.patientId)),
    );
    if (p.archived || p.mergedInto)
      throw new AppError(400, "Choose an active patient record.");
    if (id) {
      idSchema.parse(id);
      const { version } = z
        .object({ version: z.number().int().positive() })
        .parse(input);
      const [a] = await tx
        .update(appointments)
        .set({
          ...data,
          startAt: new Date(data.startAt),
          endAt: new Date(data.endAt),
          version: version + 1,
        })
        .where(
          and(
            eq(appointments.id, id),
            eq(appointments.version, version),
            eq(appointments.status, "confirmed"),
          ),
        )
        .returning();

      if (!a)
        throw new AppError(
          409,
          "This appointment changed or has started. Reload the schedule.",
        );
      await event(tx, actor, "reschedule", "appointment", id, { version });
      return a;
    }
    const [a] = await tx
      .insert(appointments)
      .values({
        ...data,
        startAt: new Date(data.startAt),
        endAt: new Date(data.endAt),
      })
      .returning();
    await event(tx, actor, "confirm", "appointment", a.id);
    return a;
  });
}
export async function appointmentStatus(
  actor: Staff,
  id: string,
  input: unknown,
) {
  const { version, status } = z
    .object({
      version: z.number().int(),
      status: z.enum(["cancelled", "arrived"]),
```

```
    })
    .parse(input);
  return db.transaction(async (tx) => {
    const [a] = await tx
      .update(appointments)
      .set({ status, version: version + 1 })
      .where(
        and(
          eq(appointments.id, idSchema.parse(id)),
          eq(appointments.version, version),
          eq(appointments.status, "confirmed"),
        ),
      )
      .returning();
    if (!a)
      throw new AppError(
        409,
        "This appointment changed or cannot enter that state.",
      );
    await event(tx, actor, status, "appointment", id);
    return a;
  });
}
export async function startVisit(actor: Staff, input: unknown) {
  const data = z
    .object({

      patientId: idSchema,
      doctorId: idSchema,
      appointmentId: idSchema.optional(),
    })
    .parse(input);
  return db.transaction(async (tx) => {
    const p = await found(
      await tx.select().from(patients).where(eq(patients.id, data.patientId)),
    );
    if (p.archived) throw new AppError(400, "This patient record is archived.");
    await found(
      await rows(
        sql`SELECT id FROM users WHERE id=${data.doctorId}::uuid AND active AND role='doctor'`,
        tx,
      ),
      "Choose an active doctor.",
    );
    if (data.appointmentId) {
      const a = await found(
        await rows(
          sql`SELECT * FROM appointments WHERE id=${data.appointmentId}::uuid FOR UPDATE`,
          tx,
        ),
      );
      if (
        a.patient_id !== data.patientId ||
        a.doctor_id !== data.doctorId ||
        !["confirmed", "arrived"].includes(String(a.status))
      )
        throw new AppError(
          400,
          "Use a matching confirmed or arrived appointment.",
        );
      await tx
        .update(appointments)
        .set({ status: "in_consultation", version: Number(a.version) + 1 })
        .where(eq(appointments.id, data.appointmentId));
    }
    const [v] = await tx.insert(visits).values(data).returning();
    await event(tx, actor, "start", "visit", v.id);
    return v;
  });
}
export async function saveClinical(actor: Staff, id: string, input: unknown) {
  await requireRole(actor);
  idSchema.parse(id);
  const data = clinicalInput.parse(input);
  return db.transaction(async (tx) => {
    const v = await found(
      await rows(sql`SELECT * FROM visits WHERE id=${id}::uuid FOR UPDATE`, tx),
    );
```

```
    if (Number(v.revision) !== data.expectedRevision)
      throw new AppError(
        409,
        "Another doctor saved a revision. Reload this visit.",
      );
    const previous = await rows(
      sql`SELECT state FROM clinical_revisions WHERE visit_id=${id}::uuid ORDER BY revision DESC LIMIT 1`,
      tx,
    );
    if (
      previous[0]?.state === "final" &&
      (!data.reason || data.state !== "final")
    )
      throw new AppError(
        400,
        "Add a correction reason and save a final correction.",
      );
    const { expectedRevision, state, reason, ...content } = data;
    const revision = expectedRevision + 1;
    const [r] = await tx
      .insert(clinicalRevisions)
      .values({
        visitId: id,
        revision,
        authorId: actor.id,
        state,
        reason,
        content,
      })
      .returning();
    await tx
      .update(visits)
      .set({
        revision,
        status:
          v.status === "filed" ? "filed" : state === "final" ? "ready" : "open",
      })
      .where(eq(visits.id, id));
    if (state === "final")
      for (const medication of content.medications)
        await tx.execute(
          sql`INSERT INTO prescriptions(visit_id,revision_id,patient_id,content)
VALUES(${id},${r.id},${v.patient_id}::uuid,${JSON.stringify(medication)}::jsonb)`,
        );
    await event(
      tx,
      actor,
      previous[0]?.state === "final" ? "correct" : "save_revision",
      "visit",
      id,
      { revision, state, reason },
    );
    return r;
  });
}
export async function fileVisit(actor: Staff, id: string, input: unknown) {
  const { expectedRevision } = z
    .object({ expectedRevision: z.number().int().positive() })
    .parse(input);

  return db.transaction(async (tx) => {
    const v = await found(
      await rows(
        sql`SELECT * FROM visits WHERE id=${idSchema.parse(id)}::uuid FOR UPDATE`,
        tx,
      ),
    );
    const c = await rows(
      sql`SELECT content,state FROM clinical_revisions WHERE visit_id=${id}::uuid AND
revision=${expectedRevision}`,
      tx,
    );
    if (
      v.status !== "ready" ||
      Number(v.revision) !== expectedRevision ||
      c[0]?.state !== "final" ||
      !(c[0].content as Row)?.diagnosis
```

```
    )
      throw new AppError(
        409,
        "The doctor must complete the diagnosis before this visit can be filed.",
      );
    await tx
      .update(visits)
      .set({ status: "filed", filedAt: new Date() })
      .where(eq(visits.id, id));
    if (v.appointment_id)
      await tx.execute(
        sql`UPDATE appointments SET status='completed',version=version+1 WHERE id=${v.appointment_id}::uuid`,
      );
    await event(tx, actor, "file", "visit", id);
    return { filed: true };
  });
}
export async function saveInvoice(actor: Staff, input: unknown) {
  const data = invoiceInput.parse(input);
  const totalCents = invoiceTotal(data.lines);
  return db.transaction(async (tx) => {
    await found(
      await tx
        .select({ id: patients.id })
        .from(patients)
        .where(eq(patients.id, data.patientId)),
    );
    const [i] = await tx
      .insert(invoices)
      .values({ ...data, totalCents, number: sql`DEFAULT` })
      .returning();
    await event(tx, actor, "create", "invoice", i.id);
    return i;
  });
}
export async function setInvoiceStatus(
  actor: Staff,
  id: string,
  input: unknown,
) {
  const { version, status, reason } = z
    .object({
      version: z.number().int(),
      status: z.enum(["issued", "void"]),
      reason: z.string().trim().max(500).default(""),
    })
    .parse(input);
  if (status === "void" && !reason)
    throw new AppError(400, "Enter a reason to void the invoice.");
  return db.transaction(async (tx) => {
    const [i] = await tx
      .update(invoices)
      .set({ status, reason, version: version + 1 })
      .where(
        and(
          eq(invoices.id, idSchema.parse(id)),
          eq(invoices.version, version),
          eq(invoices.status, status === "issued" ? "draft" : "issued"),
        ),
      )
      .returning();
    if (!i)
      throw new AppError(
        409,
        "This invoice changed or cannot enter that state.",
      );
    await event(tx, actor, status, "invoice", id, { reason });
    return i;
  });
}
export async function invoiceList(actor: Staff) {
  await event(db, actor, "view", "invoice_list");
  return rows(
    sql`SELECT i.*,p.first_name,p.last_name,p.mrn FROM invoices i JOIN patients p ON p.id=i.patient_id ORDER BY i.created_at DESC LIMIT 100`,
  );
```

```
}
export async function manageUser(actor: Staff, input: unknown, id?: string) {
  await requireRole(actor);
  if (!id) {
    const d = userInput.parse(input);
    const passwordHash = await hashPassword(d.password);
    return db.transaction(async (tx) => {
      const [u] = await tx
        .insert(users)
        .values({
          username: d.username,
          name: d.name,
          role: d.role,
          passwordHash,
        })
        .returning({ id: users.id });

      if (d.role === "doctor")
        await tx.execute(
          sql`INSERT INTO doctor_hours(doctor_id,weekday,start_min,end_min) SELECT ${u.id}::uuid,n,480,1020 FROM generate_series(1,5)n`,
        );
      await event(tx, actor, "create", "user", u.id);
      return u;
    });
  }
  const d = z
    .object({
      role: z.enum(["doctor", "front_desk"]),
      active: z.boolean(),
      version: z.number().int(),
      password: z.string().min(12).max(128).optional(),
    })
    .parse(input);
  const passwordHash = d.password ? await hashPassword(d.password) : undefined;
  return db.transaction(async (tx) => {
    const old = await found(
      await tx
        .select()
        .from(users)
        .where(eq(users.id, idSchema.parse(id))),
    );
    if (old.protected && process.env.DEMO_MODE === "true")
      throw new AppError(403, "The supplied demo accounts cannot be changed.");
    const [u] = await tx
      .update(users)
      .set({
        role: d.role,
        active: d.active,
        version: d.version + 1,
        ...(passwordHash ? { passwordHash } : {}),
      })
      .where(and(eq(users.id, id), eq(users.version, d.version)))
      .returning({ id: users.id });
    if (!u) throw new AppError(409, "This account changed. Reload the list.");
    if (d.role === "doctor")
      await tx.execute(
        sql`INSERT INTO doctor_hours(doctor_id,weekday,start_min,end_min) SELECT ${id}::uuid,n,480,1020 FROM generate_series(1,5)n ON CONFLICT DO NOTHING`,
      );
    await tx.delete(sessions).where(eq(sessions.userId, id));
    await event(tx, actor, "update", "user", id);
    return u;
  });
}
export async function changePassword(actor: Staff, input: unknown) {
  const { currentPassword, password } = z
    .object({
      currentPassword: z.string().max(128),
      password: z.string().min(12).max(128),
    })
    .parse(input);

  if (actor.protected && process.env.DEMO_MODE === "true")
    throw new AppError(403, "The supplied demo password cannot be changed.");
  const [u] = await db.select().from(users).where(eq(users.id, actor.id));
  const { verify } = await import("@node-rs/argon2");
  if (!(await verify(u.passwordHash, currentPassword)))
```

```
    throw new AppError(400, "The current password is incorrect.");
  const passwordHash = await hashPassword(password);
  await db.transaction(async (tx) => {
    await tx
      .update(users)
      .set({ passwordHash, version: u.version + 1 })
      .where(eq(users.id, actor.id));
    await tx.delete(sessions).where(eq(sessions.userId, actor.id));
    await event(tx, actor, "password_change", "user", actor.id);
  });
  return { logout: true };
}
export async function dashboard(actor: Staff) {
  return db.transaction(async (tx) => {
    await event(tx, actor, "view", "dashboard");
    const counts = await rows(
      sql`SELECT (SELECT count(*) FROM patients WHERE NOT archived) AS patients,(SELECT count(*) FROM appointments WHERE status IN('confirmed','arrived','in_consultation') AND (start_at AT TIME ZONE 'America/Jamaica')::date=(now() AT TIME ZONE 'America/Jamaica')::date) AS appointments,(SELECT count(*) FROM visits WHERE status='ready') AS ready,(SELECT count(*) FROM visits WHERE status='filed') AS filed`,
      tx,
    );
    const ready = await rows(
      sql`SELECT v.id,v.revision,v.status,v.patient_id,p.first_name,p.last_name,p.mrn FROM visits v JOIN patients p ON p.id=v.patient_id WHERE v.status='ready' ORDER BY v.visit_at LIMIT 10`,
      tx,
    );
    const reminders = await rows(
      sql`SELECT r.id,a.start_at,p.first_name,p.last_name,p.mrn FROM reminders r JOIN appointments a ON a.id=r.appointment_id JOIN patients p ON p.id=a.patient_id WHERE r.due_date >= (now() AT TIME ZONE 'America/Jamaica')::date AND a.status='confirmed' ORDER BY a.start_at LIMIT 10`,
      tx,
    );
    return { counts: counts[0], ready, reminders };
  });
}
export async function reportData(actor: Staff, type: string, id: string) {
  if (
    ![
      "schedule",
      "attendance",
      "invoice",
      "history",
      "prescription",
      "certificate",
      "summary",
      "visits",
    ].includes(type)
  )
    throw new AppError(404, "The report was not found.");

  if (["history", "prescription", "certificate", "summary"].includes(type))
    await requireRole(actor);
  if (type === "schedule")
    return { type, appointments: await schedule(actor, id) };
  if (type === "visits") {
    await event(db, actor, "export", "visit_counts");
    return {
      type,
      counts: await rows(
        sql`SELECT (visit_at AT TIME ZONE 'America/Jamaica')::date AS day,count(*) AS visits,count(*) FILTER(WHERE status='filed') AS filed FROM visits GROUP BY day ORDER BY day DESC LIMIT 30`,
      ),
    };
  }
  idSchema.parse(id);
  await event(db, actor, "export", type, id);
  if (type === "invoice")
    return {
      type,
      invoice: await found(
        await rows(
          sql`SELECT i.*,p.first_name,p.last_name,p.mrn,p.address FROM invoices i JOIN patients p ON p.id=i.patient_id WHERE i.id=${id}::uuid`,
        ),
      ),
    };
  if (type === "history") return { type, ...(await patientDetail(actor, id)) };
```

```
  const v = await found(
    await rows(
      sql`SELECT v.*,p.first_name,p.last_name,p.mrn,u.name AS doctor_name FROM visits v JOIN patients p ON p.id=v.patient_id JOIN users u ON u.id=v.doctor_id WHERE v.id=${id}::uuid`,
    ),
  );
  if (type === "attendance")
    return {
      type,
      visit: {
        id: v.id,
        mrn: v.mrn,
        first_name: v.first_name,
        last_name: v.last_name,
        visit_at: v.visit_at,
        doctor_name: v.doctor_name,
        status: v.status,
      },
    };
  const c = await found(
    await rows(
      sql`SELECT content,state,revision,created_at FROM clinical_revisions WHERE visit_id=${id}::uuid ORDER BY revision DESC LIMIT 1`,
    ),
    "No clinical entry is available.",
  );
  if (c.state !== "final")
    throw new AppError(

      400,
      "Complete the clinical entry before you print this document.",
    );
  return { type, visit: v, clinical: c };
}
```

src/lib/storage.ts

```
import { put, get } from "@vercel/blob";
import { mkdir, writeFile, readFile } from "node:fs/promises";
import path from "node:path";
import { createHash, randomUUID } from "node:crypto";
import { fileMime } from "./validation";
import { AppError } from "./auth";
function makeSamplePdf() {
  const stream =
    "BT /F1 18 Tf 50 720 Td (PracticeChart fictional lab report) Tj 0 -30 Td /F1 12 Tf (Sample only. No clinical findings.) Tj ET";
  const objects = [
    "<</Type/Catalog/Pages 2 0 R>>",
    "<</Type/Pages/Kids[3 0 R]/Count 1>>",
    "<</Type/Page/Parent 2 0 R/MediaBox[0 0 612 792]/Resources<</Font<</F1 4 0 R>>>>/Contents 5 0 R>>",
    "<</Type/Font/Subtype/Type1/BaseFont/Helvetica>>",
    `<</Length ${Buffer.byteLength(stream)}>>\nstream\n${stream}\nendstream`,
  ];
  let data = "%PDF-1.4\n";
  const offsets = [0];
  for (let i = 0; i < objects.length; i++) {
    offsets.push(Buffer.byteLength(data));
    data += `${i + 1} 0 obj\n${objects[i]}\nendobj\n`;
  }
  const xref = Buffer.byteLength(data);
  data += `xref\n0 6\n0000000000 65535 f \n${offsets
    .slice(1)
    .map((n) => String(n).padStart(10, "0") + " 00000 n ")
    .join("\n")}\ntrailer\n<</Size 6/Root 1 0 R>>\nstartxref\n${xref}\n%%EOF\n`;
  return Buffer.from(data);
}
export const samplePdf = makeSamplePdf();
export async function storeFile(bytes: Buffer, claimedMime: string) {
  if (bytes.length < 1 || bytes.length > 4194304)
    throw new AppError(400, "Use a file from 1 byte to 4 MiB.");
  const mime = fileMime(bytes);
  if (!mime || mime !== claimedMime)
    throw new AppError(
      400,
      "Use a PDF, PNG, or JPEG file with a matching file type.",
    );
  if (process.env.DEMO_MODE === "true" && !bytes.equals(samplePdf))
    throw new AppError(
      400,
      "The public demo accepts only the supplied sample report.",
    );
  const pathname = `practicechart/${process.env.VERCEL_ENV === "preview" ? "preview" : "records"}/${randomUUID()}`;
  if (process.env.STORAGE_DRIVER === "blob") {
    await put(pathname, bytes, {
      access: "private",
      contentType: mime,
      addRandomSuffix: false,
    });
  } else {
    const root = path.resolve(
      /* turbopackIgnore: true */ process.env.PRIVATE_STORAGE_DIR ||
        ".private/uploads",
    );
    await mkdir(path.dirname(path.join(root, pathname)), { recursive: true });
    await writeFile(path.join(root, pathname), bytes, { flag: "wx" });
  }
  return {
    pathname,
    mime,
    size: bytes.length,
    sha256: createHash("sha256").update(bytes).digest("hex"),
  };
}
export async function readPrivateFile(pathname: string) {
  if (!/^practicechart\/(records|preview)\/[a-f0-9-]{36}$/.test(pathname))
    throw new AppError(404, "The file was not found.");
  try {
    if (process.env.STORAGE_DRIVER === "blob") {
      const r = await get(pathname, { access: "private" });
```

```
      if (!r || r.statusCode !== 200) throw new Error("Missing");
      return Buffer.from(await new Response(r.stream).arrayBuffer());
    }
    return await readFile(
      /* turbopackIgnore: true */ path.join(
        path.resolve(
          /* turbopackIgnore: true */ process.env.PRIVATE_STORAGE_DIR ||
            ".private/uploads",
        ),
        pathname,
      ),
    );
  } catch {
    throw new AppError(
      404,
      "The stored file is not available. Contact the practice operator.",
    );
  }
}
```

src/lib/validation.ts

```
import { z } from "zod";
export const idSchema = z.string().uuid();
const text = (max = 200) =>
  z.string().trim().min(1, "This field is required.").max(max);
export const patientInput = z.object({
  firstName: text(80),
  lastName: text(80),
  dob: z
    .string()
    .regex(/^\d{4}-\d{2}-\d{2}$/)
    .refine((s) => {
      const d = new Date(s + "T12:00:00Z");
      return (
        !Number.isNaN(d.valueOf()) &&
        d.toISOString().slice(0, 10) === s &&
        s <=
          new Intl.DateTimeFormat("en-CA", {
            timeZone: "America/Jamaica",
          }).format(new Date())
      );
    }, "Use a valid date of birth that is not in the future."),
  phone: z
    .string()
    .trim()
    .min(7)
    .max(30)
    .regex(/^[+0-9 ()-]+$/, "Use a valid phone number."),
  address: text(300),
  gender: z
    .enum(["Not stated", "Female", "Male", "Other"])
    .default("Not stated"),
  emergencyContact: z.string().trim().max(200).default(""),
  insurance: z.string().trim().max(100).default(""),
  provisionalReason: z.string().trim().max(500).default(""),
});
export const appointmentInput = z
  .object({
    patientId: idSchema,
    doctorId: idSchema,
    startAt: z.string().datetime({ offset: true }),
    endAt: z.string().datetime({ offset: true }),
  })
  .refine(
    (v) =>
      Date.parse(v.endAt) - Date.parse(v.startAt) >= 60000 &&
      Date.parse(v.endAt) - Date.parse(v.startAt) <= 14400000,
    "Use an appointment length from 1 minute to 4 hours.",
  )
  .refine(
    (v) => Date.parse(v.startAt) > Date.now(),
    "Choose a future appointment.",
  );
export const medicationInput = z.object({
  medication: text(150),
  dosage: text(100),

  route: text(80),
  frequency: text(100),
  duration: text(100),
  quantity: text(80),
  instructions: z.string().trim().max(500).default(""),
});
export const clinicalInput = z
  .object({
    expectedRevision: z.number().int().min(0),
    state: z.enum(["draft", "final"]),
    symptoms: z.string().trim().max(8000).default(""),
    notes: z.string().trim().max(12000).default(""),
    diagnosis: z.string().trim().max(4000).default(""),
    treatment: z.string().trim().max(8000).default(""),
    allergies: z.string().trim().max(2000).default(""),
    vitals: z.string().trim().max(2000).default(""),
    labResults: z.string().trim().max(6000).default(""),
    medications: z.array(medicationInput).max(20).default([]),
    certificate: z.string().trim().max(6000).default(""),
```

```
    reason: z.string().trim().max(1000).default(""),
  })
  .refine(
    (v) => v.state !== "final" || Boolean(v.diagnosis),
    "Enter a diagnosis before you complete this visit.",
  );
export const invoiceInput = z.object({
  patientId: idSchema,
  lines: z
    .array(
      z.object({
        description: text(200),
        quantity: z.number().int().min(1).max(1000),
        unitCents: z.number().int().min(0).max(100000000),
      }),
    )
    .min(1)
    .max(30),
});
export function invoiceTotal(lines: z.infer<typeof invoiceInput>["lines"]) {
  const total = lines.reduce((n, l) => n + l.quantity * l.unitCents, 0);
  if (!Number.isSafeInteger(total) || total > 2000000000)
    throw new Error("Invoice total is too large.");
  return total;
}
export const userInput = z.object({
  username: z
    .string()
    .trim()
    .toLowerCase()
    .regex(/^[a-z0-9._-]{3,40}$/),
  name: text(100),
  role: z.enum(["doctor", "front_desk"]),
  password: z.string().min(12).max(128),
});
export type ClinicalContent = Omit<

  z.infer<typeof clinicalInput>,
  "expectedRevision" | "state" | "reason"
>;
export const emptyClinical: ClinicalContent = {
  symptoms: "",
  notes: "",
  diagnosis: "",
  treatment: "",
  allergies: "",
  vitals: "",
  labResults: "",
  medications: [],
  certificate: "",
};
export function fileMime(bytes: Buffer) {
  if (bytes.subarray(0, 5).toString() === "%PDF-") return "application/pdf";
  if (
    bytes.subarray(0, 8).equals(Buffer.from([137, 80, 78, 71, 13, 10, 26, 10]))
  )
    return "image/png";
  if (bytes[0] === 255 && bytes[1] === 216 && bytes[2] === 255)
    return "image/jpeg";
  return null;
}
```

src/workflows/daily.ts

```
export async function dailyPracticeTasks() {
  "use workflow";
  await createReminders();
  await reconcileFiles();
  return { complete: true };
}
export async function createReminders() {
  "use step";
  const { db } = await import("../lib/db");
  const { sql } = await import("drizzle-orm");
  await db.execute(
    sql`INSERT INTO reminders(appointment_id,due_date) SELECT id,(start_at AT TIME ZONE 'America/Jamaica')::date FROM appointments WHERE status='confirmed' AND start_at>now() AND start_at<now()+interval '7 days' ON CONFLICT DO NOTHING`,
  );
  await db.execute(sql`DELETE FROM sessions WHERE expires_at<now()`);
  await db.execute(sql`DELETE FROM login_attempts WHERE expires_at<now()`);
  return { complete: true };
}
export async function reconcileFiles() {
  "use step";
  const { db } = await import("../lib/db");
  const { sql } = await import("drizzle-orm");
  const { readPrivateFile } = await import("../lib/storage");
  const { createHash } = await import("node:crypto");
  const r = await db.execute(
    sql`SELECT pathname,sha256 FROM documents ORDER BY created_at DESC LIMIT 100`,
  );
  let missing = 0;
  for (const row of r.rows) {
    try {
      const b = await readPrivateFile(String(row.pathname));
      if (createHash("sha256").update(b).digest("hex") !== row.sha256)
        missing++;
    } catch {
      missing++;
    }
  }
  await db.execute(
    sql`INSERT INTO audit_events(action,entity,metadata) VALUES('file_reconciliation','maintenance',${JSON.stringify({ checked: r.rows.length, unavailable: missing })}::jsonb)`,
  );
  return { checked: r.rows.length, unavailable: missing };
}
```

tests/browser/workflows.spec.ts

```
import { test, expect, type Page } from "@playwright/test";
import AxeBuilder from "@axe-core/playwright";
import { mkdir, writeFile } from "node:fs/promises";
const base = process.env.E2E_BASE_URL || "http://localhost:3000";
async function login(page: Page, role = "doctor") {
  await page.goto("/login");
  await page.getByLabel("Username", { exact: true }).fill(role);
  await page
    .getByLabel("Password", { exact: true })
    .fill(role === "doctor" ? "DoctorDemo!2026" : "DeskDemo!2026");
  await page.getByRole("button", { name: "Log in", exact: true }).click();
  await page.waitForURL("**/app");
  await expect(
    page.getByRole("heading", { name: "Today at the practice" }),
  ).toBeVisible();
  await expect(
    page.getByRole("heading", { name: "Patient worklist" }),
  ).toBeVisible();
}
async function screenshot(page: Page, name: string) {
  await mkdir("docs/screenshots", { recursive: true });
  if (name === "patient-registration") {
    await page
      .getByRole("dialog")
      .screenshot({ path: `docs/screenshots/${name}.png` });
    return;
  }
  await page.screenshot({
    path: `docs/screenshots/${name}.png`,
    fullPage: true,
  });
}
function futureDay(offset = 30) {
  const d = new Date();
  d.setUTCDate(d.getUTCDate() + offset);
  while ([0, 6].includes(d.getUTCDay())) d.setUTCDate(d.getUTCDate() + 1);
  return d.toISOString().slice(0, 10);
}
test("both roles complete registration, booking, clinical correction, filing, and invoice workflows",
async ({
  page,
}) => {
  const suffix = Date.now().toString().slice(-7);
  const name = "Demo" + suffix;
  await login(page, "frontdesk");
  await screenshot(page, "front-desk-overview");
  await page
    .getByRole("navigation", { name: "Main navigation", exact: true })
    .getByRole("link", { name: "Patients", exact: true })
    .click();
  await page
    .getByRole("button", { name: "Register patient", exact: true })
    .click();
  await page.getByLabel("First name", { exact: true }).fill("Avery");
  await page.getByLabel("Last name", { exact: true }).fill(name);

  await page.getByLabel("Date of birth", { exact: true }).fill("1995-06-15");
  await page.getByLabel("Phone", { exact: true }).fill("876-555-0110");
  await page
    .getByLabel("Address", { exact: true })
    .fill("10 Fictional Lane, Demo Town");
  await screenshot(page, "patient-registration");
  await page
    .getByRole("dialog")
    .getByRole("button", { name: "Register patient", exact: true })
    .click();
  await expect(
    page.getByRole("heading", { name: "Avery " + name, exact: true }).first(),
  ).toBeVisible();
  const patientId = page.url().split("/").at(-1)!;
  await page
    .getByRole("button", { name: "Book appointment", exact: true })
    .first()
    .click();
  const booking = page.getByRole("dialog");
```

```
  await booking.getByLabel("Date", { exact: true }).fill(futureDay());
  await expect
    .poll(async () =>
      booking.getByLabel("Available start time").locator("option").count(),
    )
    .toBeGreaterThan(1);
  await booking.getByLabel("Available start time").selectOption({ index: 1 });
  await booking.getByRole("button", { name: "Review appointment" }).click();
  await expect(booking.getByText("Ready to confirm?")).toBeVisible();
  await screenshot(page, "appointment-confirmation");
  await booking.getByRole("button", { name: "Confirm appointment" }).click();
  await expect(booking).not.toBeVisible();
  await page
    .getByRole("button", { name: "Start walk-in visit", exact: true })
    .click();
  await page
    .getByRole("dialog")
    .getByRole("button", { name: "Start visit", exact: true })
    .click();
  await expect(page.getByRole("dialog")).not.toBeVisible();
  await page.getByRole("tab", { name: "Visits", exact: true }).click();
  await expect(
    page.getByRole("button", { name: "File visit", exact: true }),
  ).toHaveCount(0);
  await page.context().clearCookies();
  await login(page);
  await page.goto("/app/patients/" + patientId);
  await page.getByRole("tab", { name: "Clinical record", exact: true }).click();
  await page
    .getByLabel("Symptoms and concerns")
    .fill("Fictional concern for a course test.");
  await page
    .getByLabel("Consultation notes")
    .fill("Fictional consultation note.");
  await page
    .getByLabel("Diagnosis", { exact: true })
    .fill("Fictional diagnosis entered by the doctor.");
  await page.getByLabel("Treatment record").fill("Fictional treatment record.");
  await page.getByRole("button", { name: "Add medicine" }).click();
  for (const [label, value] of Object.entries({
    Medication: "Sample medicine",
    Dosage: "Sample dose",
    Route: "Sample route",
    Frequency: "Sample frequency",
    Duration: "Sample duration",
    Quantity: "Sample quantity",
  }))
    await page.getByLabel(label, { exact: true }).fill(value);
  await page
    .getByLabel("Medical certificate text (optional)")
    .fill("This is a fictional certificate for a course test.");
  await page.getByLabel("Entry state").selectOption("final");
  await page.getByRole("button", { name: "Save clinical entry" }).click();
  await expect(
    page.getByRole("button", { name: "Save final correction" }),
  ).toBeVisible();
  await screenshot(page, "doctor-clinical-record");
  await page
    .getByLabel("Diagnosis", { exact: true })
    .fill("Corrected fictional diagnosis.");
  await page
    .getByLabel("Correction reason", { exact: true })
    .fill("Corrected a typing error in the fictional entry.");
  await page.getByRole("button", { name: "Save final correction" }).click();
  await expect(page.getByText("Revision 2", { exact: true })).toBeVisible();
  await expect(page.getByText("Revision 1", { exact: true })).toBeVisible();
  await page.getByRole("tab", { name: "Visits", exact: true }).click();
  const reportPromise = page.waitForEvent("popup");
  await page.getByRole("link", { name: "Prescription", exact: true }).click();
  const report = await reportPromise;
  await expect(
    report.getByRole("heading", { name: "Doctor-entered prescription" }),
  ).toBeVisible();
  await expect(
    report.getByRole("cell", { name: "Sample medicine", exact: true }),
  ).toBeVisible();
  await screenshot(report, "prescription");
```

```
  await report.close();
  await page.context().clearCookies();
  await login(page, "frontdesk");
  await page.goto("/app/patients/" + patientId);
  await expect(
    page.getByRole("tab", { name: "Clinical record", exact: true }),
  ).toHaveCount(0);
  const response = await page.request.get("/api/patients/" + patientId);
  const safe = await response.json();
  expect(safe).not.toHaveProperty("clinical");
  expect(safe).not.toHaveProperty("documents");
  await page.getByRole("tab", { name: "Visits", exact: true }).click();
  await page.getByRole("button", { name: "File visit", exact: true }).click();
  await expect(page.getByText("filed", { exact: true }).first()).toBeVisible();

  await page.getByRole("tab", { name: "Overview", exact: true }).click();
  await page
    .getByRole("button", { name: "Create invoice", exact: true })
    .click();
  await page
    .getByLabel("Service 1", { exact: true })
    .fill("Fictional consultation");
  await page
    .getByLabel("Unit charge (JMD)", { exact: true })
    .first()
    .fill("100.25");
  await page.getByRole("button", { name: "Create invoice draft" }).click();
  await expect(page.getByRole("dialog")).not.toBeVisible();
  await page.goto("/app/invoices");
  const row = page.getByRole("row").filter({ hasText: "Avery " + name });
  await expect(row).toBeVisible();
  await row.getByRole("button", { name: "Issue", exact: true }).click();
  await expect(row.getByText("issued", { exact: true })).toBeVisible();
  await screenshot(page, "manual-invoices");
});
test("doctor retrieves a private report and front desk is denied direct clinical URLs", async ({
  page,
}) => {
  await login(page);
  await page.goto("/app/documents");
  await expect(
    page
      .getByRole("heading", { name: "Fictional lab report", exact: true })
      .first(),
  ).toBeVisible();
  const docs = await (await page.request.get("/api/documents")).json();
  const file = await page.request.get("/api/documents/" + docs[0].id);
  expect(file.status()).toBe(200);
  expect(file.headers()["cache-control"]).toContain("no-store");
  expect((await file.body()).subarray(0, 5).toString()).toBe("%PDF-");
  await screenshot(page, "private-diagnostic-documents");
  await page.context().clearCookies();
  await login(page, "frontdesk");
  for (const path of [
    "documents",
    "documents/" + docs[0].id,
    "users",
    "audit",
    "reports/history?id=" + docs[0].patientId,
  ])
    expect((await page.request.get("/api/" + path)).status()).toBe(403);
  const denied = await page.request.post(
    "/api/visits/" + docs[0].patientId + "/clinical",
    { headers: { origin: base }, data: {} },
  );
  expect(denied.status()).toBe(403);
  await page.context().clearCookies();
  expect(
    (await page.request.get("/api/documents/" + docs[0].id)).status(),
  ).toBe(401);

});
test("doctor uploads a sample report and retrieves the exact bytes", async ({
  page,
}) => {
  await login(page);
  const docs = await (await page.request.get("/api/documents")).json();
  const p = docs[0].patientId;
```

```
  await page.goto("/app/patients/" + p);
  await page.getByRole("tab", { name: "Documents", exact: true }).click();
  await page.getByRole("button", { name: "Add report", exact: true }).click();
  await page
    .getByLabel("Report title", { exact: true })
    .fill("Fictional browser upload");
  const sample = await page.request.get("/api/sample-report");
  const bytes = await sample.body();
  await page.getByLabel("Report file", { exact: true }).setInputFiles({
    name: "fictional-report.pdf",
    mimeType: "application/pdf",
    buffer: bytes,
  });
  await page.getByRole("button", { name: "Store private report" }).click();
  await expect(page.getByRole("dialog")).not.toBeVisible();
  await page.getByRole("tab", { name: "Documents", exact: true }).click();
  await expect(
    page
      .getByRole("heading", { name: "Fictional browser upload", exact: true })
      .first(),
  ).toBeVisible();
});
test("appointment can be rescheduled and cancelled with browser forms", async ({
  page,
}) => {
  await login(page, "frontdesk");
  const patientResponse = await page.request.post("/api/patients", {
    headers: { origin: base },
    data: {
      firstName: "Fictional",
      lastName: "ZZBooking" + Date.now(),
      dob: "1990-05-10",
      gender: "Not stated",
      phone: "876-555-0111",
      address: "Fictional Test Lane",
    },
  });
  expect(patientResponse.ok()).toBe(true);
  const { patient } = await patientResponse.json();
  await page.goto("/app/appointments");
  await expect(page.getByText("Loading practice records...")).toHaveCount(0);
  await page
    .getByRole("button", { name: "Book appointment", exact: true })
    .first()
    .click();
  let dialog = page.getByRole("dialog");
  await dialog.getByLabel("Find patient by name or MRN").fill(patient.mrn);
  await dialog.getByLabel("Patient", { exact: true }).selectOption(patient.id);

  await dialog.getByLabel("Date", { exact: true }).fill(futureDay(45));
  await expect
    .poll(async () =>
      dialog.getByLabel("Available start time").locator("option").count(),
    )
    .toBeGreaterThan(1);
  await dialog.getByLabel("Available start time").selectOption({ index: 1 });
  await dialog.getByRole("button", { name: "Review appointment" }).click();
  await expect(dialog.getByText(new RegExp(patient.mrn)).last()).toBeVisible();
  await dialog.getByRole("button", { name: "Confirm appointment" }).click();
  await expect(dialog).not.toBeVisible();
  await page.goto("/app/appointments");
  await page.getByLabel("Schedule date").fill(futureDay(45));
  await page
    .getByRole("row")
    .filter({ hasText: patient.mrn })
    .getByRole("button", { name: /Manage appointment/ })
    .click();
  await page.getByRole("button", { name: "Reschedule", exact: true }).click();
  dialog = page.getByRole("dialog");
  await dialog.getByLabel("Date", { exact: true }).fill(futureDay(46));
  await expect
    .poll(async () =>
      dialog.getByLabel("Available start time").locator("option").count(),
    )
    .toBeGreaterThan(1);
  await dialog.getByLabel("Available start time").selectOption({ index: 2 });
  await dialog.getByRole("button", { name: "Review appointment" }).click();
  await dialog.getByRole("button", { name: "Confirm appointment" }).click();
```

```
  await expect(dialog).not.toBeVisible();
  await page.getByLabel("Schedule date").fill(futureDay(46));
  await page
    .getByRole("row")
    .filter({ hasText: patient.mrn })
    .getByRole("button", { name: /Manage appointment/ })
    .click();
  await page
    .getByRole("button", { name: "Cancel appointment", exact: true })
    .click();
  await expect(
    page.getByText("cancelled", { exact: true }).first(),
  ).toBeVisible();
});
test("keyboard forms and desktop pages pass axe checks", async ({ page }) => {
  await page.goto("/login");
  await page.keyboard.press("Tab");
  await expect(page.locator(":focus")).toBeVisible();
  await page.getByLabel("Username", { exact: true }).fill("doctor");
  await page.getByLabel("Password", { exact: true }).fill("DoctorDemo!2026");
  await page.getByRole("button", { name: "Log in", exact: true }).click();
  await page.waitForURL("**/app");
  await expect(
    page.getByRole("heading", { name: "Today at the practice" }),
  ).toBeVisible();
  const results = [];

  for (const path of [
    "/app",
    "/app/patients",
    "/app/appointments",
    "/app/documents",
    "/app/invoices",
    "/app/reports",
    "/app/accounts",
    "/app/audit",
    "/app/settings",
  ]) {
    await page.goto(path);
    await expect(page.getByRole("heading", { level: 1 })).toBeVisible();
    await expect(page.getByText("Loading practice records...")).toHaveCount(0);
    const a = await new AxeBuilder({ page })
      .withTags(["wcag2a", "wcag2aa", "wcag21aa"])
      .analyze();
    results.push({
      path,
      violations: a.violations.map((v) => ({
        id: v.id,
        impact: v.impact,
        description: v.description,
        nodes: v.nodes.map((n) => n.target),
      })),
    });
  }
  await mkdir("docs/evidence", { recursive: true });
  await writeFile(
    "docs/evidence/axe-desktop.json",
    JSON.stringify(results, null, 2),
  );
  expect(results.flatMap((r) => r.violations)).toEqual([]);
});
test("phone layout, navigation, and reduced motion remain usable", async ({
  page,
}) => {
  await page.setViewportSize({ width: 390, height: 844 });
  await page.emulateMedia({ reducedMotion: "reduce" });
  await login(page, "frontdesk");
  await expect(
    page.getByRole("navigation", { name: "Mobile navigation" }),
  ).toBeVisible();
  await expect
    .poll(() =>
      page.evaluate(
        () => document.documentElement.scrollWidth <= window.innerWidth,
      ),
    )
    .toBe(true);
  await screenshot(page, "phone-overview");
```

```
  await page
    .getByRole("navigation", { name: "Mobile navigation" })
    .getByRole("link", { name: "Patients", exact: true })
    .click();

  await expect(
    page.getByRole("heading", { name: "Patient directory" }),
  ).toBeVisible();
  await page
    .getByRole("button", { name: "Register patient", exact: true })
    .click();
  await page.getByLabel("First name", { exact: true }).fill("Fictional");
  await expect(page.getByLabel("First name", { exact: true })).toHaveCSS(
    "font-size",
    "16px",
  );
  await expect(page.getByRole("dialog")).toBeVisible();
  await page.keyboard.press("Escape");
  await expect(page.getByRole("dialog")).not.toBeVisible();
  const a = await new AxeBuilder({ page })
    .withTags(["wcag2a", "wcag2aa", "wcag21aa"])
    .analyze();
  await writeFile(
    "docs/evidence/axe-phone.json",
    JSON.stringify(
      {
        viewport: { width: 390, height: 844 },
        emulated: true,
        realDeviceTested: false,
        violations: a.violations,
      },
      null,
      2,
    ),
  );
  expect(a.violations).toEqual([]);
  await screenshot(page, "phone-patient-directory");
});
test("rejects forged origins and does not expose a public upload folder", async ({
  page,
}) => {
  await login(page);
  const denied = await page.request.post("/api/patients", {
    headers: { origin: "https://untrusted.invalid" },
    data: {},
  });
  expect(denied.status()).toBe(403);
  expect((await page.request.get("/.private/uploads")).status()).toBe(404);
  expect((await page.request.get("/api/cron")).status()).toBe(401);
});

test("a delayed earlier patient response cannot replace the selected patient", async ({
  page,
}) => {
  await login(page);
  const patients = await (await page.request.get("/api/patients")).json();
  const a = patients[0],
    b = patients[1];
  const response = await (
    await page.request.get("/api/patients/" + a.id)

  ).text();
  await page.route("**/api/patients/" + a.id, async (route) => {
    await new Promise((resolve) => setTimeout(resolve, 1800));
    await route
      .fulfill({ status: 200, contentType: "application/json", body: response })
      .catch(() => {});
  });
  await page.goto("/app/patients");
  await page
    .getByRole("link", { name: new RegExp(a.firstName + " " + a.lastName) })
    .first()
    .click();
  await page
    .getByRole("navigation", { name: "Main navigation", exact: true })
    .getByRole("link", { name: "Patients", exact: true })
    .click();
  await page
```

```
    .getByRole("link", { name: new RegExp(b.firstName + " " + b.lastName) })
    .first()
    .click();
  await expect(
    page
      .getByRole("heading", {
        name: b.firstName + " " + b.lastName,
        exact: true,
      })
      .first(),
  ).toBeVisible();
  await page.waitForTimeout(2000);
  await expect(
    page
      .getByRole("heading", {
        name: b.firstName + " " + b.lastName,
        exact: true,
      })
      .first(),
  ).toBeVisible();
  await page.setViewportSize({ width: 768, height: 1000 });
  expect(
    await page.evaluate(
      () => document.documentElement.scrollWidth <= innerWidth,
    ),
  ).toBe(true);
});
```

tests/integration.test.ts

```
import { beforeAll, afterAll, describe, it, expect } from "vitest";
import { sql, eq } from "drizzle-orm";
import { db, pool } from "../src/lib/db";
import { users, sessions, type Staff } from "../src/lib/schema";
import * as s from "../src/lib/service";
import { login, getSession, digest } from "../src/lib/auth";
import { storeFile, readPrivateFile, samplePdf } from "../src/lib/storage";
import { emptyClinical } from "../src/lib/validation";
let doctor: Staff, desk: Staff, patientId: string, doctorId: string;
let minute = 0;
function booking(patient = patientId, start = 0, duration = 30) {
  const d = new Date("2032-01-05T08:00:00-05:00");
  d.setMinutes(d.getMinutes() + start);
  return {
    patientId: patient,
    doctorId,
    startAt: d.toISOString(),
    endAt: new Date(d.valueOf() + duration * 60000).toISOString(),
  };
}
beforeAll(async () => {
  doctor = (
    await db.select().from(users).where(eq(users.username, "doctor"))
  )[0];
  desk = (
    await db.select().from(users).where(eq(users.username, "frontdesk"))
  )[0];
  doctorId = doctor.id;
  patientId = (
    await s.savePatient(desk, {
      firstName: "Service",
      lastName: "Fictional",
      dob: "2000-01-01",
      phone: "876-555-0100",
      address: "Sample Road",
    })
  ).patient.id;
});
afterAll(async () => {
  await pool.end();
});
describe("additional independent service paths", () => {
  it("checks the full appointment length at closing and rejects excessive availability durations", async
() => {
    const slots = await s.availability(desk, doctorId, "2032-01-06", {
      duration: 120,
      patientId,
    });
    expect(slots.length).toBeGreaterThan(0);
    for (const slot of slots) {
      expect(
        Date.parse(String(slot.end_at)) - Date.parse(String(slot.start_at)),
      ).toBe(120 * 60000);
      expect(
        new Date(String(slot.start_at)).toISOString().slice(11, 16) <= "20:00",

      ).toBe(true);
    }
    await expect(
      s.availability(desk, doctorId, "2032-01-06", { duration: 241 }),
    ).rejects.toThrow();
  });
  it("excludes the current reschedule row and checks patient conflicts across doctors", async () => {
    const start = "2032-01-07T08:00:00-05:00";
    const a = await s.bookAppointment(desk, {
      patientId,
      doctorId,
      startAt: start,
      endAt: "2032-01-07T09:00:00-05:00",
    });
    const atStart = (rows: Record<string, unknown>[]) =>
      rows.some((r) => Date.parse(String(r.start_at)) === Date.parse(start));
    expect(
      atStart(
        await s.availability(desk, doctorId, "2032-01-07", {
```

```
          duration: 60,
          patientId,
        }),
      ),
    ).toBe(false);
    expect(
      atStart(
        await s.availability(desk, doctorId, "2032-01-07", {
          duration: 60,
          patientId,
          excludeId: a.id,
        }),
      ),
    ).toBe(true);
    const other = await s.manageUser(doctor, {
      username: "availability-doctor",
      name: "Fictional Availability Doctor",
      role: "doctor",
      password: "Availability!2026",
    });
    expect(
      atStart(
        await s.availability(desk, other.id, "2032-01-07", {
          duration: 60,
          patientId,
        }),
      ),
    ).toBe(false);
    expect(
      atStart(
        await s.availability(desk, other.id, "2032-01-07", { duration: 60 }),
      ),
    ).toBe(true);
    await s.manageUser(
      doctor,
      { version: 1, role: "doctor", active: false },
      other.id,
    );
  });
  it("reads a schedule, availability, dashboard, invoices, and administrative reports", async () => {
    expect(await s.schedule(desk, "2032-01-05")).toBeInstanceOf(Array);
    expect(await s.availability(desk, doctorId, "2032-01-06")).toBeInstanceOf(
      Array,
    );
    expect(await s.dashboard(desk)).toHaveProperty("counts");
    expect(await s.invoiceList(desk)).toBeInstanceOf(Array);
    expect(await s.reportData(desk, "schedule", "2032-01-05")).toHaveProperty(
      "appointments",
    );
    expect(await s.reportData(desk, "visits", "all")).toHaveProperty("counts");
  });
  it("rejects an invalid availability date and a missing patient", async () => {
    await expect(
      s.availability(desk, doctorId, "bad-date"),
    ).rejects.toMatchObject({ status: 400 });
    await expect(
      s.patientDetail(doctor, "11111111-1111-4111-8111-111111111111"),
    ).rejects.toMatchObject({ status: 404 });
  });
  it("changes a private account password and revokes its session", async () => {
    const u = await s.manageUser(doctor, {
      username: "password-test",
      name: "Fictional Password Tester",
      role: "front_desk",
      password: "OldPassword!2026",
    });
    const l = await login("password-test", "OldPassword!2026", "password-ip");
    const staff = (await db.select().from(users).where(eq(users.id, u.id)))[0];
    await expect(
      s.changePassword(staff, {
        currentPassword: "wrong",
        password: "NewPassword!2026",
      }),
    ).rejects.toMatchObject({ status: 400 });
    await s.changePassword(staff, {
      currentPassword: "OldPassword!2026",
      password: "NewPassword!2026",
```

```
    });
    expect(await getSession(l.token)).toBeNull();
    await expect(
      login("password-test", "NewPassword!2026", "password-ip"),
    ).resolves.toHaveProperty("token");
  });
  it("protects supplied public demo accounts", async () => {
    await db
      .update(users)
      .set({ protected: true })
      .where(eq(users.id, desk.id));
    process.env.DEMO_MODE = "true";
    try {
      await expect(
        s.manageUser(
          doctor,
          { version: desk.version, role: "front_desk", active: false },
          desk.id,
        ),
      ).rejects.toMatchObject({ status: 403 });
      await expect(
        s.changePassword(
          { ...desk, protected: true },
          { currentPassword: "DeskDemo!2026", password: "Replacement!2026" },
        ),
      ).rejects.toMatchObject({ status: 403 });
    } finally {
      process.env.DEMO_MODE = "false";
      await db
        .update(users)
        .set({ protected: false })
        .where(eq(users.id, desk.id));
    }
  });
  it("generates record numbers without truncating a long sequence value", async () => {
    await db.execute(
      sql`CREATE TEMP SEQUENCE long_record_sequence START 1000000`,
    );
    const result = await s.rows(
      sql`SELECT next_record_number('PC-','long_record_sequence') AS mrn`,
    );
    expect(result[0].mrn).toBe("PC-1000000");
  });
  it("returns clinical outputs only after a final entry", async () => {
    const v = await s.startVisit(doctor, { patientId, doctorId });
    await expect(s.reportData(doctor, "summary", v.id)).rejects.toMatchObject({
      status: 404,
    });
    await s.saveClinical(doctor, v.id, {
      ...emptyClinical,
      expectedRevision: 0,
      state: "draft",
    });
    await expect(s.reportData(doctor, "summary", v.id)).rejects.toMatchObject({
      status: 400,
    });
    await s.saveClinical(doctor, v.id, {
      ...emptyClinical,
      expectedRevision: 1,
      state: "final",
      diagnosis: "Fictional diagnosis",
    });
    for (const kind of ["summary", "prescription", "certificate"])
      expect(await s.reportData(doctor, kind, v.id)).toHaveProperty("clinical");
    expect(await s.reportData(desk, "attendance", v.id)).not.toHaveProperty(
      "clinical",
    );
    expect(await s.reportData(doctor, "history", patientId)).toHaveProperty(
      "clinical",
    );
    );
    await expect(
      s.reportData(doctor, "unknown", patientId),
    ).rejects.toMatchObject({ status: 404 });
  });
});
describe("authentication and role enforcement", () => {
```

```
  it("rejects wrong and missing accounts with the same error", async () => {
    for (const username of ["doctor", "missing-account"])
      await expect(login(username, "wrong", "test-ip")).rejects.toMatchObject({
        status: 401,
        message: "The username or password is incorrect.",
      });
  });
  it("logs in and stores only a token hash", async () => {
    const result = await login("doctor", "DoctorDemo!2026", "login-test");
    expect(await getSession(result.token)).toMatchObject({ role: "doctor" });
    const rows = await db
      .select()
      .from(sessions)
      .where(eq(sessions.id, result.token));
    expect(rows).toHaveLength(0);
    await db.delete(sessions).where(eq(sessions.id, digest(result.token)));
    expect(await getSession(result.token)).toBeNull();
  });
  it("rejects an expired session", async () => {
    await db.insert(sessions).values({
      id: digest("expired"),
      userId: doctor.id,
      expiresAt: new Date(0),
    });
    expect(await getSession("expired")).toBeNull();
  });
  it("enforces the login attempt limit", async () => {
    for (let i = 0; i < 8; i++)
      await expect(login("limited", "wrong", "limit-ip")).rejects.toMatchObject(
        { status: 401 },
      );
    await expect(login("limited", "wrong", "limit-ip")).rejects.toMatchObject({
      status: 429,
    });
  });
  it("denies front desk account creation", async () => {
    await expect(s.manageUser(desk, {})).rejects.toMatchObject({ status: 403 });
  });
  it("denies front desk clinical changes and clinical report export", async () => {
    await expect(s.saveClinical(desk, patientId, {})).rejects.toMatchObject({
      status: 403,
    });
    await expect(
      s.reportData(desk, "history", patientId),
    ).rejects.toMatchObject({ status: 403 });
  });
  it("does not include clinical fields in front desk patient responses", async () => {
    const p = await s.patientDetail(desk, patientId);

    expect(p).not.toHaveProperty("clinical");
    expect(p).not.toHaveProperty("documents");
  });
  it("prevents all doctors from being disabled", async () => {
    await expect(
      db.execute(
        sql`UPDATE users SET active=false WHERE role='doctor' AND active`,
      ),
    ).rejects.toThrow();
    expect(
      (await db.select().from(users).where(eq(users.id, doctorId)))[0].active,
    ).toBe(true);
  });
  it("creates an account, changes access, and revokes its session", async () => {
    const u = await s.manageUser(doctor, {
      username: "test-desk",
      name: "Fictional Tester",
      role: "front_desk",
      password: "TestPassword!2026",
    });
    const l = await login("test-desk", "TestPassword!2026", "new-user");
    await s.manageUser(
      doctor,
      { version: 1, role: "front_desk", active: false },
      u.id,
    );
    expect(await getSession(l.token)).toBeNull();
    await expect(
      login("test-desk", "TestPassword!2026", "new-user"),
```

```
    ).rejects.toMatchObject({ status: 401 });
  });
});
describe("patient changes and appointment concurrency", () => {
  it("preserves distinct MRNs and warns on possible duplicate people", async () => {
    const input = {
      firstName: "Same",
      lastName: "Fictional",
      dob: "2000-01-01",
      phone: "876-555-0100",
      address: "Sample Road",
    };
    const [a, b] = await Promise.all([
      s.savePatient(desk, input),
      s.savePatient(desk, input),
    ]);
    expect(a.patient.mrn).not.toBe(b.patient.mrn);
    const c = await s.savePatient(desk, input);
    expect(c.duplicates!.length).toBeGreaterThan(0);
  });
  it("permits one of two stale demographic changes", async () => {
    const p = await s.patientDetail(desk, patientId);
    const input = {
      ...p.patient,
      firstName: "Changed",
      version: p.patient.version,

    };
    const r = await Promise.allSettled([
      s.savePatient(desk, input, patientId),
      s.savePatient(desk, input, patientId),
    ]);
    expect(r.filter((v) => v.status === "fulfilled")).toHaveLength(1);
  });
  it("permits adjacent appointments and rejects an overlap", async () => {
    await s.bookAppointment(desk, booking(patientId, 0));
    await s.bookAppointment(desk, booking(patientId, 30));
    await expect(
      s.bookAppointment(desk, booking(patientId, 15)),
    ).rejects.toThrow();
  });
  it("allows exactly one simultaneous booking for one doctor", async () => {
    const p = (await s.patientList(desk)).find((p) => p.id !== patientId)!;
    const r = await Promise.allSettled([
      s.bookAppointment(desk, booking(patientId, 90)),
      s.bookAppointment(desk, booking(String(p.id), 90)),
    ]);
    expect(r.filter((v) => v.status === "fulfilled")).toHaveLength(1);
  });
  it("rejects booking outside hours and at the end boundary", async () => {
    await expect(
      s.bookAppointment(desk, booking(patientId, -1, 1)),
    ).rejects.toThrow();
    await expect(
      s.bookAppointment(desk, booking(patientId, 540, 30)),
    ).rejects.toThrow();
    await expect(
      s.bookAppointment(desk, booking(patientId, 510, 30)),
    ).resolves.toHaveProperty("id");
  });
  it("keeps the original booking when a reschedule conflicts", async () => {
    const a = await s.bookAppointment(desk, booking(patientId, 180));
    await expect(
      s.bookAppointment(
        desk,
        { ...booking(patientId, 0), version: a.version },
        a.id,
      ),
    ).rejects.toThrow();
    const r = await s.rows(
      sql`SELECT start_at FROM appointments WHERE id=${a.id}`,
    );
    expect(new Date(String(r[0].start_at)).toISOString()).toBe(
      a.startAt.toISOString(),
    );
  });
  it("releases a cancelled slot and retains its history", async () => {
    const a = await s.bookAppointment(desk, booking(patientId, 240));
```

```
    await s.appointmentStatus(desk, a.id, {
      status: "cancelled",
      version: a.version,
    });

    await expect(
      s.bookAppointment(desk, booking(patientId, 240)),
    ).resolves.toHaveProperty("id");
    expect(
      (await s.rows(sql`SELECT status FROM appointments WHERE id=${a.id}`))[0]
        .status,
    ).toBe("cancelled");
  });
  it("rejects a booking during a blocked interval", async () => {
    await db.execute(
      sql`INSERT INTO schedule_blocks(doctor_id,start_at,end_at)
VALUES(${doctorId},'2032-01-05T15:00:00-05:00','2032-01-05T15:30:00-05:00')`,
    );
    await expect(
      s.bookAppointment(desk, booking(patientId, 420)),
    ).rejects.toThrow();
  });
  it("does not return an archived patient as bookable", async () => {
    const p = (
      await s.savePatient(doctor, {
        firstName: "Archived",
        lastName: "Fictional",
        dob: "2000-01-01",
        phone: "876-555-0100",
        address: "Sample Road",
      })
    ).patient;
    await s.archivePatient(doctor, p.id, {
      version: p.version,
      archived: true,
    });
    await expect(
      s.bookAppointment(desk, booking(p.id, 330)),
    ).rejects.toMatchObject({ status: 400 });
  });
});
describe("clinical corrections, filing, documents, and invoices", () => {
  it("preserves revisions, requires a final diagnosis and correction reason, and prevents overwriting",
async () => {
    const v = await s.startVisit(desk, { patientId, doctorId });
    await expect(
      s.fileVisit(desk, v.id, { expectedRevision: 1 }),
    ).rejects.toThrow();
    await s.saveClinical(doctor, v.id, {
      ...emptyClinical,
      expectedRevision: 0,
      state: "draft",
      notes: "First fictional draft.",
    });
    await expect(
      s.saveClinical(doctor, v.id, {
        ...emptyClinical,
        expectedRevision: 0,
        state: "draft",
      }),

    ).rejects.toMatchObject({ status: 409 });
    await s.saveClinical(doctor, v.id, {
      ...emptyClinical,
      expectedRevision: 1,
      state: "final",
      diagnosis: "Fictional diagnosis",
      treatment: "Doctor-entered sample",
    });
    await expect(
      s.saveClinical(doctor, v.id, {
        ...emptyClinical,
        expectedRevision: 2,
        state: "final",
        diagnosis: "Corrected",
      }),
    ).rejects.toMatchObject({ status: 400 });
    await s.saveClinical(doctor, v.id, {
```

```
      ...emptyClinical,
      expectedRevision: 2,
      state: "final",
      diagnosis: "Corrected fictional diagnosis",
      reason: "Corrected a typing error.",
    });
    const entries = await s.rows(
      sql`SELECT * FROM clinical_revisions WHERE visit_id=${v.id}`,
    );
    expect(entries).toHaveLength(3);
    await expect(
      db.execute(
        sql`UPDATE clinical_revisions SET reason='overwritten' WHERE visit_id=${v.id}`,
      ),
    ).rejects.toThrow();
    await s.fileVisit(desk, v.id, { expectedRevision: 3 });
    expect(
      (await s.rows(sql`SELECT status FROM visits WHERE id=${v.id}`))[0].status,
    ).toBe("filed");
  });
  it("writes only medication details supplied by the doctor", async () => {
    const v = await s.startVisit(doctor, { patientId, doctorId });
    const medication = {
      medication: "Fictional medicine",
      dosage: "Sample dose",
      route: "Sample route",
      frequency: "Sample frequency",
      duration: "Sample duration",
      quantity: "Sample quantity",
      instructions: "For demonstration only",
    };
    await s.saveClinical(doctor, v.id, {
      ...emptyClinical,
      expectedRevision: 0,
      state: "final",
      diagnosis: "Sample",
      medications: [medication],
    });

    const rows = await s.rows(
      sql`SELECT content FROM prescriptions WHERE visit_id=${v.id}`,
    );
    expect(rows[0].content).toEqual(medication);
  });
  it("keeps linked provisional records and their MRNs", async () => {
    const p = (
      await s.savePatient(desk, {
        firstName: "Provisional",
        lastName: "Fictional",
        dob: "2000-01-01",
        phone: "876-555-0100",
        address: "Sample Road",
        provisionalReason: "Primary record needs review",
      })
    ).patient;
    await s.reconcilePatient(doctor, p.id, {
      targetId: patientId,
      reason: "Identity checked in fictional test",
    });
    const result = await s.patientDetail(doctor, p.id);
    expect(result.patient.mrn).toBe(p.mrn);
    expect(result.patient.mergedInto).toBe(patientId);
  });
  it("checks file signatures, MIME type, size boundaries, private paths, and missing files", async () => {
    await expect(
      storeFile(Buffer.alloc(0), "application/pdf"),
    ).rejects.toMatchObject({ status: 400 });
    await expect(
      storeFile(Buffer.alloc(4194305), "application/pdf"),
    ).rejects.toMatchObject({ status: 400 });
    await expect(storeFile(samplePdf, "image/png")).rejects.toMatchObject({
      status: 400,
    });
    const data = await storeFile(samplePdf, "application/pdf");
    expect(await readPrivateFile(data.pathname)).toEqual(samplePdf);
    await expect(readPrivateFile("../.env")).rejects.toMatchObject({
      status: 404,
    });
```

```
    await expect(
      readPrivateFile(
        "practicechart/records/11111111-1111-4111-8111-111111111111",
      ),
    ).rejects.toMatchObject({ status: 404 });
    const max = Buffer.alloc(4194304);
    samplePdf.copy(max);
    await expect(storeFile(max, "application/pdf")).resolves.toHaveProperty(
      "size",
      4194304,
    );
  });
  it("restricts public demo files to the supplied sample", async () => {
    process.env.DEMO_MODE = "true";
    try {
      await expect(
        storeFile(Buffer.from("%PDF-changed"), "application/pdf"),
      ).rejects.toMatchObject({ status: 400 });
      await expect(
        storeFile(samplePdf, "application/pdf"),
      ).resolves.toHaveProperty("pathname");
    } finally {
      process.env.DEMO_MODE = "false";
    }
  });
  it("issues and voids invoices without recording a payment", async () => {
    const i = await s.saveInvoice(desk, {
      patientId,
      lines: [{ description: "Sample service", quantity: 2, unitCents: 10001 }],
    });
    expect(i.totalCents).toBe(20002);
    await s.setInvoiceStatus(desk, i.id, { version: 1, status: "issued" });
    await expect(
      s.setInvoiceStatus(desk, i.id, { version: 2, status: "void" }),
    ).rejects.toThrow();
    const v = await s.setInvoiceStatus(desk, i.id, {
      version: 2,
      status: "void",
      reason: "Duplicate service line",
    });
    expect(v.status).toBe("void");
  });
  it("keeps audit history append only", async () => {
    await expect(db.execute(sql`DELETE FROM audit_events`)).rejects.toThrow();
    expect(
      (await s.rows(sql`SELECT count(*) FROM audit_events`))[0].count,
    ).not.toBe("0");
  });
});
```

tests/recovery.test.ts

```
import {describe,it,expect,beforeAll}from 'vitest';
import {generateKeyPairSync}from 'node:crypto';
import {encryptBundle,decryptBundle}from '../scripts/recovery-lib';
let publicKey:string,privateKey:string;
beforeAll(()=>{({publicKey,privateKey}=generateKeyPairSync('rsa',{modulusLength:2048,publicKeyEncoding:{type:'spki',format:'pem'},privateKeyEncoding:{type:'pkcs8',format:'pem'}}));});
describe('authenticated recovery envelope failure paths',()=>{
 it('decrypts exact original bytes',()=>{const b=Buffer.from('Fictional database dump and files.');expect(decryptBundle(encryptBundle(b,publicKey),privateKey)).toEqual(b);});
 it('rejects modified ciphertext before restore',()=>{const b=encryptBundle(Buffer.from('Sample'),publicKey);b[b.length-1]^=1;expect(()=>decryptBundle(b,privateKey)).toThrow();});
 it('rejects unknown backup formats',()=>expect(()=>decryptBundle(Buffer.from('invalid backup'),privateKey)).toThrow());
 it('requires the matching recovery key',()=>{const other=generateKeyPairSync('rsa',{modulusLength:2048,publicKeyEncoding:{type:'spki',format:'pem'},privateKeyEncoding:{type:'pkcs8',format:'pem'}});expect(()=>decryptBundle(encryptBundle(Buffer.from('Sample'),publicKey),other.privateKey)).toThrow();});
});
```

tests/validation.test.ts

```
import { describe, it, expect } from "vitest";
import {
  patientInput,
  appointmentInput,
  clinicalInput,
  invoiceInput,
  invoiceTotal,
  fileMime,
  userInput,
} from "../src/lib/validation";
const patient = {
  firstName: "Test",
  lastName: "Fictional",
  dob: "2000-02-29",
  phone: "876-555-0100",
  address: "1 Sample Road",
};
describe("patient partitions and date boundaries", () => {
  it("accepts a valid leap day", () =>
    expect(patientInput.safeParse(patient).success).toBe(true));
  it.each(["2001-02-29", "2025-02-30", "2100-01-01", "not-a-date", ""])(
    "rejects invalid or future date %s",
    (dob) =>
      expect(patientInput.safeParse({ ...patient, dob }).success).toBe(false),
  );
  it("accepts today as a date of birth", () =>
    expect(
      patientInput.safeParse({
        ...patient,
        dob: new Intl.DateTimeFormat("en-CA", {
          timeZone: "America/Jamaica",
        }).format(new Date()),
      }).success,
    ).toBe(true));
  it.each([0, 81])("rejects name length %i", (n) =>
    expect(
      patientInput.safeParse({ ...patient, firstName: "a".repeat(n) }).success,
    ).toBe(false),
  );
  it.each([1, 80])("accepts name length %i", (n) =>
    expect(
      patientInput.safeParse({ ...patient, firstName: "a".repeat(n) }).success,
    ).toBe(true),
  );
  it.each(["123456", "letterphone", ""])("rejects invalid phone %s", (phone) =>
    expect(patientInput.safeParse({ ...patient, phone }).success).toBe(false),
  );
});
describe("appointment boundaries", () => {
  const base = {
    patientId: "11111111-1111-4111-8111-111111111111",
    doctorId: "22222222-2222-4222-8222-222222222222",
    startAt: "2035-01-01T08:00:00-05:00",
  };
  it("accepts a valid interval", () =>

    expect(
      appointmentInput.safeParse({
        ...base,
        endAt: "2035-01-01T08:30:00-05:00",
      }).success,
    ).toBe(true));
  it.each([
    "2035-01-01T08:00:00-05:00",
    "2035-01-01T07:59:00-05:00",
    "2035-01-01T12:01:00-05:00",
  ])("rejects zero, negative, or excessive interval %s", (endAt) =>
    expect(appointmentInput.safeParse({ ...base, endAt }).success).toBe(false),
  );
  it("accepts the maximum four hour interval", () =>
    expect(
      appointmentInput.safeParse({
        ...base,
        endAt: "2035-01-01T12:00:00-05:00",
      }).success,
```

```
    ).toBe(true));
  it("rejects past appointments", () =>
    expect(
      appointmentInput.safeParse({
        ...base,
        startAt: "2000-01-01T08:00:00Z",
        endAt: "2000-01-01T08:30:00Z",
      }).success,
    ).toBe(false));
});
describe("clinical and invoice rules", () => {
  it("permits incomplete drafts", () =>
    expect(
      clinicalInput.safeParse({ expectedRevision: 0, state: "draft" }).success,
    ).toBe(true));
  it("requires diagnosis for final entries", () =>
    expect(
      clinicalInput.safeParse({ expectedRevision: 0, state: "final" }).success,
    ).toBe(false));
  it("does not fill medication details", () =>
    expect(
      clinicalInput.safeParse({
        expectedRevision: 0,
        state: "draft",
        medications: [{ medication: "Sample" }],
      }).success,
    ).toBe(false));
  it("calculates exact integer cents", () =>
    expect(
      invoiceTotal([{ description: "Sample", quantity: 3, unitCents: 101 }]),
    ).toBe(303));
  it.each([0, -1, 1.5])("rejects invalid invoice quantity %s", (quantity) =>
    expect(
      invoiceInput.safeParse({
        patientId: "11111111-1111-4111-8111-111111111111",
        lines: [{ description: "Sample", quantity, unitCents: 100 }],
      }).success,
    ).toBe(false),
  );
  it("rejects empty invoices", () =>
    expect(
      invoiceInput.safeParse({
        patientId: "11111111-1111-4111-8111-111111111111",
        lines: [],
      }).success,
    ).toBe(false));
  it("rejects a total above the database bound", () =>
    expect(() =>
      invoiceTotal([
        { description: "Sample", quantity: 1000, unitCents: 100000000 },
      ]),
    ).toThrow());
  it.each(["front_desk", "doctor"])("accepts the defined role %s", (role) =>
    expect(
      userInput.safeParse({
        username: "testuser",
        name: "Test User",
        role,
        password: "abcdefghijkl",
      }).success,
    ).toBe(true),
  );
  it("rejects an injected role", () =>
    expect(
      userInput.safeParse({
        username: "testuser",
        name: "Test User",
        role: "admin",
        password: "abcdefghijkl",
      }).success,
    ).toBe(false));
  it("rejects a short password", () =>
    expect(
      userInput.safeParse({
        username: "testuser",
        name: "Test User",
        role: "doctor",
```

```
        password: "abcdefghijk",
      }).success,
    ).toBe(false));
});
describe("file signature partitions", () => {
  it("recognises PDF", () =>
    expect(fileMime(Buffer.from("%PDF-1.4"))).toBe("application/pdf"));
  it("recognises PNG", () =>
    expect(fileMime(Buffer.from([137, 80, 78, 71, 13, 10, 26, 10]))).toBe(
      "image/png",
    ));
  it("recognises JPEG", () =>
    expect(fileMime(Buffer.from([255, 216, 255]))).toBe("image/jpeg"));
  it.each(["", "<script>alert(1)</script>", "not a PDF"])(
    "rejects an unapproved signature",
    (s) => expect(fileMime(Buffer.from(s))).toBeNull(),
  );
});
```

package.json

```
{
  "name": "practicechart-ehr",
  "version": "1.0.0",
  "private": true,
  "type": "module",
  "scripts": {
    "dev": "next dev --hostname 0.0.0.0",
    "build": "next build",
    "start": "next start --hostname 0.0.0.0",
    "typecheck": "tsc --noEmit",
    "test": "tsx scripts/test-run.ts",
    "test:e2e": "playwright test",
    "db:migrate": "tsx scripts/migrate.ts",
    "db:seed": "tsx scripts/seed.ts",
    "backup": "tsx scripts/backup.ts",
    "restore:test": "tsx scripts/restore-test.ts",
    "setup": "tsx scripts/local-setup.ts"
  },
  "overrides": {
    "zod": "$zod",
    "undici": "^7.30.1",
    "minimatch": "^11.0.1"
  },
  "dependencies": {
    "@node-rs/argon2": "2.2.2",
    "@vercel/blob": "2.8.2",
    "drizzle-orm": "0.45.4",
    "gsap": "3.15.0",
    "lucide-react": "1.65.0",
    "next": "16.4.3",
    "pg": "8.23.1",
    "react": "19.3.1",
    "react-dom": "19.3.1",
    "workflow": "5.2.3",
    "zod": "4.6.7"
  },
  "devDependencies": {
    "@axe-core/playwright": "4.14.0",
    "@playwright/test": "1.64.1",
    "@tailwindcss/postcss": "4.3.3",
    "@types/libsodium-wrappers": "0.7.14",
    "@types/node": "26.6.6",
    "@types/pg": "8.23.1",
    "@types/react": "19.3.0",
    "@types/react-dom": "19.3.0",
    "@vitest/coverage-v8": "5.0.5",
    "@viz-js/viz": "3.31.0",
    "dotenv": "18.0.9",
    "libsodium-wrappers": "0.8.4",
    "prettier": "3.9.10",
    "tailwindcss": "4.3.3",
    "tsx": "4.23.15",
    "typescript": "7.0.2",
    "typescript-compiler": "npm:typescript@5.9.3",
    "vitest": "5.0.5"
  }
}
```

tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2022",
    "lib": ["dom", "dom.iterable", "esnext"],
    "allowJs": true,
    "skipLibCheck": true,
    "strict": true,
    "noEmit": true,
    "esModuleInterop": true,
    "module": "esnext",
    "moduleResolution": "bundler",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "jsx": "react-jsx",
    "incremental": true,
    "plugins": [{ "name": "next" }],
    "paths": { "@/*": ["./src/*"] }
  },
  "include": [
    "next-env.d.ts",
    "**/*.ts",
    "**/*.tsx",
    ".next/types/**/*.ts",
    ".next/dev/types/**/*.ts"
  ],
  "exclude": ["node_modules"]
}
```

next.config.ts

```
import { withWorkflow } from "workflow/next";
import type { NextConfig } from "next";
const config: NextConfig = {
  poweredByHeader: false,
  serverExternalPackages: ["@node-rs/argon2", "pg"],
  outputFileTracingExcludes: {
    "/*": [
      "./.private/**/*",
      "./backups/**/*",
      "./docs/**/*",
      "./.env*",
      "./.git/**/*",
    ],
  },
  async headers() {
    return [
      {
        source: "/(.*)",
        headers: [
          { key: "X-Content-Type-Options", value: "nosniff" },
          { key: "Referrer-Policy", value: "same-origin" },
          { key: "X-Frame-Options", value: "DENY" },
          {
            key: "Permissions-Policy",
            value: "camera=(), microphone=(), geolocation=()",
          },
        ],
      },
    ];
  },
};
export default withWorkflow(config);
```

playwright.config.ts

```
import { defineConfig } from "@playwright/test";
export default defineConfig({
  testDir: "./tests/browser",
  fullyParallel: false,
  workers: 1,
  timeout: 60000,
  expect: { timeout: 12000 },
  reporter: [
    ["list"],
    ["json", { outputFile: "docs/evidence/playwright-results.json" }],
    ["html", { open: "never" }],
  ],
  use: {
    baseURL: process.env.E2E_BASE_URL || "http://localhost:3000",
    viewport: { width: 1440, height: 1000 },
    trace: "retain-on-failure",
    screenshot: "only-on-failure",
  },
  webServer: process.env.E2E_BASE_URL
    ? undefined
    : {
        command: process.env.CI ? "npm run start" : "npm run dev",
        url: "http://localhost:3000/api/health",
        reuseExistingServer: !process.env.CI,
        timeout: 120000,
      },
});
```

vitest.config.ts

```
import { defineConfig } from "vitest/config";
export default defineConfig({
  test: {
    include: ["tests/*.test.ts"],
    fileParallelism: false,
    maxWorkers: 1,
    testTimeout: 20000,
    hookTimeout: 30000,
    coverage: {
      provider: "v8",
      include: [
        "src/lib/validation.ts",
        "src/lib/auth.ts",
        "src/lib/service.ts",
        "src/lib/storage.ts",
      ],
      reporter: ["text", "json-summary", "html"],
    },
    reporters: [
      "default",
      ["json", { outputFile: "docs/evidence/vitest-results.json" }],
    ],
  },
});
```

docker-compose.yml

```
services:
  postgres:
    image: postgres:18
    environment:
      POSTGRES_USER: practicechart
      POSTGRES_PASSWORD: local-owner-password
      POSTGRES_DB: practicechart_ehr
    ports:
      - "127.0.0.1:5438:5432"
    volumes:
      - practicechart-data:/var/lib/postgresql
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U practicechart -d practicechart_ehr"]
      interval: 5s
      timeout: 5s
      retries: 15
volumes:
  practicechart-data:
```

vercel.json

```
{"framework":"nextjs","regions":["iad1"],"crons":[{"path":"/api/cron","schedule":"0 5 * * *"}]}
```